

**UNIVERSIDAD AUTÓNOMA**

**GABRIEL RENÉ MORENO**

**FACULTAD DE INGENIERÍA EN CIENCIAS  
DE LA COMPUTACIÓN Y TELECOMUNICACIONES**



**PRIMER PARCIAL**

**Plataforma Colaborativa de Modelado UML con Generación  
Inteligente de Backend Spring Boot y Aplicación Móvil Flutter**

|                   |                                      |
|-------------------|--------------------------------------|
| <b>ESTUDIANTE</b> | Sanchez Paniagua Felix Santiago      |
| <b>REGISTRO</b>   | 222115203                            |
| <b>MATERIA</b>    | INGENIERÍA DE SOFTWARE I - SC        |
| <b>DOCENTE</b>    | ING. ROLANDO ANTONIO MARTINEZ CANEDO |

**UNIVERSIDAD AUTÓNOMA**

**GABRIEL RENÉ MORENO**

**FACULTAD DE INGENIERÍA EN CIENCIAS  
DE LA COMPUTACIÓN Y TELECOMUNICACIONES**



**PRIMER PARCIAL**

**Plataforma Colaborativa de Modelado UML con Generación  
Inteligente de Backend Spring Boot y Aplicación Móvil Flutter**

|                   |                                      |
|-------------------|--------------------------------------|
| <b>ESTUDIANTE</b> | Sanchez Paniagua Felix Santiago      |
| <b>REGISTRO</b>   | 222115203                            |
| <b>MATERIA</b>    | INGENIERÍA DE SOFTWARE I - SC        |
| <b>DOCENTE</b>    | ING. ROLANDO ANTONIO MARTINEZ CANEDO |

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>1 FUNDAMENTACIÓN TEÓRICA</b>                             | <b>6</b>  |
| 1.1 HERRAMIENTAS CASE COMO SOPORTE DEL PROCESO              | 6         |
| 1.2 CLASIFICACIÓN Y ALCANCE DE HERRAMIENTAS CASE            | 7         |
| 1.3 DESARROLLO BASADO EN COMPONENTES                        | 8         |
| 1.4 DEL DOMINIO AL MODELO CONCEPTUAL                        | 9         |
| 1.5 TRANSFORMACIÓN HACIA EL DISEÑO LÓGICO                   | 10        |
| 1.6 NORMALIZACIÓN Y CONSISTENCIA DE DATOS                   | 11        |
| 1.7 UML Y DIAGRAMAS DE CLASES                               | 12        |
| 1.8 ARQUITECTURA CLIENTE-SERVIDOR MULTICAPA                 | 13        |
| 1.9 COLABORACIÓN EN TIEMPO REAL                             | 14        |
| 1.10 INTELIGENCIA ARTIFICIAL APLICADA AL MODELADO           | 15        |
| 1.11 IA HÍBRIDA: DISPOSITIVO Y NUBE                         | 16        |
| 1.12 INTERACCIÓN POR VOZ SIN CONEXIÓN                       | 17        |
| 1.13 SPRING BOOT, JPA Y ARQUITECTURA POR CAPAS              | 18        |
| 1.14 FLUTTER Y EXPERIENCIA MULTIPLATAFORMA                  | 19        |
| 1.15 PATRÓN OFFLINE-FIRST                                   | 20        |
| 1.16 VERSIONADO INTERNO DE ARTEFACTOS                       | 21        |
| 1.17 INTEROPERABILIDAD MEDIANTE XMI                         | 22        |
| 1.18 PERSISTENCIA Y ALMACENAMIENTO DE OBJETOS               | 23        |
| 1.19 DESPLIEGUE ADMINISTRADO EN GOOGLE CLOUD                | 24        |
| 1.20 PROCESO UNIFICADO DE DESARROLLO DE SOFTWARE            | 25        |
| <b>2 PUDS (PROCESO UNIFICADO DE DESARROLLO DE SOFTWARE)</b> | <b>26</b> |
| 2.1 CAPTURA DE REQUISITOS                                   | 26        |
| 2.1.1 Actores y responsabilidades                           | 27        |
| 2.1.2 Catálogo de casos de uso                              | 28        |
| 2.1.3 Priorización funcional                                | 29        |
| 2.1.4 Matriz de trazabilidad RF-CU                          | 30        |
| 2.2 MODELO DE CASOS DE USO                                  | 31        |
| 2.2.1 Panorama general de casos de uso                      | 31        |
| 2.2.2 Acceso, proyectos y permisos                          | 32        |
| 2.2.3 Modelado UML colaborativo                             | 33        |
| 2.2.4 Generación de software                                | 34        |
| 2.2.5 IA móvil y trabajo offline                            | 35        |
| 2.2.6 Repositorio e interoperabilidad                       | 36        |
| 2.2.7 Paridad funcional Android                             | 37        |
| 2.2.8 Invitaciones y artefactos móviles                     | 38        |
| 2.3 ESPECIFICACIÓN DE CASOS DE USO                          | 39        |
| 2.3.1 CU-01 - Gestionar acceso al sistema                   | 39        |
| 2.3.2 CU-02 - Consultar y administrar proyectos             | 40        |
| 2.3.3 CU-03 - Construir el modelo UML del dominio           | 41        |
| 2.3.4 CU-04 - Sincronizar el diagrama colaborativo          | 42        |
| 2.3.5 CU-05 - Incorporar participantes al proyecto          | 43        |
| 2.3.6 CU-06 - Archivar o retirar diagramas                  | 44        |
| 2.3.7 CU-07 - Generar backend Spring Boot por reglas        | 45        |

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| 2.3.8 CU-08 - Construir un modelo con asistencia inteligente         | 46        |
| 2.3.9 CU-09 - Crear y configurar un espacio de trabajo               | 47        |
| 2.3.10 CU-10 - Producir una colección de pruebas API                 | 48        |
| 2.3.11 CU-11 - Generar una aplicación Flutter                        | 49        |
| 2.3.12 CU-12 - Gestionar el repositorio interno de código            | 50        |
| 2.3.13 CU-13 - Interactuar con la IA por texto o voz                 | 51        |
| 2.3.14 CU-14 - Importar y exportar modelos UML                       | 52        |
| 2.3.15 CU-15 - Trabajar offline y sincronizar cambios                | 53        |
| 2.3.16 CU-16 - Generar o refinar backend con IA                      | 54        |
| 2.3.17 CU-17 - Revisar colaborativamente código generado             | 55        |
| 2.3.18 CU-18 - Detectar y resolver conflictos de sincronización      | 56        |
| 2.3.19 CU-19 - Administrar roles y permisos                          | 57        |
| 2.3.20 CU-20 - Descargar, comparar o restaurar una revisión          | 58        |
| 2.3.21 CU-21 - Gestionar cuenta, proyectos y diagramas desde Android | 59        |
| 2.3.22 CU-22 - Editar un diagrama UML táctil                         | 60        |
| 2.3.23 CU-23 - Invitar mediante enlace o código                      | 61        |
| 2.3.24 CU-24 - Generar y revisar artefactos desde Android            | 62        |
| 2.3.25 CU-25 - Intercambiar y sincronizar trabajo desde Android      | 63        |
| 2.4 ESTRUCTURA DEL MODELO DE CASOS DE USO                            | 64        |
| <b>3 ANÁLISIS</b>                                                    | <b>65</b> |
| 3.1 ANÁLISIS DE LA ARQUITECTURA                                      | 65        |
| 3.2 RELACIÓN ENTRE PAQUETES Y CASOS DE USO                           | 66        |
| 3.3 ANÁLISIS DE CASOS DE USO                                         | 67        |
| 3.3.1 Comunicación CU-01 y CU-02                                     | 67        |
| 3.3.2 Comunicación CU-03 y CU-04                                     | 68        |
| 3.3.3 Comunicación CU-05 y CU-06                                     | 69        |
| 3.3.4 Comunicación CU-07 y CU-08                                     | 70        |
| 3.3.5 Comunicación CU-09 y CU-10                                     | 71        |
| 3.3.6 Comunicación CU-11 y CU-12                                     | 72        |
| 3.3.7 Comunicación CU-13 y CU-14                                     | 73        |
| 3.3.8 Comunicación CU-15 y CU-16                                     | 74        |
| 3.3.9 Comunicación CU-17 y CU-18                                     | 75        |
| 3.3.10 Comunicación CU-19 y CU-20                                    | 76        |
| 3.3.11 Comunicación CU-21 y CU-22                                    | 77        |
| 3.3.12 Comunicación CU-23 a CU-25                                    | 78        |
| 3.4 COHESIÓN Y DEPENDENCIAS DE PAQUETES                              | 79        |
| <b>4 DISEÑO</b>                                                      | <b>80</b> |
| 4.1 DISEÑO FÍSICO PARA GOOGLE CLOUD                                  | 80        |
| 4.2 DISEÑO LÓGICO POR SERVICIOS                                      | 81        |
| 4.3 DISEÑO DE COMPONENTES                                            | 82        |
| 4.4 MODELO DE CLASES DEL DOMINIO                                     | 83        |
| 4.5 MODELO ENTIDAD-RELACIÓN                                          | 84        |
| 4.6 MAPEO LÓGICO-FÍSICO                                              | 85        |
| 4.7 DISEÑO FÍSICO - EXTRACTO SQL                                     | 86        |

|                                              |            |
|----------------------------------------------|------------|
| 4.8 DIAGRAMAS DE SECUENCIA                   | 87         |
| 4.8.1 Secuencia CU-01 y CU-02                | 87         |
| 4.8.2 Secuencia CU-03 y CU-04                | 88         |
| 4.8.3 Secuencia CU-05 y CU-06                | 89         |
| 4.8.4 Secuencia CU-07 y CU-08                | 90         |
| 4.8.5 Secuencia CU-09 y CU-10                | 91         |
| 4.8.6 Secuencia CU-11 y CU-12                | 92         |
| 4.8.7 Secuencia CU-13 y CU-14                | 93         |
| 4.8.8 Secuencia CU-15 y CU-16                | 94         |
| 4.8.9 Secuencia CU-17 y CU-18                | 95         |
| 4.8.10 Secuencia CU-19 y CU-20               | 96         |
| 4.8.11 Secuencia CU-21 y CU-22               | 97         |
| 4.8.12 Secuencia CU-23 a CU-25               | 98         |
| 4.9 DIAGRAMAS DE ACTIVIDAD                   | 99         |
| 4.9.1 Actividad de IA híbrida                | 99         |
| 4.9.2 Actividad de sincronización offline    | 100        |
| 4.9.3 Actividad de importación XML           | 101        |
| <b>5 IMPLEMENTACIÓN</b>                      | <b>102</b> |
| 5.1 LENGUAJES Y PLATAFORMAS                  | 102        |
| 5.2 PERSISTENCIA CON POSTGRESQL Y PRISMA     | 103        |
| 5.3 FRONTEND NEXT.JS                         | 104        |
| 5.4 BACKEND NESTJS                           | 105        |
| 5.5 COLABORACIÓN Y REPOSITORIO INTERNO       | 106        |
| 5.6 IA LOCAL Y SERVICIO AVANZADO             | 107        |
| 5.7 DESPLIEGUE EN GOOGLE CLOUD               | 108        |
| 5.8 SEGURIDAD, PRUEBAS Y ENTREGA             | 109        |
| <b>6 DOCUMENTACIÓN PARA USO DEL SOFTWARE</b> | <b>110</b> |
| 6.1 CREAR UNA CUENTA                         | 110        |
| 6.2 ABRIR UN ESPACIO DE TRABAJO              | 111        |
| 6.3 CREAR UN DIAGRAMA UML                    | 112        |
| 6.4 AÑADIR UNA CLASE                         | 113        |
| 6.5 DEFINIR UNA RELACIÓN                     | 114        |
| 6.6 EDITAR ATRIBUTOS                         | 115        |
| 6.7 CONVERSAR CON EL ASISTENTE               | 116        |
| 6.8 INVITAR PARTICIPANTES                    | 117        |
| 6.9 GENERAR UN PROYECTO BACKEND              | 118        |
| 6.10 EXPLORAR EL REPOSITORIO INTERNO         | 119        |
| 6.11 TRABAJAR CON IA MÓVIL OFFLINE           | 120        |
| <b>7 BIBLIOGRAFÍA</b>                        | <b>121</b> |
| 7.1 BIBLIOGRAFÍA                             | 121        |
| 7.2 BIBLIOGRAFÍA - CONTINUACIÓN              | 122        |
| <b>8 ANEXOS</b>                              | <b>123</b> |
| 8.1 REPOSITORIO OFICIAL DEL PROYECTO         | 123        |
| 8.2 INVENTARIO DE ESTADO FUNCIONAL           | 124        |

## 1.1 Herramientas CASE como soporte del proceso

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

Una herramienta CASE reúne técnicas y automatizaciones que ayudan a describir, revisar y transformar productos de ingeniería de software. Su valor no se limita al dibujo: aumenta cuando el modelo conserva reglas que otras etapas pueden reutilizar.

En esta plataforma, el diagrama UML funciona como una fuente estructurada para validar el dominio, coordinar al equipo y producir artefactos ejecutables. Esa continuidad reduce interpretaciones manuales entre análisis, diseño e implementación.

### Conceptos esenciales

- Modelado con semántica verificable.
- Trazabilidad entre requisitos, clases y código.
- Automatización de tareas repetitivas y revisiones.

#### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de herramientas case como soporte del proceso dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

## 1.2 Clasificación y alcance de herramientas CASE

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

Las soluciones Upper-CASE se concentran en requisitos y diseño; las Lower-CASE apoyan construcción, pruebas y mantenimiento. Un entorno integrado enlaza ambos extremos y mantiene relaciones entre sus artefactos.

El proyecto adopta una orientación integrada: combina edición UML, colaboración, generación de código, pruebas de API, repositorio interno e interoperabilidad, aunque su implementación se desarrolla de manera incremental.

### Conceptos esenciales

- Upper-CASE: análisis y modelos.
- Lower-CASE: código y verificación.
- I-CASE: ciclo conectado con trazabilidad.

#### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de clasificación y alcance de herramientas case dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

# 1.3 Desarrollo basado en componentes

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

El desarrollo por componentes separa responsabilidades en unidades con contratos explícitos. Cada unidad puede evolucionar sin exigir conocimiento de sus detalles internos siempre que conserve su interfaz.

Frontend web, API, colaboración, generación, almacenamiento, IA e importación se diseñan como componentes independientes. Esta organización facilita sustituir proveedores y desplegar servicios con escalado diferente.

## Conceptos esenciales

- Alta cohesión dentro de cada servicio.
- Bajo acoplamiento mediante API y eventos.
- Pruebas enfocadas por responsabilidad.

### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de desarrollo basado en componentes dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

## 1.4 Del dominio al modelo conceptual

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

El modelado conceptual identifica objetos relevantes, propiedades y asociaciones sin comprometerse todavía con una tecnología de persistencia. UML permite expresar esa estructura mediante clases, atributos, operaciones y multiplicidades.

Una clase del diagrama puede convertirse en una entidad persistente, pero antes deben aclararse identidad, opcionalidad, cardinalidad y reglas del negocio. La plataforma valida estas decisiones antes de generar código.

### Conceptos esenciales

- Clases como conceptos del dominio.
- Relaciones como restricciones semánticas.
- Multiplicidades como reglas de participación.

#### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de del dominio al modelo conceptual dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

## 1.5 Transformación hacia el diseño lógico

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

La transformación lógica convierte el modelo conceptual en estructuras que un sistema de datos puede implementar. Las asociaciones uno a muchos suelen materializarse mediante claves foráneas; las relaciones muchos a muchos requieren una entidad asociativa.

El generador conserva identificadores estables y un manifiesto de decisiones para que el código resultante pueda relacionarse con la versión exacta del diagrama que lo originó.

### Conceptos esenciales

- Mapeo de tipos UML a tipos de persistencia.
- Creación de claves y restricciones.
- Registro de decisiones de transformación.

#### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de transformación hacia el diseño lógico dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

## 1.6 Normalización y consistencia de datos

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

Normalizar significa organizar los atributos y dependencias para evitar redundancias que provoquen anomalías de inserción, actualización o borrado. La tercera forma normal constituye una referencia práctica para sistemas transaccionales.

El editor puede detectar duplicaciones evidentes y relaciones incompletas, pero la decisión final depende del significado del dominio. La automatización debe explicar sus advertencias en lugar de alterar silenciosamente el modelo.

### Conceptos esenciales

- Atributos atómicos.
- Dependencia respecto de la clave completa.
- Ausencia de dependencias transitivas no justificadas.

#### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de normalización y consistencia de datos dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

## 1.7 UML y diagramas de clases

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

UML proporciona una notación común para comunicar estructuras y comportamientos de un sistema. El diagrama de clases describe tipos, miembros, visibilidad y relaciones sin imponer un lenguaje de programación.

La plataforma utiliza un subconjunto controlado de UML para asegurar que cada elemento pueda validarse, intercambiarse y transformarse de forma reproducible.

### Conceptos esenciales

- Asociación, agregación, composición y herencia.
- Visibilidad y multiplicidad explícitas.
- Paquetes para organizar modelos extensos.

#### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de uml y diagramas de clases dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

# 1.8 Arquitectura cliente-servidor multicapa

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

Una arquitectura multicapa distribuye presentación, reglas de aplicación y persistencia. El navegador y la aplicación móvil consumen una API central que aplica autorización y coordina servicios especializados.

Las operaciones en tiempo real viajan por WebSocket, mientras que las consultas, cargas y descargas utilizan interfaces HTTP. Ambos canales comparten las mismas reglas de pertenencia y rol.

## Conceptos esenciales

- Presentación web y móvil.
- API y servicios de dominio.
- Persistencia relacional y de objetos.

### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de arquitectura cliente-servidor multicapa dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

## 1.9 Colaboración en tiempo real

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

La colaboración exige ordenar eventos, identificar al autor y recuperar el estado después de una desconexión. Mostrar cursores o presencia es útil, pero no reemplaza una confirmación persistente de cada cambio.

El servidor asigna secuencias y los clientes aplican operaciones idempotentes. Cuando una versión base diverge, el sistema conserva la operación y abre un proceso de conciliación.

### Conceptos esenciales

- Autenticación del canal WebSocket.
- Secuencias y reintentos idempotentes.
- Reconexión basada en la última confirmación.

#### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de colaboración en tiempo real dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

# 1.10 Inteligencia artificial aplicada al modelado

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

La IA generativa puede convertir una descripción en una propuesta estructurada, explicar relaciones y detectar omisiones. Su respuesta debe tratarse como una sugerencia sujeta a validación, no como una modificación automática del proyecto.

El diseño fuerza formatos de salida conocidos y presenta diferencias antes de aplicar cambios. Esto reduce el riesgo de incorporar entidades o reglas inventadas por el modelo.

## Conceptos esenciales

- Contexto mínimo necesario.
- Salida estructurada y validada.
- Confirmación humana antes de publicar.

### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de inteligencia artificial aplicada al modelado dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

# 1.11 IA híbrida: dispositivo y nube

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

Una estrategia híbrida asigna órdenes limitadas a un modelo local y reserva el razonamiento complejo para un servicio remoto. La aplicación decide según conectividad, capacidad del dispositivo y riesgo de la operación.

FunctionGemma o un modelo compacto equivalente puede seleccionar funciones declaradas para crear clases, consultar elementos o ejecutar plantillas. La nube atiende refinamientos extensos y propuestas arquitectónicas.

## Conceptos esenciales

- Funciones locales acotadas y verificables.
- Degradación útil sin conexión.
- Escalamiento transparente a IA avanzada.

### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de ia híbrida: dispositivo y nube dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

# 1.12 Interacción por voz sin conexión

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

El canal de voz combina reconocimiento de habla, confirmación visual del texto y síntesis de respuesta. Mantener estas etapas separadas permite corregir una transcripción antes de ejecutar una orden.

Los modelos de STT y TTS pueden residir en el dispositivo. El audio temporal se elimina cuando deja de ser necesario y la aplicación conserva únicamente el texto autorizado para el historial.

## Conceptos esenciales

- Pulsar para hablar y revisión previa.
- STT/TTS locales.
- Privacidad y eliminación del audio temporal.

### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de interacción por voz sin conexión dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

## 1.13 Spring Boot, JPA y arquitectura por capas

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

Spring Boot facilita construir servicios Java configurables y desplegados. JPA describe la correspondencia entre objetos y tablas, mientras Hibernate ejecuta gran parte del trabajo de persistencia.

El generador separa entidad, repositorio, servicio y controlador. Los DTO aíslan los contratos de entrada y salida para evitar que la API dependa directamente de la representación persistente.

### Conceptos esenciales

- Entity para el estado persistente.
- Repository para acceso a datos.
- Service para reglas y Controller para HTTP.

#### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de spring boot, jpa y arquitectura por capas dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

# 1.14 Flutter y experiencia multiplataforma

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

Flutter permite compartir una base de código entre Android, iOS y otras plataformas. Su árbol de widgets y su sistema reactivo resultan adecuados para editores, formularios y estados de sincronización.

La aplicación móvil conserva paridad de resultados con la web mediante una interfaz táctil adaptada. Las tareas complejas cambian su composición visual, pero mantienen permisos, datos y trazabilidad.

## Conceptos esenciales

- Navegación declarativa.
- Estado predecible y persistente.
- Diseño adaptado a pantallas pequeñas.

### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de flutter y experiencia multiplataforma dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

## 1.15 Patrón offline-first

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

Offline-first considera la copia local como una interfaz operativa y no como un simple caché. Cada cambio se registra en una cola durable con identificador, versión base y estado de envío.

Al reconectar, el cliente obtiene novedades, evalúa compatibilidad y transmite operaciones pendientes. Una confirmación del servidor es la única señal que permite retirar una operación de la cola.

### Conceptos esenciales

- Base local versionada.
- Cola durable de operaciones.
- Sincronización reanudable e idempotente.

#### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de patrón offline-first dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

## 1.16 Versionado interno de artefactos

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

El repositorio interno ofrece historial sin exigir que cada usuario conozca Git. Una revisión agrupa archivos, huellas, autor, fecha, generador y versión del modelo de origen.

Las revisiones publicadas son inmutables. Restaurar crea una revisión nueva, lo que permite volver a un resultado anterior sin borrar decisiones posteriores.

### Conceptos esenciales

- Árbol de archivos navegable.
- Comparación textual entre revisiones.
- Descargas temporales y autorizadas.

#### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de versionado interno de artefactos dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

# 1.17 Interoperabilidad mediante XMI

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

XMI utiliza XML para intercambiar metamodelos y modelos entre herramientas. Aunque establece una base común, los proveedores suelen incluir extensiones propias para coordenadas, estilos o características particulares.

La importación convierte primero a un modelo canónico y muestra advertencias antes de confirmar. La exportación prioriza clases, atributos, operaciones, relaciones y multiplicidades reconocibles por otras herramientas UML.

## Conceptos esenciales

- Compatibilidad inicial con XMI 2.1 y XMI 2.5.1.
- Vista previa y reporte de pérdidas.
- Auto-layout cuando no existen coordenadas compatibles.

### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de interoperabilidad mediante xmi dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

# 1.18 Persistencia y almacenamiento de objetos

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

Los datos transaccionales y relaciones de autorización se adaptan a PostgreSQL. Los árboles de código, paquetes ZIP e imágenes pueden crecer y se administran mejor en almacenamiento de objetos.

La base conserva metadatos y referencias; el contenido se entrega mediante accesos temporales. Esta separación evita depender del disco efímero de una instancia de aplicación.

## Conceptos esenciales

- Cloud SQL para consistencia relacional.
- Cloud Storage para artefactos.
- Huellas para verificar integridad.

### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de persistencia y almacenamiento de objetos dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

# 1.19 Despliegue en Google Cloud

Fundamentación de una primera etapa verificable y su posible evolución.

Compute Engine permite ejecutar el frontend, la API, PostgreSQL y Socket.IO en una VM única.

El disco persistente conserva la base de datos y los artefactos fuera de cada release; Caddy ofrece HTTPS.

El piloto no proporciona alta disponibilidad. El acceso público se limita a 80/443 y se programan respaldos.

Cloud Run, Cloud SQL, Cloud Storage y Redis son opciones para una fase distribuida posterior; requieren adaptar almacenamiento y eventos entre instancias.

El despliegue remoto aún no se ha efectuado; la guía y las plantillas se verifican localmente.

| Criterio        | Aplicación en el proyecto                                                        |
|-----------------|----------------------------------------------------------------------------------|
| Persistencia    | Base y artefactos en disco persistente, fuera de /opt/proyecto/current.          |
| Seguridad       | Secretos fuera del repositorio, TLS y puertos internos cerrados.                 |
| Escalado futuro | Separar estado y agregar coordinación distribuida antes de múltiples instancias. |

# 1.20 Proceso Unificado de Desarrollo de Software

Fundamentación aplicada al alcance real y objetivo de ProyectoSOFTWARE1.

El PUDS organiza el trabajo en iteraciones que recorren requisitos, análisis, diseño, implementación y pruebas con distinto énfasis. Cada ciclo produce una línea base evaluable y reduce riesgos específicos.

Este documento registra el primer parcial como una visión integrada: distingue el sistema base comprobado de las ampliaciones que guían los ciclos siguientes.

## Conceptos esenciales

- Inicio para alcance y viabilidad.
- Elaboración para arquitectura y riesgos.
- Construcción y transición para producto verificable.

### Relación con el proyecto

Aplicación concreta: este fundamento orienta las decisiones de proceso unificado de desarrollo de software dentro del editor UML, los servicios de generación y la trazabilidad documental. La plataforma separa lo que ya puede demostrarse en el código base de aquello que pertenece a la arquitectura prevista para las siguientes iteraciones.

| Perspectiva          | Aplicación en el sistema                                                          |
|----------------------|-----------------------------------------------------------------------------------|
| Problema que atiende | Reduce ambigüedad y mantiene continuidad entre modelo, código y colaboración.     |
| Decisión adoptada    | Usar contratos, versiones y validaciones explícitas antes de publicar resultados. |
| Riesgo controlado    | Evitar que una automatización o desconexión altere el proyecto sin trazabilidad.  |

## 2 PUDS - Captura de requisitos

Primera línea base del producto y ampliaciones acordadas para colaboración, IA, móvil e interoperabilidad.

La captura de requisitos convierte necesidades de los participantes en comportamientos verificables. El sistema base aporta autenticación, workspaces, edición UML, colaboración y generación; la ampliación incorpora repositorio interno, aplicación móvil offline-first, IA multimodal y compatibilidad XMI.

Los casos de uso se numeran de CU-01 a CU-25. Cada uno se relaciona con requisitos funcionales y aparece nuevamente en análisis y diseño, evitando que una función nueva quede documentada solamente como un anexo.

| Ciclo           | Objetivo                                                      | Resultado documental                                     |
|-----------------|---------------------------------------------------------------|----------------------------------------------------------|
| Ciclo 1         | Establecer modelado, colaboración y generación base.          | CU-01 a CU-11, arquitectura y evidencia web.             |
| Ampliación      | Incorporar repositorio, móvil, IA híbrida, XMI y cloud.       | CU-12 a CU-20 integrados en todos los flujos.            |
| Paridad Android | Completar las funciones web con interacción táctil y offline. | CU-21 a CU-25, invitaciones portables y editor completo. |

## 2.1.1 Actores y responsabilidades

Los componentes técnicos aparecen como actores de apoyo únicamente cuando intercambian información con un caso de uso.

| ID | Actor                     | Responsabilidad                                                                                               |
|----|---------------------------|---------------------------------------------------------------------------------------------------------------|
| A1 | Ingeniero propietario     | Responsable de crear el espacio de trabajo, administrar participantes y controlar las versiones del proyecto. |
| A2 | Ingeniero editor          | Participante autorizado para modelar, generar artefactos y colaborar dentro de un proyecto compartido.        |
| A3 | Ingeniero observador      | Participante con acceso de lectura a diagramas, código, comentarios y revisiones publicadas.                  |
| A4 | Herramienta UML externa   | Aplicación compatible que intercambia modelos mediante XMI, JSON o un paquete exportable.                     |
| A5 | Motor de IA local         | Modelo compacto del dispositivo que interpreta órdenes básicas sin depender de internet.                      |
| A6 | Servicio de IA en la nube | Motor avanzado utilizado para razonamiento, generación y refinamiento de mayor complejidad.                   |

### Aclaración

La aplicación móvil es un canal del producto, no una persona. En los diagramas de arquitectura se representa como cliente; los usuarios conservan sus roles A1, A2 o A3.

## 2.1.2 Catálogo de casos de uso

Resumen funcional reformulado y ampliado a veinticinco comportamientos trazables.

| ID    | Nombre                                                | Actores            |
|-------|-------------------------------------------------------|--------------------|
| CU-01 | Gestionar acceso al sistema                           | A1, A2, A3         |
| CU-02 | Consultar y administrar proyectos                     | A1, A2, A3         |
| CU-03 | Construir el modelo UML del dominio                   | A1, A2             |
| CU-04 | Sincronizar el diagrama colaborativo                  | A1, A2             |
| CU-05 | Incorporar participantes al proyecto                  | A1                 |
| CU-06 | Archivar o retirar diagramas                          | A1                 |
| CU-07 | Generar backend Spring Boot por reglas                | A1, A2             |
| CU-08 | Construir un modelo con asistencia inteligente        | A1, A2, A6         |
| CU-09 | Crear y configurar un espacio de trabajo              | A1                 |
| CU-10 | Producir una colección de pruebas API                 | A1, A2             |
| CU-11 | Generar una aplicación Flutter                        | A1, A2             |
| CU-12 | Gestionar el repositorio interno de código            | A1, A2, A3         |
| CU-13 | Interactuar con la IA por texto o voz                 | A1, A2, A5, A6     |
| CU-14 | Importar y exportar modelos UML                       | A1, A2, A4         |
| CU-15 | Trabajar offline y sincronizar cambios                | A1, A2, A5         |
| CU-16 | Generar o refinar backend con IA                      | A1, A2, A6         |
| CU-17 | Revisar colaborativamente código generado             | A1, A2, A3         |
| CU-18 | Detectar y resolver conflictos de sincronización      | A1, A2             |
| CU-19 | Administrar roles y permisos                          | A1                 |
| CU-20 | Descargar, comparar o restaurar una revisión          | A1, A2, A3         |
| CU-21 | Gestionar cuenta, proyectos y diagramas desde Android | A1, A2, A3         |
| CU-22 | Editar un diagrama UML táctil                         | A1, A2             |
| CU-23 | Invitar mediante enlace o código                      | A1, A2, A3         |
| CU-24 | Generar y revisar artefactos desde Android            | A1, A2, A3         |
| CU-25 | Intercambiar y sincronizar trabajo desde Android      | A1, A2, A3, A4, A5 |

## 2.1.3 Priorización funcional

La prioridad combina valor académico, dependencia técnica y riesgo de implementación.

| ID    | Caso de uso                                           | Prioridad | Justificación                                    |
|-------|-------------------------------------------------------|-----------|--------------------------------------------------|
| CU-01 | Gestionar acceso al sistema                           | ALTA      | Base del flujo o control de seguridad.           |
| CU-02 | Consultar y administrar proyectos                     | ALTA      | Base del flujo o control de seguridad.           |
| CU-03 | Construir el modelo UML del dominio                   | ALTA      | Base del flujo o control de seguridad.           |
| CU-04 | Sincronizar el diagrama colaborativo                  | ALTA      | Base del flujo o control de seguridad.           |
| CU-05 | Incorporar participantes al proyecto                  | MEDIA     | Complementa el flujo principal y puede iterarse. |
| CU-06 | Archivar o retirar diagramas                          | MEDIA     | Complementa el flujo principal y puede iterarse. |
| CU-07 | Generar backend Spring Boot por reglas                | ALTA      | Base del flujo o control de seguridad.           |
| CU-08 | Construir un modelo con asistencia inteligente        | MEDIA     | Complementa el flujo principal y puede iterarse. |
| CU-09 | Crear y configurar un espacio de trabajo              | ALTA      | Base del flujo o control de seguridad.           |
| CU-10 | Producir una colección de pruebas API                 | MEDIA     | Complementa el flujo principal y puede iterarse. |
| CU-11 | Generar una aplicación Flutter                        | ALTA      | Base del flujo o control de seguridad.           |
| CU-12 | Gestionar el repositorio interno de código            | ALTA      | Base del flujo o control de seguridad.           |
| CU-13 | Interactuar con la IA por texto o voz                 | ALTA      | Base del flujo o control de seguridad.           |
| CU-14 | Importar y exportar modelos UML                       | ALTA      | Base del flujo o control de seguridad.           |
| CU-15 | Trabajar offline y sincronizar cambios                | ALTA      | Base del flujo o control de seguridad.           |
| CU-16 | Generar o refinar backend con IA                      | ALTA      | Base del flujo o control de seguridad.           |
| CU-17 | Revisar colaborativamente código generado             | MEDIA     | Complementa el flujo principal y puede iterarse. |
| CU-18 | Detectar y resolver conflictos de sincronización      | ALTA      | Base del flujo o control de seguridad.           |
| CU-19 | Administrar roles y permisos                          | ALTA      | Base del flujo o control de seguridad.           |
| CU-20 | Descargar, comparar o restaurar una revisión          | ALTA      | Base del flujo o control de seguridad.           |
| CU-21 | Gestionar cuenta, proyectos y diagramas desde Android | ALTA      | Base del flujo o control de seguridad.           |
| CU-22 | Editar un diagrama UML táctil                         | ALTA      | Base del flujo o control de seguridad.           |
| CU-23 | Invitar mediante enlace o código                      | ALTA      | Base del flujo o control de seguridad.           |
| CU-24 | Generar y revisar artefactos desde Android            | ALTA      | Base del flujo o control de seguridad.           |
| CU-25 | Intercambiar y sincronizar trabajo desde Android      | ALTA      | Base del flujo o control de seguridad.           |

## 2.1.4 Matriz de trazabilidad RF-CU

Cada requisito funcional se conecta con al menos un caso de uso y una evidencia esperada.

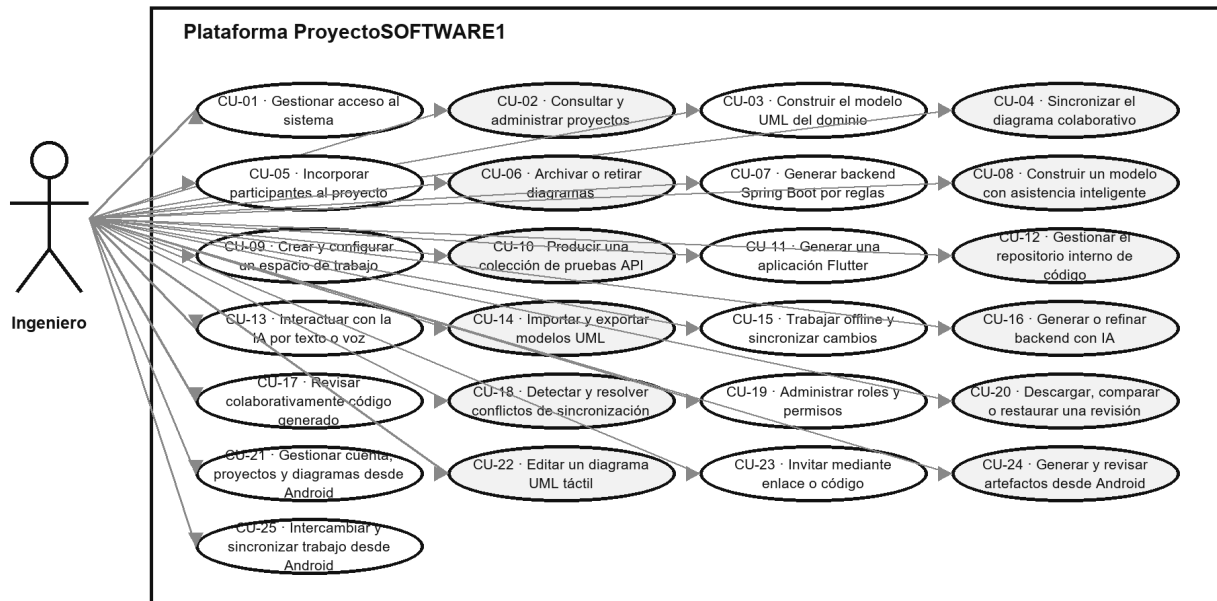
| RF    | Descripción resumida                                                                             | CU                                                                   |
|-------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| RF-01 | Registrar usuarios e iniciar o cerrar una sesión protegida.                                      | CU-01, CU-21                                                         |
| RF-02 | Crear, consultar y organizar espacios de trabajo UML.                                            | CU-02, CU-09, CU-21                                                  |
| RF-03 | Editar clases, atributos, operaciones y relaciones en un lienzo gráfico.                         | CU-03, CU-22                                                         |
| RF-04 | Distribuir cambios del diagrama entre participantes conectados.                                  | CU-04, CU-25                                                         |
| RF-05 | Invitar personas y controlar su participación en un proyecto.                                    | CU-05, CU-19, CU-23                                                  |
| RF-06 | Archivar modelos sin perder su trazabilidad histórica.                                           | CU-06, CU-21                                                         |
| RF-07 | Transformar un diagrama válido en un backend Spring Boot reproducible.                           | CU-07, CU-16, CU-24                                                  |
| RF-08 | Crear o completar modelos UML mediante instrucciones en lenguaje natural.                        | CU-08                                                                |
| RF-09 | Configurar metadatos, permisos y opciones del espacio de trabajo.                                | CU-09, CU-21                                                         |
| RF-10 | Producir colecciones de pruebas para la API generada.                                            | CU-10, CU-24                                                         |
| RF-11 | Generar una aplicación Flutter conectada a la API del proyecto.                                  | CU-11, CU-24                                                         |
| RF-12 | Guardar el código generado en un repositorio interno por proyecto.                               | CU-07, CU-11, CU-12, CU-24                                           |
| RF-13 | Mantener revisiones inmutables y comparables de cada generación.                                 | CU-07, CU-10, CU-11, CU-12, CU-17, CU-20, CU-24                      |
| RF-14 | Aceptar solicitudes de IA por texto y por audio.                                                 | CU-13, CU-25                                                         |
| RF-15 | Ejecutar comandos UML básicos sin conexión a internet.                                           | CU-13, CU-15, CU-22, CU-25                                           |
| RF-16 | Sincronizar operaciones móviles pendientes al recuperar conectividad.                            | CU-15, CU-18, CU-22, CU-25                                           |
| RF-17 | Importar y exportar modelos mediante XMI, JSON y ZIP propio.                                     | CU-14, CU-25                                                         |
| RF-18 | Refinar código y decisiones de diseño con una IA en la nube.                                     | CU-08, CU-13, CU-16                                                  |
| RF-19 | Permitir revisión colaborativa del código y su historial.                                        | CU-17, CU-24                                                         |
| RF-20 | Detectar y resolver conflictos conservando ambas variantes.                                      | CU-15, CU-18, CU-22, CU-25                                           |
| RF-21 | Administrar roles OWNER, EDITOR y VIEWER en todos los canales.                                   | CU-02, CU-05, CU-17, CU-19, CU-21, CU-23, CU-24                      |
| RF-22 | Descargar, comparar y restaurar revisiones autorizadas.                                          | CU-20, CU-24                                                         |
| RF-23 | Persistir datos y artefactos fuera de instancias efímeras de ejecución.                          | CU-12, CU-20                                                         |
| RF-24 | Registrar eventos relevantes para auditoría y diagnóstico.                                       | CU-01, CU-03, CU-04, CU-06, CU-14, CU-16, CU-18, CU-19, CU-23, CU-25 |
| RF-25 | Ofrecer en Android las funciones de cuenta, perfil, idioma, tema y acceso disponibles en la web. | CU-21                                                                |
| RF-26 | Crear y editar clases, atributos, métodos y relaciones mediante un editor UML táctil.            | CU-22                                                                |
| RF-27 | Administrar desde Android el ciclo de vida de proyectos y diagramas según permisos.              | CU-21                                                                |
| RF-28 | Invitar por correo, enlace o código temporal, revocable y asociado a un rol.                     | CU-23                                                                |
| RF-29 | Colaborar en tiempo real desde Android con presencia y operaciones confirmadas.                  | CU-22, CU-25                                                         |
| RF-30 | Generar, navegar, comentar, descargar, comparar y restaurar código desde Android.                | CU-24                                                                |
| RF-31 | Importar y exportar XMI, JSON y ZIP desde el almacenamiento del dispositivo.                     | CU-25                                                                |
| RF-32 | Visualizar y resolver conflictos móviles conservando variantes, autor y versión.                 | CU-22, CU-25                                                         |

## 2.2 Panorama general de casos de uso

Vista reconstruida del modelo de casos de uso.

### Panorama general de casos de uso

Modelo de casos de uso - PUDS



#### Lectura del diagrama

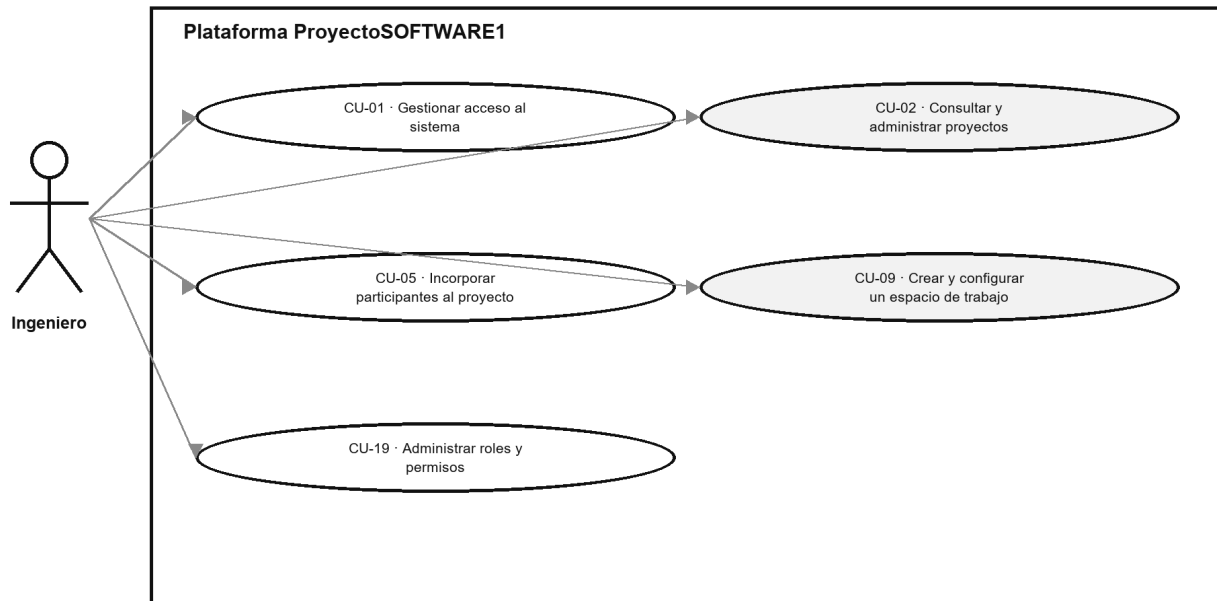
Esta vista cubre: CU-01, CU-02, CU-03, CU-04, CU-05, CU-06, CU-07, CU-08, CU-09, CU-10, CU-11, CU-12, CU-13, CU-14, CU-15, CU-16, CU-17, CU-18, CU-19, CU-20, CU-21, CU-22, CU-23, CU-24, CU-25. Las líneas expresan participación; las dependencias de inclusión o extensión se justifican en cada ficha detallada.

## 2.2 Acceso, proyectos y permisos

Vista reconstruida del modelo de casos de uso.

### Acceso, proyectos y permisos

Modelo de casos de uso - PUDS



#### Lectura del diagrama

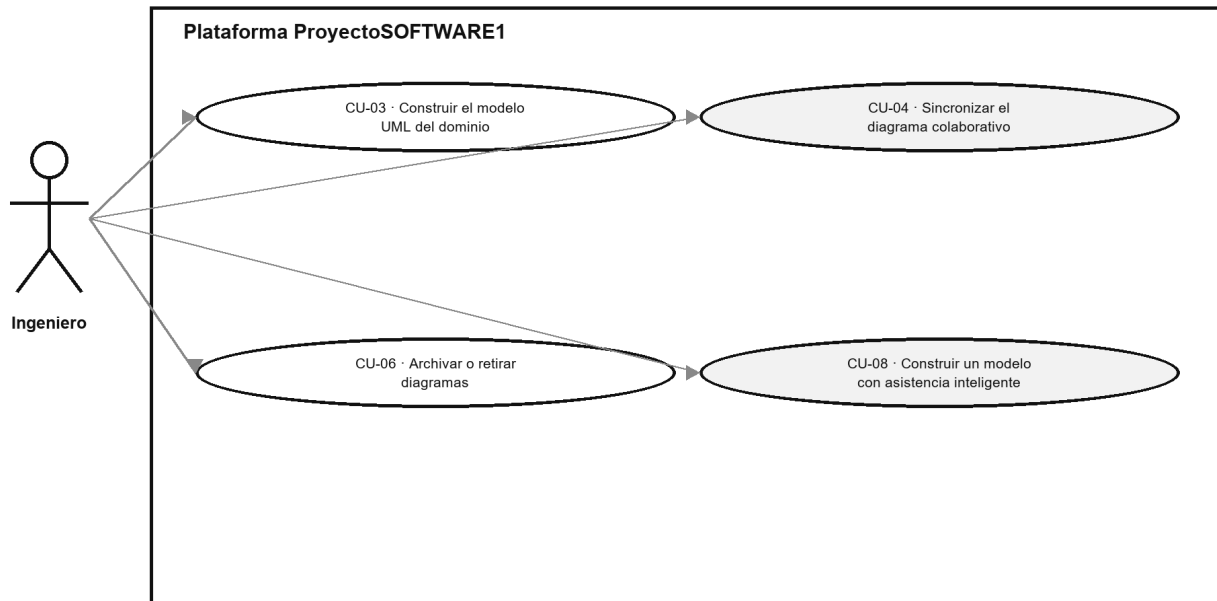
Esta vista cubre: CU-01, CU-02, CU-05, CU-09, CU-19. Las líneas expresan participación; las dependencias de inclusión o extensión se justifican en cada ficha detallada.

## 2.2 Modelado UML colaborativo

Vista reconstruida del modelo de casos de uso.

### Modelado UML colaborativo

Modelo de casos de uso - PUDS



#### Lectura del diagrama

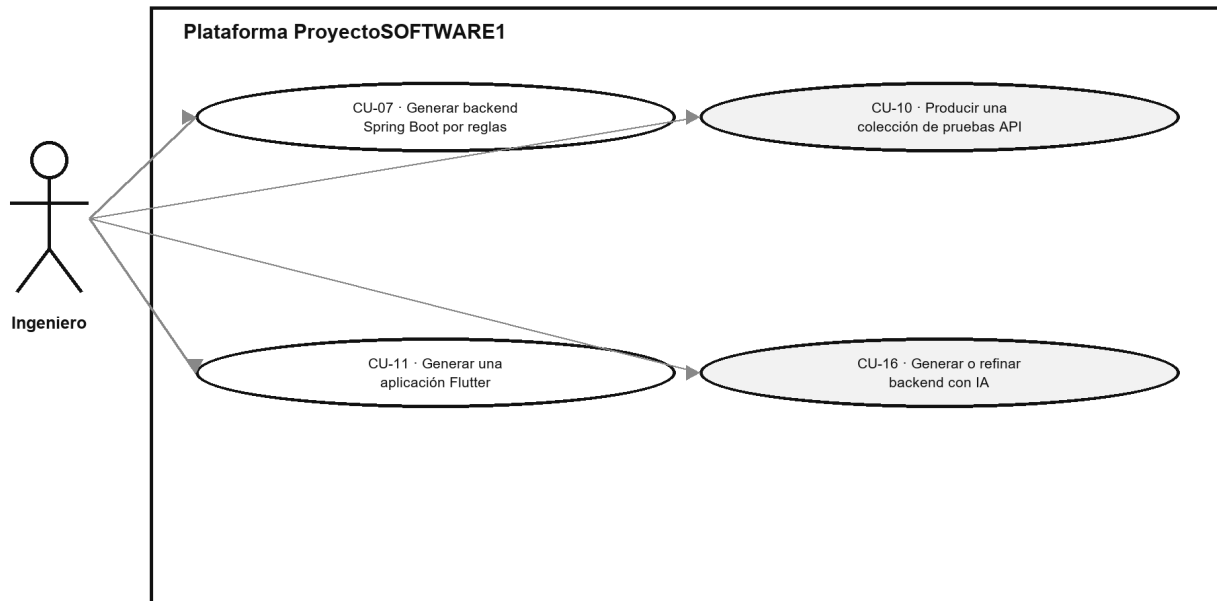
Esta vista cubre: CU-03, CU-04, CU-06, CU-08. Las líneas expresan participación; las dependencias de inclusión o extensión se justifican en cada ficha detallada.

## 2.2 Generación de software

Vista reconstruida del modelo de casos de uso.

### Generación de software

Modelo de casos de uso - PUDS

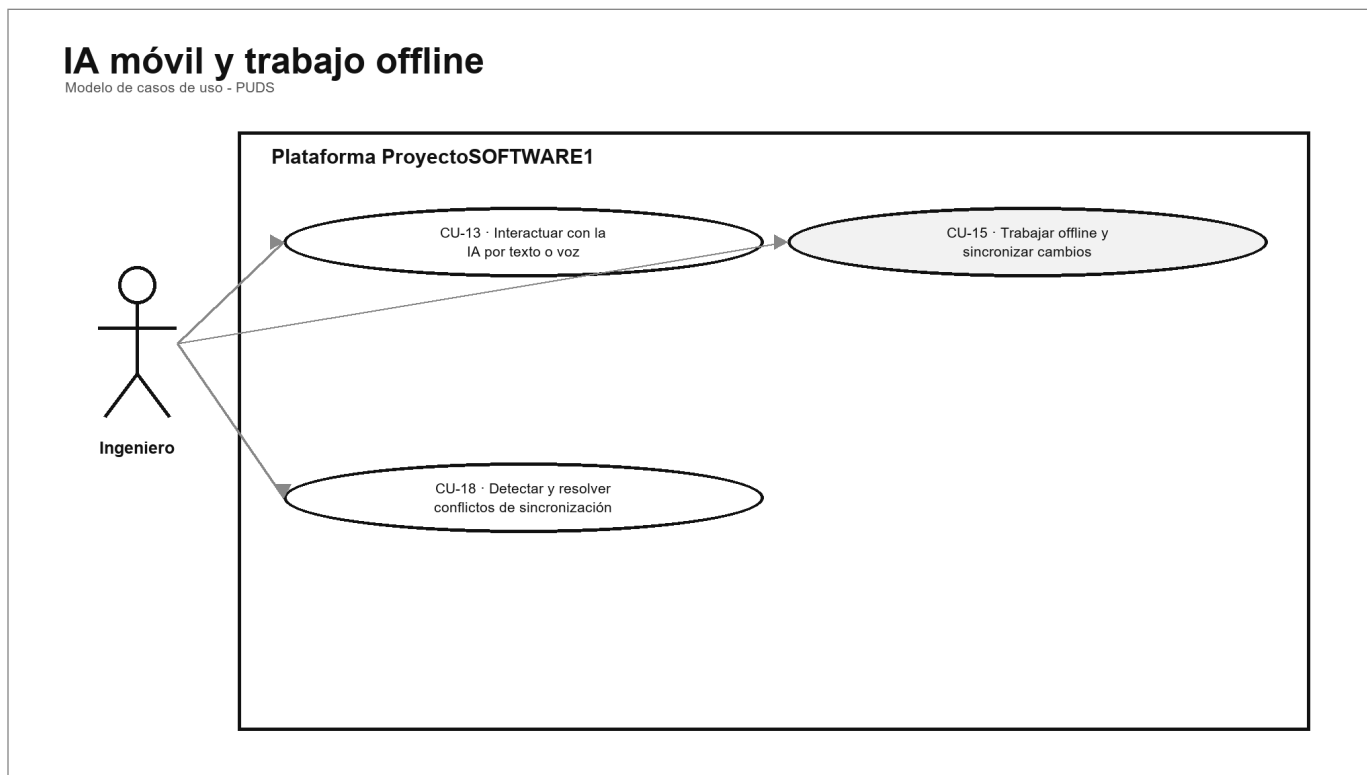


#### Lectura del diagrama

Esta vista cubre: CU-07, CU-10, CU-11, CU-16. Las líneas expresan participación; las dependencias de inclusión o extensión se justifican en cada ficha detallada.

## 2.2 IA móvil y trabajo offline

Vista reconstruida del modelo de casos de uso.



### Lectura del diagrama

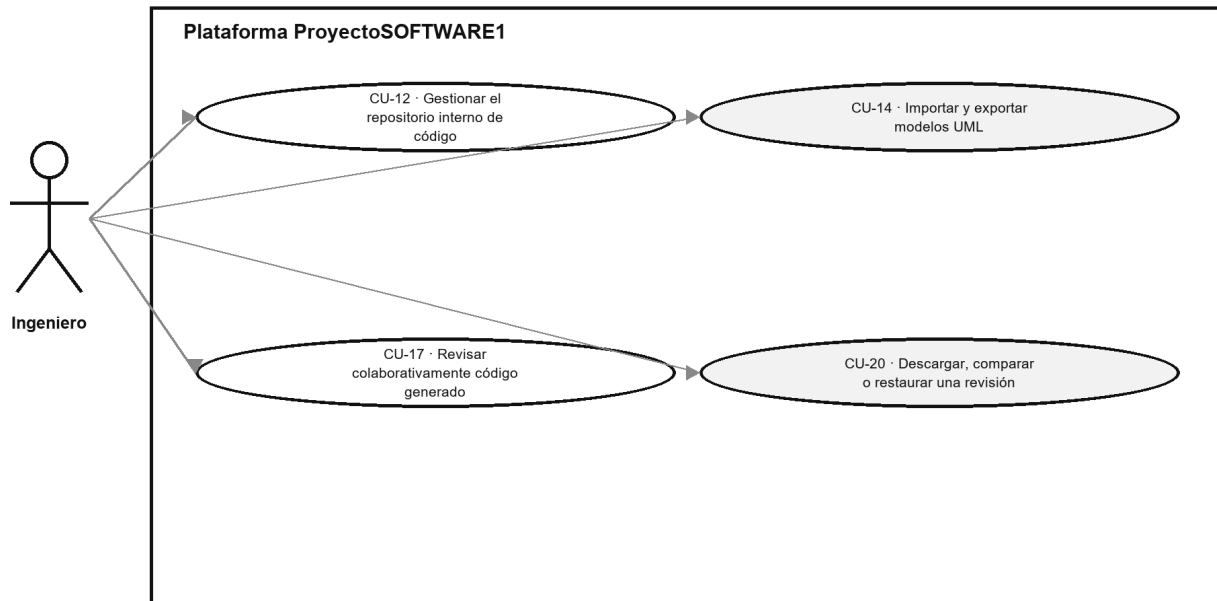
Esta vista cubre: CU-13, CU-15, CU-18. Las líneas expresan participación; las dependencias de inclusión o extensión se justifican en cada ficha detallada.

## 2.2 Repositorio e interoperabilidad

Vista reconstruida del modelo de casos de uso.

### Repositorio e interoperabilidad

Modelo de casos de uso - PUDS



#### Lectura del diagrama

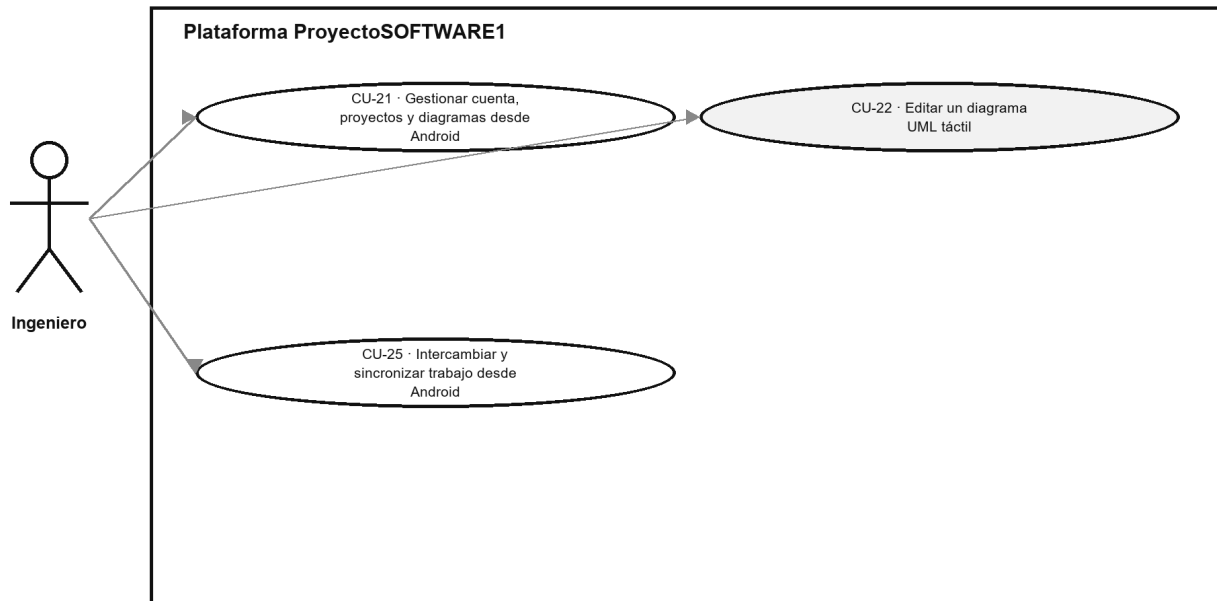
Esta vista cubre: CU-12, CU-14, CU-17, CU-20. Las líneas expresan participación; las dependencias de inclusión o extensión se justifican en cada ficha detallada.

## 2.2 Paridad funcional Android

Vista reconstruida del modelo de casos de uso.

### Paridad funcional Android

Modelo de casos de uso - PUDS



#### Lectura del diagrama

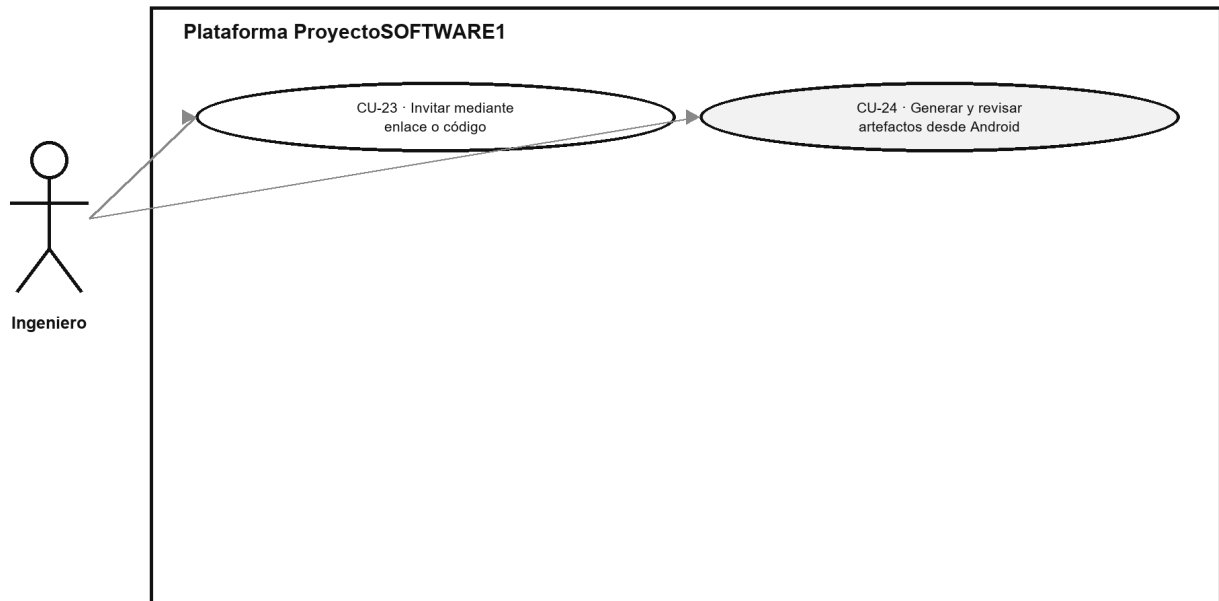
Esta vista cubre: CU-21, CU-22, CU-25. Las líneas expresan participación; las dependencias de inclusión o extensión se justifican en cada ficha detallada.

## 2.2 Invitaciones y artefactos móviles

Vista reconstruida del modelo de casos de uso.

### Invitaciones y artefactos móviles

Modelo de casos de uso - PUDS



#### Lectura del diagrama

Esta vista cubre: CU-23, CU-24. Las líneas expresan participación; las dependencias de inclusión o extensión se justifican en cada ficha detallada.

## 2.3.1 CU-01 · Gestionar acceso al sistema

Permitir que una persona cree su cuenta, se autentique y finalice la sesión sin exponer sus credenciales.

| Campo      | Especificación                                                         |
|------------|------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A3 Ingeniero observador |
| Prioridad  | ALTA                                                                   |
| Disparador | El usuario abre la pantalla de acceso o solicita cerrar su sesión.     |
| Entradas   | Nombre, correo y contraseña; o correo y contraseña para autenticación. |
| Salidas    | Cuenta creada, token de sesión y perfil mínimo.                        |
| Requisitos | RF-01, RF-24                                                           |

### Flujo principal

- 1. El sistema presenta registro e inicio de sesión.
- 2. El usuario introduce los datos requeridos.
- 3. La API valida formato y credenciales.
- 4. El servidor emite una sesión firmada.
- 5. La interfaz dirige al panel de proyectos.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                                                            |
|--------------|------------------------------------------------------------------------------------------------------------------------------------|
| Alternativas | Una sesión vigente permite ingresar directamente al panel. El usuario puede solicitar nuevamente el acceso cuando expire el token. |
| Excepciones  | Correo ya registrado. Credenciales incorrectas o cuenta deshabilitada.                                                             |
| Reglas       | Las contraseñas se almacenan mediante hash. Los tokens poseen expiración y no se incluyen en registros de diagnóstico.             |

## 2.3.2 CU-02 · Consultar y administrar proyectos

Ofrecer una vista única de los proyectos propios y compartidos, sus roles y su actividad reciente.

| Campo      | Especificación                                                         |
|------------|------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A3 Ingeniero observador |
| Prioridad  | ALTA                                                                   |
| Disparador | El usuario autenticado accede al panel principal.                      |
| Entradas   | Filtros, criterio de orden y selección.                                |
| Salidas    | Listado autorizado y navegación al proyecto.                           |
| Requisitos | RF-02, RF-21                                                           |

### Flujo principal

- 1. La interfaz solicita los proyectos autorizados.
- 2. La API filtra por propiedad o membresía.
- 3. El sistema muestra nombre, rol, diagramas y última actividad.
- 4. El usuario ordena o filtra la lista.
- 5. Selecciona un proyecto para abrirlo.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                             |
|--------------|---------------------------------------------------------------------|
| Alternativas | Una lista vacía ofrece crear un espacio o aceptar una invitación.   |
| Excepciones  | Sesión vencida. Proyecto retirado durante la consulta.              |
| Reglas       | Ningún proyecto ajeno se expone por enumeración de identificadores. |

## 2.3.3 CU-03 · Construir el modelo UML del dominio

Representar entidades del negocio mediante clases UML, atributos, operaciones y relaciones editables.

| Campo      | Especificación                                |
|------------|-----------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor |
| Prioridad  | ALTA                                          |
| Disparador | El editor abre un diagrama o crea uno nuevo.  |
| Entradas   | Elementos UML y propiedades gráficas.         |
| Salidas    | Nueva versión del modelo conceptual.          |
| Requisitos | RF-03, RF-24                                  |

### Flujo principal

- 1. El sistema carga la versión confirmada del diagrama.
- 2. El actor agrega o selecciona un elemento.
- 3. Define nombre, tipos, visibilidad y restricciones.
- 4. Conecta clases e indica relación y multiplicidad.
- 5. El editor valida y persiste la modificación.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                    |
|--------------|--------------------------------------------------------------------------------------------|
| Alternativas | El auto-layout reorganiza elementos sin alterar la semántica.                              |
| Excepciones  | Nombre duplicado. Tipo o multiplicidad inválida.                                           |
| Reglas       | Cada elemento posee un identificador estable. Una relación solo referencia clases activas. |

## 2.3.4 CU-04 · Sincronizar el diagrama colaborativo

Propagar ediciones del lienzo entre participantes conectados al mismo proyecto.

| Campo      | Especificación                                      |
|------------|-----------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor       |
| Prioridad  | ALTA                                                |
| Disparador | Un editor confirma una operación sobre el diagrama. |
| Entradas   | Operación, versión base y token de socket.          |
| Salidas    | Diagrama convergente y evento auditado.             |
| Requisitos | RF-04, RF-24                                        |

### Flujo principal

- 1. El cliente envía la operación con versión base.
- 2. El servidor autentica la conexión WebSocket.
- 3. La operación se valida contra permisos y modelo.
- 4. El cambio se persiste y recibe una secuencia.
- 5. Los demás clientes aplican el evento en orden.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                          |
|--------------|--------------------------------------------------------------------------------------------------|
| Alternativas | Tras una reconexión se solicitan eventos posteriores a la última secuencia.                      |
| Excepciones  | Evento desactualizado. Participante sin permiso de edición.                                      |
| Reglas       | Los eventos se aplican de forma idempotente. La presencia temporal no sustituye la persistencia. |

## 2.3.5 CU-05 · Incorporar participantes al proyecto

Invitar a otros ingenieros mediante correo y asignar un rol inicial controlado.

| Campo      | Especificación                                  |
|------------|-------------------------------------------------|
| Actores    | A1 Ingeniero propietario                        |
| Prioridad  | MEDIA                                           |
| Disparador | El propietario selecciona la opción de invitar. |
| Entradas   | Correo y rol solicitado.                        |
| Salidas    | Membresía o invitación registrada.              |
| Requisitos | RF-05, RF-21                                    |

### Flujo principal

- 1. El sistema muestra el formulario de invitación.
- 2. El propietario indica correo y rol.
- 3. La API comprueba propiedad y duplicados.
- 4. Se registra la membresía o invitación pendiente.
- 5. El nuevo participante recibe acceso según su estado.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                         |
|--------------|-------------------------------------------------------------------------------------------------|
| Alternativas | Una cuenta inexistente conserva la invitación hasta su registro.                                |
| Excepciones  | Invitación repetida. El propietario intenta asignar un rol no permitido.                        |
| Reglas       | Solo OWNER administra miembros. Nunca se concede un rol superior al autorizado por la política. |

## 2.3.6 CU-06 · Archivar o retirar diagramas

Ocultar un diagrama obsoleto de las vistas activas conservando historial y auditoría.

| Campo      | Especificación                                |
|------------|-----------------------------------------------|
| Actores    | A1 Ingeniero propietario                      |
| Prioridad  | MEDIA                                         |
| Disparador | El propietario solicita archivar un diagrama. |
| Entradas   | Identificador y confirmación explícita.       |
| Salidas    | Estado archivado o reactivado.                |
| Requisitos | RF-06, RF-24                                  |

### Flujo principal

- 1. La interfaz muestra el impacto y solicita confirmación.
- 2. La API verifica propiedad y dependencias.
- 3. El diagrama cambia a estado archivado.
- 4. Las revisiones permanecen disponibles para auditoría.
- 5. La lista activa se actualiza.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                    |
|--------------|----------------------------------------------------------------------------|
| Alternativas | El propietario puede reactivar un diagrama archivado.                      |
| Excepciones  | Diagrama utilizado por una generación en curso.                            |
| Reglas       | La operación es un archivado lógico y no una eliminación física inmediata. |

## 2.3.7 CU-07 · Generar backend Spring Boot por reglas

Transformar un modelo UML válido en un proyecto backend reproducible mediante plantillas deterministas.

| Campo      | Especificación                                |
|------------|-----------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor |
| Prioridad  | ALTA                                          |
| Disparador | El actor selecciona Generar backend.          |
| Entradas   | Versión UML y opciones de generación.         |
| Salidas    | Proyecto Spring Boot, manifiesto y ZIP.       |
| Requisitos | RF-07, RF-12, RF-13                           |

### Flujo principal

- 1. El sistema fija una versión del diagrama.
- 2. Valida clases, claves y relaciones.
- 3. El actor selecciona nombre, paquete y base de datos.
- 4. El generador produce entidades, repositorios, servicios y controladores.
- 5. Se empaqueta el proyecto y se registra la revisión.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                    |
|--------------|----------------------------------------------------------------------------|
| Alternativas | Puede generarse solo una vista previa antes del paquete completo.          |
| Excepciones  | Modelo inconsistente. Error al renderizar una plantilla.                   |
| Reglas       | La misma entrada y versión de plantillas generan un resultado equivalente. |

## 2.3.8 CU-08 · Construir un modelo con asistencia inteligente

Convertir una descripción del dominio en una propuesta UML editable y validada por el usuario.

| Campo      | Especificación                                                              |
|------------|-----------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A6 Servicio de IA en la nube |
| Prioridad  | MEDIA                                                                       |
| Disparador | El usuario describe el sistema o elige una plantilla de dominio.            |
| Entradas   | Descripción, restricciones y contexto del proyecto.                         |
| Salidas    | Propuesta UML validada y diferencias visibles.                              |
| Requisitos | RF-08, RF-18                                                                |

### Flujo principal

- 1. La interfaz recopila la descripción y restricciones.
- 2. La API prepara contexto y esquema de respuesta.
- 3. La IA propone clases, atributos y relaciones.
- 4. El sistema valida la estructura recibida.
- 5. El usuario revisa y confirma la incorporación.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                 |
|--------------|-------------------------------------------------------------------------|
| Alternativas | El usuario solicita una segunda propuesta o aplica solo una parte.      |
| Excepciones  | Respuesta incompleta o fuera del esquema. Servicio de IA no disponible. |
| Reglas       | Ninguna propuesta modifica el diagrama sin confirmación.                |

## 2.3.9 CU-09 · Crear y configurar un espacio de trabajo

Establecer el contenedor principal para modelos, miembros, código y decisiones del proyecto.

| Campo      | Especificación                               |
|------------|----------------------------------------------|
| Actores    | A1 Ingeniero propietario                     |
| Prioridad  | ALTA                                         |
| Disparador | El propietario selecciona Nuevo proyecto.    |
| Entradas   | Nombre, descripción y configuración inicial. |
| Salidas    | Workspace listo para modelado.               |
| Requisitos | RF-02, RF-09                                 |

### Flujo principal

- 1. El sistema solicita nombre y descripción.
- 2. El usuario define opciones iniciales.
- 3. La API valida unicidad dentro de su ámbito.
- 4. Se crea el workspace con rol OWNER.
- 5. La interfaz abre el panel del nuevo proyecto.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                 |
|--------------|---------------------------------------------------------|
| Alternativas | Puede crearse desde una importación compatible.         |
| Excepciones  | Nombre inválido. Límite de proyectos alcanzado.         |
| Reglas       | Todo workspace tiene exactamente un propietario activo. |

## 2.3.10 CU-10 · Producir una colección de pruebas API

Derivar solicitudes y verificaciones básicas de los endpoints generados.

| Campo      | Especificación                                          |
|------------|---------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor           |
| Prioridad  | MEDIA                                                   |
| Disparador | El actor solicita pruebas para una revisión de backend. |
| Entradas   | Revisión de backend y URL base de ejemplo.              |
| Salidas    | Colección de pruebas y variables seguras.               |
| Requisitos | RF-10, RF-13                                            |

### Flujo principal

- 1. El sistema lee el manifiesto del backend.
- 2. Enumera recursos y operaciones CRUD.
- 3. Genera variables de entorno y ejemplos JSON.
- 4. Añade aserciones de estado y estructura.
- 5. Guarda la colección con la revisión.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                  |
|--------------|----------------------------------------------------------|
| Alternativas | Puede producirse OpenAPI además de la colección Postman. |
| Excepciones  | Revisión sin manifiesto de endpoints.                    |
| Reglas       | Los ejemplos no contienen credenciales reales.           |

## 2.3.11 CU-11 · Generar una aplicación Flutter

Crear una base móvil multiplataforma coherente con los recursos expuestos por el backend.

| Campo      | Especificación                                |
|------------|-----------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor |
| Prioridad  | ALTA                                          |
| Disparador | El actor selecciona Generar aplicación móvil. |
| Entradas   | Modelo, contrato API y opciones Flutter.      |
| Salidas    | Proyecto Flutter, manifiesto y ZIP.           |
| Requisitos | RF-11, RF-12, RF-13                           |

### Flujo principal

- 1. El sistema fija la versión de modelo y API.
- 2. El usuario define identificador, tema y plataformas.
- 3. Se generan modelos, navegación, servicios y pantallas CRUD.
- 4. Se crean configuración y pruebas iniciales.
- 5. El resultado se almacena como revisión descargable.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                       |
|--------------|-------------------------------------------------------------------------------|
| Alternativas | Puede generarse únicamente el cliente API para integrarlo en otra aplicación. |
| Excepciones  | Tipos no compatibles con Dart. Contrato API ausente.                          |
| Reglas       | Las claves de producción nunca se incorporan al código generado.              |

## 2.3.12 CU-12 · Gestionar el repositorio interno de código

Centralizar archivos y revisiones generadas dentro del proyecto sin exigir conocimientos de Git.

| Campo      | Especificación                                                         |
|------------|------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A3 Ingeniero observador |
| Prioridad  | ALTA                                                                   |
| Disparador | Una generación termina o un participante abre la pestaña Código.       |
| Entradas   | Artefactos, manifiesto y metadatos de generación.                      |
| Salidas    | Revisión navegable del código.                                         |
| Requisitos | RF-12, RF-13, RF-23                                                    |

### Flujo principal

- 1. El sistema registra origen, autor y versión del modelo.
- 2. Calcula la estructura y huellas de los archivos.
- 3. Almacena contenido en objetos y metadatos en la base.
- 4. La interfaz presenta árbol, vista previa e historial.
- 5. El actor navega por una revisión autorizada.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                          |
|--------------|----------------------------------------------------------------------------------|
| Alternativas | Los archivos grandes se entregan mediante una URL temporal.                      |
| Excepciones  | Almacenamiento no disponible. Revisión incompleta.                               |
| Reglas       | Una revisión publicada es inmutable. Cada archivo pertenece a una sola revisión. |

## 2.3.13 CU-13 · Interactuar con la IA por texto o voz

Recibir instrucciones escritas o habladas y ejecutarlas mediante el motor apropiado.

| Campo      | Especificación                                                                                    |
|------------|---------------------------------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A5 Motor de IA local, A6 Servicio de IA en la nube |
| Prioridad  | ALTA                                                                                              |
| Disparador | El usuario envía texto o mantiene presionado el control de micrófono.                             |
| Entradas   | Texto o audio, proyecto y contexto visible.                                                       |
| Salidas    | Respuesta, función propuesta y resultado estructurado.                                            |
| Requisitos | RF-14, RF-15, RF-18                                                                               |

### Flujo principal

- 1. La aplicación obtiene texto directo o transcribe audio localmente.
- 2. Muestra el mensaje para revisión.
- 3. Clasifica la intención y la complejidad.
- 4. Selecciona IA local o servicio cloud según conectividad.
- 5. Presenta respuesta y, si se solicita, la reproduce por voz.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                                        |
|--------------|----------------------------------------------------------------------------------------------------------------|
| Alternativas | Sin internet se limita a funciones locales declaradas.                                                         |
| Excepciones  | Audio ininteligible. Orden fuera de permisos o función no disponible offline.                                  |
| Reglas       | Toda operación destructiva requiere confirmación explícita. El audio puede descartarse después de transcribir. |

## 2.3.14 CU-14 · Importar y exportar modelos UML

Intercambiar la semántica del modelo con herramientas externas compatibles.

| Campo      | Especificación                                                            |
|------------|---------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A4 Herramienta UML externa |
| Prioridad  | ALTA                                                                      |
| Disparador | El actor selecciona Importar o Exportar.                                  |
| Entradas   | XMI 2.1/2.5.1, JSON o ZIP propio.                                         |
| Salidas    | Modelo importado o paquete exportado con informe.                         |
| Requisitos | RF-17, RF-24                                                              |

### Flujo principal

- 1. El usuario elige formato y archivo o modelo origen.
- 2. El sistema valida tamaño, codificación y estructura.
- 3. Convierte al esquema canónico sin alterar todavía el proyecto.
- 4. Muestra elementos aceptados, advertencias y pérdidas potenciales.
- 5. La confirmación crea una nueva versión o descarga el paquete.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                                             |
|--------------|---------------------------------------------------------------------------------------------------------------------|
| Alternativas | Cuando faltan coordenadas se aplica auto-layout.                                                                    |
| Excepciones  | XML mal formado. Versión XMI o extensión propietaria no admitida.                                                   |
| Reglas       | La importación nunca ejecuta código incluido en el archivo. Los elementos no soportados se reportan, no se ocultan. |

## 2.3.15 CU-15 · Trabajar offline y sincronizar cambios

Permitir edición móvil básica sin red y enviar operaciones pendientes cuando vuelva la conexión.

| Campo      | Especificación                                                           |
|------------|--------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A5 Motor de IA local      |
| Prioridad  | ALTA                                                                     |
| Disparador | El usuario abre un proyecto disponible localmente o pierde conectividad. |
| Entradas   | Copia local, cola de operaciones y versión base.                         |
| Salidas    | Proyecto sincronizado o conflicto explícito.                             |
| Requisitos | RF-15, RF-16, RF-20                                                      |

### Flujo principal

- 1. La aplicación carga la última versión local confirmada.
- 2. Cada edición se guarda como operación ordenada.
- 3. La interfaz indica estado pendiente.
- 4. Al recuperar red se renueva la sesión y consulta la versión remota.
- 5. Las operaciones compatibles se envían y confirman.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                    |
|--------------|----------------------------------------------------------------------------|
| Alternativas | El usuario puede posponer la sincronización en una red limitada.           |
| Excepciones  | Sesión no renovable. Versión base divergente o almacenamiento local lleno. |
| Reglas       | Ninguna operación pendiente se elimina antes de confirmación del servidor. |

## 2.3.16 CU-16 · Generar o refinar backend con IA

Usar el diagrama y una instrucción adicional para completar decisiones o mejorar una generación backend.

| Campo      | Especificación                                                              |
|------------|-----------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A6 Servicio de IA en la nube |
| Prioridad  | ALTA                                                                        |
| Disparador | El actor solicita una generación inteligente o un refinamiento.             |
| Entradas   | Versión UML, instrucción y restricciones técnicas.                          |
| Salidas    | Propuesta, diferencias y nueva revisión de backend.                         |
| Requisitos | RF-07, RF-18, RF-24                                                         |

### Flujo principal

- 1. El sistema toma una instantánea del modelo.
- 2. El usuario expresa objetivos y restricciones.
- 3. La API elimina secretos y construye contexto acotado.
- 4. La IA devuelve un plan o parches estructurados.
- 5. El sistema valida, muestra diferencias y genera tras confirmación.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                                                     |
|--------------|-----------------------------------------------------------------------------------------------------------------------------|
| Alternativas | El actor acepta solo una parte de la propuesta.                                                                             |
| Excepciones  | Respuesta no compilable. Contexto excedido o servicio remoto indisponible.                                                  |
| Reglas       | La IA no escribe directamente sobre una revisión publicada. Toda salida conserva modelo, prompt resumido y motor utilizado. |

## 2.3.17 CU-17 · Revisar colaborativamente código generado

Facilitar que participantes autorizados consulten archivos, comentarios y decisiones sobre una revisión.

| Campo      | Especificación                                                         |
|------------|------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A3 Ingeniero observador |
| Prioridad  | MEDIA                                                                  |
| Disparador | Un participante abre una revisión del repositorio interno.             |
| Entradas   | Revisión, archivo, ubicación y comentario.                             |
| Salidas    | Hilo de revisión y actividad compartida.                               |
| Requisitos | RF-13, RF-19, RF-21                                                    |

### Flujo principal

- 1. La interfaz carga árbol y archivo seleccionado.
- 2. El actor navega a una línea o bloque.
- 3. Añade un comentario si su rol lo permite.
- 4. Los participantes reciben la actualización.
- 5. La conversación queda vinculada a archivo y revisión.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                                             |
|--------------|---------------------------------------------------------------------------------------------------------------------|
| Alternativas | Una revisión posterior muestra comentarios anteriores como referencia histórica.                                    |
| Excepciones  | Archivo binario sin vista previa. Comentario sobre una revisión retirada.                                           |
| Reglas       | VIEWER puede comentar solo si la política del proyecto lo permite. Los comentarios no alteran el archivo publicado. |

## 2.3.18 CU-18 · Detectar y resolver conflictos de sincronización

Conciliar cambios incompatibles sin perder autoría ni variantes.

| Campo      | Especificación                                                                    |
|------------|-----------------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor                                     |
| Prioridad  | ALTA                                                                              |
| Disparador | El servidor detecta operaciones concurrentes sobre el mismo campo y versión base. |
| Entradas   | Operaciones locales/remotas y versión base.                                       |
| Salidas    | Revisión conciliada o conflicto pendiente.                                        |
| Requisitos | RF-16, RF-20, RF-24                                                               |

### Flujo principal

- 1. El sistema identifica elementos y propiedades en conflicto.
- 2. Aplica automáticamente cambios independientes.
- 3. Conserva ambas variantes incompatibles.
- 4. La interfaz explica autores, fechas e impacto.
- 5. El actor elige o combina una resolución y crea una revisión.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                                  |
|--------------|----------------------------------------------------------------------------------------------------------|
| Alternativas | El propietario puede delegar la decisión a otro editor.                                                  |
| Excepciones  | Conflicto encadenado con una revisión nueva. Actor sin permiso de resolución.                            |
| Reglas       | Resolver nunca borra la versión anterior. Las fusiones automáticas deben ser deterministas y auditables. |

## 2.3.19 CU-19 · Administrar roles y permisos

Modificar el nivel de acceso de cada participante de forma coherente en REST, WebSocket y descargas.

| Campo      | Especificación                                     |
|------------|----------------------------------------------------|
| Actores    | A1 Ingeniero propietario                           |
| Prioridad  | ALTA                                               |
| Disparador | El propietario abre la administración de miembros. |
| Entradas   | Miembro, rol solicitado y confirmación.            |
| Salidas    | Política actualizada y evento de auditoría.        |
| Requisitos | RF-05, RF-21, RF-24                                |

### Flujo principal

- 1. La API devuelve miembros y roles actuales.
- 2. El propietario selecciona un participante.
- 3. Define OWNER, EDITOR, VIEWER o retiro.
- 4. El sistema valida que permanezca un propietario.
- 5. La política se actualiza e invalida sesiones incompatibles.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                      |
|--------------|------------------------------------------------------------------------------|
| Alternativas | La propiedad puede transferirse mediante confirmación doble.                 |
| Excepciones  | Intento de retirar al único propietario. Cambio concurrente de membresía.    |
| Reglas       | La autorización se valida en el servidor y no depende de controles visuales. |

## 2.3.20 CU-20 · Descargar, comparar o restaurar una revisión

Recuperar artefactos y analizar diferencias entre generaciones sin destruir el historial.

| Campo      | Especificación                                                         |
|------------|------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A3 Ingeniero observador |
| Prioridad  | ALTA                                                                   |
| Disparador | El actor selecciona una o dos revisiones del repositorio interno.      |
| Entradas   | Identificadores de revisión y acción.                                  |
| Salidas    | Diff, paquete descargable o revisión restaurada.                       |
| Requisitos | RF-13, RF-22, RF-23                                                    |

### Flujo principal

- 1. El sistema valida acceso a las revisiones.
- 2. Muestra metadatos y árbol de archivos.
- 3. Calcula diferencias textuales cuando se comparan.
- 4. Genera una descarga temporal si se solicita.
- 5. Restaurar crea una nueva revisión basada en la elegida.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                   |
|--------------|-------------------------------------------------------------------------------------------|
| Alternativas | Los archivos binarios se comparan mediante metadatos y huella.                            |
| Excepciones  | Revisión incompleta. Enlace temporal vencido o archivo no disponible.                     |
| Reglas       | Restaurar nunca modifica una revisión inmutable. Las descargas expiran y no son públicas. |

## 2.3.21 CU-21 · Gestionar cuenta, proyectos y diagramas desde Android

Permitir que los participantes completen desde Android el ciclo de acceso y administración disponible en la web.

| Campo      | Especificación                                                                                 |
|------------|------------------------------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A3 Ingeniero observador                         |
| Prioridad  | ALTA                                                                                           |
| Disparador | El usuario abre la aplicación Android y selecciona una función de cuenta, proyecto o diagrama. |
| Entradas   | Credenciales, perfil, preferencia y datos del recurso.                                         |
| Salidas    | Sesión, perfil o recurso actualizado y estado local coherente.                                 |
| Requisitos | RF-01, RF-02, RF-06, RF-09, RF-21, RF-25, RF-27                                                |

### Flujo principal

- 1. La aplicación recupera la sesión protegida o solicita autenticación.
- 2. El usuario registra o actualiza su perfil cuando corresponde.
- 3. La API devuelve proyectos y capacidades según el rol.
- 4. El usuario crea, configura, archiva o restaura recursos permitidos.
- 5. La aplicación actualiza su copia local y muestra el resultado confirmado.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                                                                                         |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Alternativas | Sin conexión se muestran datos sincronizados y solo se habilitan acciones offline admitidas. El usuario cambia idioma o tema sin afectar los datos compartidos. |
| Excepciones  | Sesión vencida. Recurso eliminado por otro participante. Rol insuficiente para la acción.                                                                       |
| Reglas       | Android conserva los mismos permisos y resultados del cliente web. Las preferencias locales no modifican la configuración del proyecto.                         |

## 2.3.22 CU-22 · Editar un diagrama UML táctil

Construir y modificar el modelo completo desde una interacción adaptada a pantallas táctiles.

| Campo      | Especificación                                                              |
|------------|-----------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor                               |
| Prioridad  | ALTA                                                                        |
| Disparador | El editor abre un diagrama en Android y selecciona el lienzo o un elemento. |
| Entradas   | Gestos, propiedades UML y versión base.                                     |
| Salidas    | Diagrama actualizado, operación durable y estado de sincronización.         |
| Requisitos | RF-03, RF-15, RF-16, RF-20, RF-26, RF-29, RF-32                             |

### Flujo principal

- 1. La aplicación carga la versión local confirmada y operaciones pendientes.
- 2. El usuario desplaza, amplía o selecciona elementos en el lienzo.
- 3. Crea o conecta clases mediante gestos visibles.
- 4. Edita atributos, métodos, relaciones y multiplicidades en formularios inferiores.
- 5. La operación se guarda localmente y se confirma con la API cuando existe conexión.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                                                                   |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Alternativas | El autoajuste ordena elementos sin cambiar su semántica. La IA local puede proponer una operación que el usuario revisa antes de aplicar. |
| Excepciones  | Nombre o tipo inválido. Relación con clase inexistente. Conflicto con una versión remota.                                                 |
| Reglas       | Toda operación usa identificadores estables y una versión base. Eliminar una clase o relación requiere confirmación explícita.            |

## 2.3.23 CU-23 · Invitar mediante enlace o código

Incorporar participantes desde web o Android sin depender únicamente de una dirección de correo.

| Campo      | Especificación                                                                         |
|------------|----------------------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A3 Ingeniero observador                 |
| Prioridad  | ALTA                                                                                   |
| Disparador | El propietario crea una invitación portable o un usuario introduce un código recibido. |
| Entradas   | Workspace, rol, vencimiento, correo opcional, token o código.                          |
| Salidas    | Membresía creada o rechazo auditable sin exponer secretos.                             |
| Requisitos | RF-05, RF-21, RF-24, RF-28                                                             |

### Flujo principal

- 1. El propietario define rol, vencimiento y correo opcional.
- 2. El servidor genera un token aleatorio y un código corto, y almacena únicamente sus huellas.
- 3. El propietario comparte el enlace o código.
- 4. El receptor autenticado abre el enlace o introduce el código.
- 5. La API valida estado, vencimiento, correo y rol antes de crear una membresía idempotente.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                                                                                |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Alternativas | El propietario revoca la invitación antes de usarla. El enlace profundo abre la aplicación o permite copiar el código manual.                          |
| Excepciones  | Código vencido, revocado, malformado o reutilizado. Correo autenticado diferente al destinatario. Usuario ya miembro.                                  |
| Reglas       | OWNER no se concede por invitación. La aceptación nunca duplica membresías ni eleva un rol existente. Los secretos no se incluyen en logs o auditoría. |

## 2.3.24 CU-24 · Generar y revisar artefactos desde Android

Ofrecer generación y revisión del repositorio interno con los mismos datos y controles de autorización de la web.

| Campo      | Especificación                                                         |
|------------|------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A3 Ingeniero observador |
| Prioridad  | ALTA                                                                   |
| Disparador | El participante abre Código o solicita una generación desde Android.   |
| Entradas   | Diagrama, tipo de proyecto, revisión, archivo o comentario.            |
| Salidas    | Revisión navegable, comparación, comentario, ZIP o restauración.       |
| Requisitos | RF-07, RF-10, RF-11, RF-12, RF-13, RF-19, RF-21, RF-22, RF-30          |

### Flujo principal

- 1. La aplicación fija diagrama y versión autorizados.
- 2. El actor solicita Spring Boot, Flutter o artefactos API si su rol lo permite.
- 3. La API publica la revisión y devuelve su manifiesto.
- 4. El usuario navega archivos, comentarios e historial.
- 5. Puede comparar, descargar o restaurar según sus capacidades.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                                                  |
|--------------|--------------------------------------------------------------------------------------------------------------------------|
| Alternativas | Un observador consulta y descarga sin modificar. Los archivos binarios muestran metadatos cuando no existe vista previa. |
| Excepciones  | Generación fallida. Archivo no disponible. Permiso retirado durante la operación.                                        |
| Reglas       | Las revisiones publicadas son inmutables. Restaurar crea una revisión nueva. Los comentarios no alteran el contenido.    |

## 2.3.25 CU-25 · Intercambiar y sincronizar trabajo desde Android

Importar, exportar, colaborar y resolver pendientes o conflictos desde el dispositivo sin perder trazabilidad.

| Campo      | Especificación                                                                                                           |
|------------|--------------------------------------------------------------------------------------------------------------------------|
| Actores    | A1 Ingeniero propietario, A2 Ingeniero editor, A3 Ingeniero observador, A4 Herramienta UML externa, A5 Motor de IA local |
| Prioridad  | ALTA                                                                                                                     |
| Disparador | El usuario selecciona un archivo, recupera conectividad o recibe una operación remota.                                   |
| Entradas   | Archivo, cola local, eventos remotos y resolución.                                                                       |
| Salidas    | Modelo importado/exportado, cola sincronizada o conflicto resuelto.                                                      |
| Requisitos | RF-04, RF-14, RF-15, RF-16, RF-17, RF-20, RF-24, RF-29, RF-31, RF-32                                                     |

### Flujo principal

- 1. La aplicación selecciona el proyecto y comprueba capacidades.
- 2. Importa con vista previa o exporta XMI, JSON o ZIP mediante el sistema de archivos.
- 3. Las operaciones confirmadas se distribuyen por el canal colaborativo.
- 4. Al reconectar se reproducen novedades y se envían pendientes en orden.
- 5. Los conflictos muestran base, variante local, variante remota, autor y versión antes de confirmar una resolución.

#### Precondiciones

El actor dispone de una sesión válida cuando la operación requiere autenticación. El proyecto y sus recursos se encuentran accesibles con el rol necesario.

#### Postcondiciones

El resultado queda persistido o registrado como operación pendiente según la conectividad. La auditoría conserva actor, fecha, proyecto y versión afectada.

| Escenario    | Detalle                                                                                                                                                              |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Alternativas | Cambios independientes se fusionan automáticamente. El usuario conserva un conflicto para revisarlo más tarde. Un archivo exportado se comparte con otra aplicación. |
| Excepciones  | Archivo incompatible. Almacenamiento local lleno. Sesión no renovable. Conflicto encadenado.                                                                         |
| Reglas       | Ninguna operación pendiente se elimina antes de confirmación. Las importaciones no ejecutan contenido recibido. Resolver conserva ambas variantes en la auditoría.   |

## 2.4 Estructura del modelo de casos de uso

Organización por objetivos y dependencias principales.

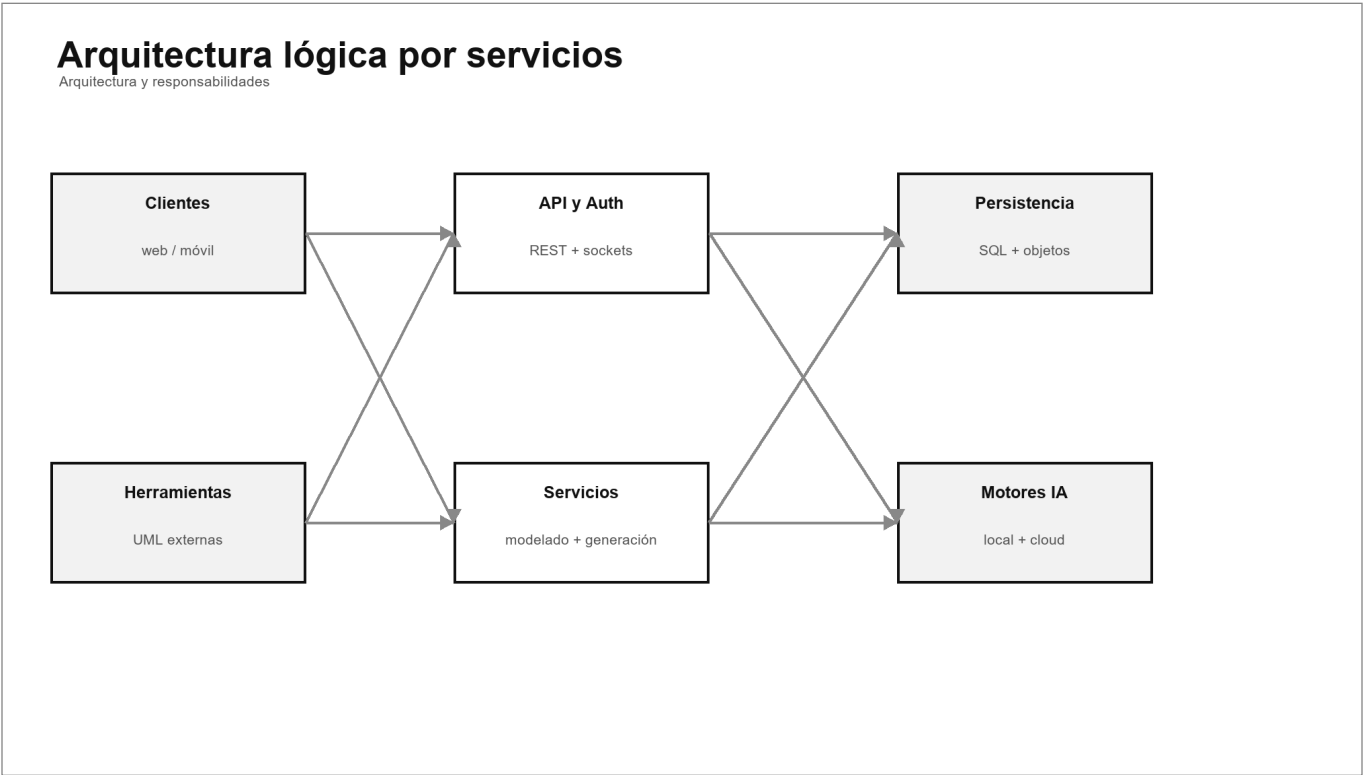
| Grupo                     | Casos                             | Resultado                                                          |
|---------------------------|-----------------------------------|--------------------------------------------------------------------|
| Acceso y administración   | CU-01, CU-02, CU-05, CU-09, CU-19 | Proyecto protegido y participantes controlados.                    |
| Modelado                  | CU-03, CU-04, CU-06, CU-08        | Modelo UML colaborativo y trazable.                                |
| Generación                | CU-07, CU-10, CU-11, CU-16        | Código y pruebas vinculados a una versión.                         |
| Repositorio               | CU-12, CU-17, CU-20               | Historial navegable y recuperable.                                 |
| Móvil e interoperabilidad | CU-13, CU-14, CU-15, CU-18        | Trabajo multimodal, portable y sincronizable.                      |
| Paridad funcional Android | CU-21 a CU-25                     | Administración, editor táctil, invitaciones, código e intercambio. |

### Dependencias

CU-07, CU-10, CU-11 y CU-16 consumen una versión válida de CU-03. CU-12 conserva sus resultados. CU-15 puede conducir a CU-18 cuando la versión base diverge. CU-19 controla las autorizaciones. CU-21 a CU-25 adaptan esos resultados al canal Android sin duplicar las reglas del servidor.

### 3.1 Análisis de la arquitectura

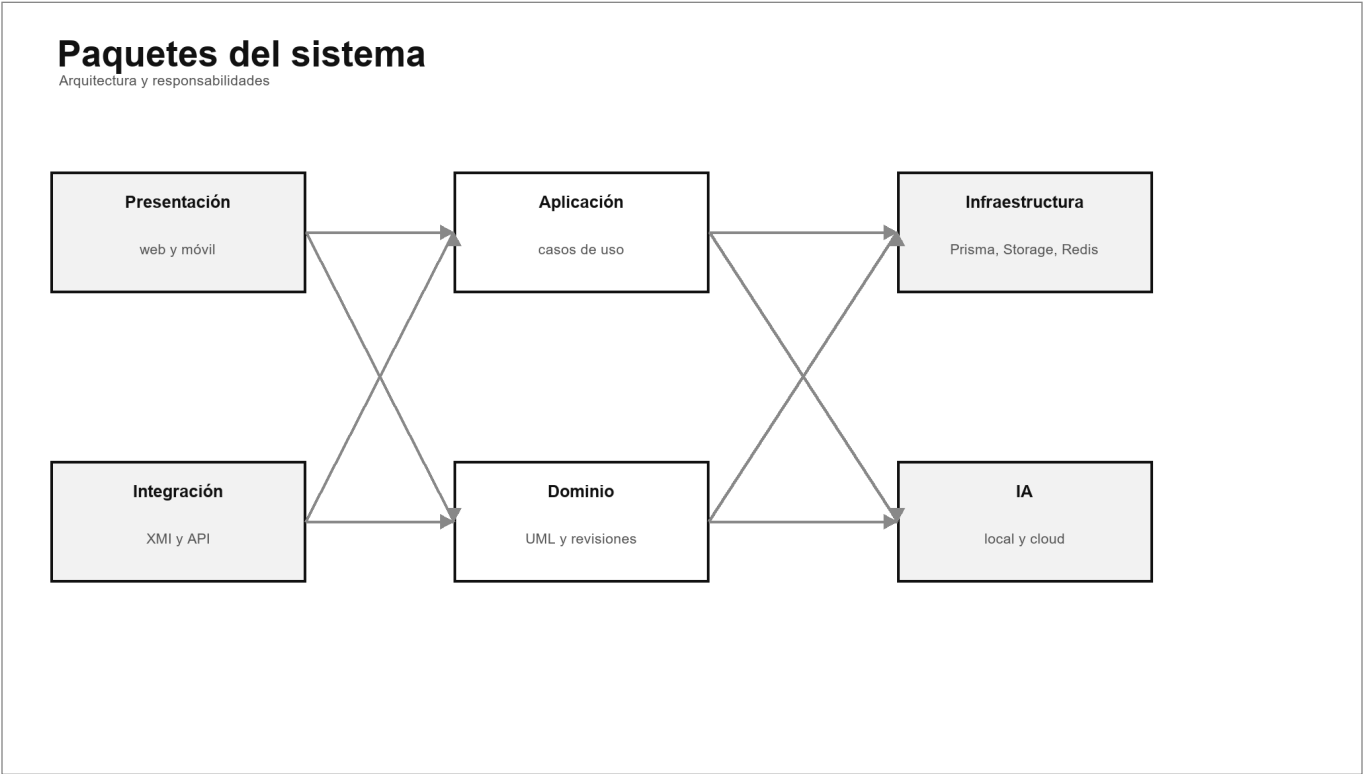
El análisis separa clientes, API, servicios, persistencia e IA para asignar responsabilidades sin atarlas todavía a una instancia física.



| Criterio        | Decisión                                                                                                                                  |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad | El análisis separa clientes, API, servicios, persistencia e IA para asignar responsabilidades sin atarlas todavía a una instancia física. |
| Seguridad       | La autorización se valida en el servidor para REST, sockets, sincronización y descargas.                                                  |
| Trazabilidad    | Toda revisión y operación conserva proyecto, autor, versión base y resultado.                                                             |

### 3.2 Relación entre paquetes y casos de uso

Los casos de uso se distribuyen entre presentación, aplicación, dominio, infraestructura, integración e inteligencia artificial.



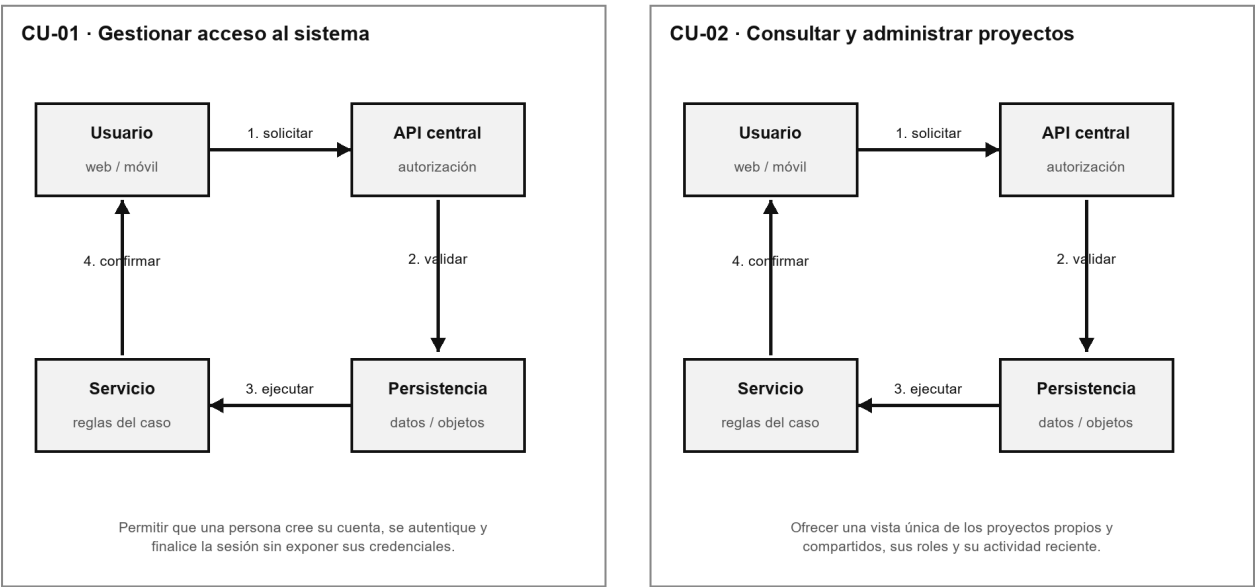
| Criterio        | Decisión                                                                                                                         |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad | Los casos de uso se distribuyen entre presentación, aplicación, dominio, infraestructura, integración e inteligencia artificial. |
| Seguridad       | La autorización se valida en el servidor para REST, sockets, sincronización y descargas.                                         |
| Trazabilidad    | Toda revisión y operación conserva proyecto, autor, versión base y resultado.                                                    |

### 3.3 Comunicación CU-01 y CU-02

Interacciones reconstruidas entre límites, control y persistencia.

#### Comunicación CU-01 y CU-02

Diagrama de colaboración / comunicación



| CU    | Servicio dominante                | Resultado                                       |
|-------|-----------------------------------|-------------------------------------------------|
| CU-01 | Gestionar acceso al sistema       | Cuenta creada, token de sesión y perfil mínimo. |
| CU-02 | Consultar y administrar proyectos | Listado autorizado y navegación al proyecto.    |

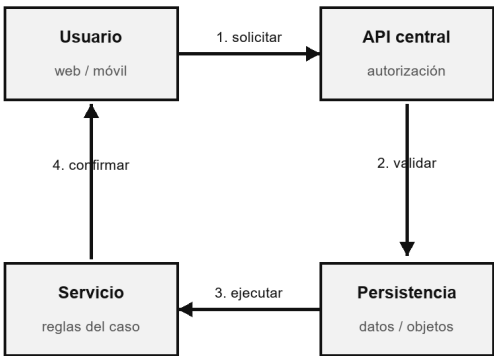
### 3.3 Comunicación CU-03 y CU-04

Interacciones reconstruidas entre límites, control y persistencia.

#### Comunicación CU-03 y CU-04

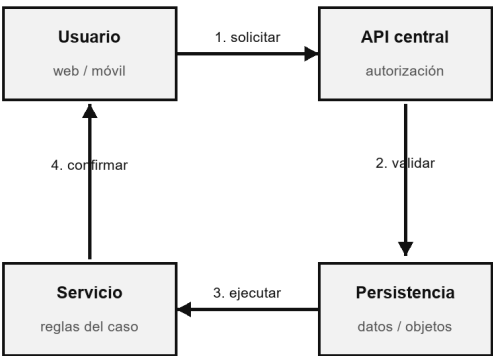
Diagrama de colaboración / comunicación

##### CU-03 · Construir el modelo UML del dominio



Representar entidades del negocio mediante clases UML, atributos, operaciones y relaciones editables.

##### CU-04 · Sincronizar el diagrama colaborativo



Propagar ediciones del lienzo entre participantes conectados al mismo proyecto.

| CU    | Servicio dominante                   | Resultado                               |
|-------|--------------------------------------|-----------------------------------------|
| CU-03 | Construir el modelo UML del dominio  | Nueva versión del modelo conceptual.    |
| CU-04 | Sincronizar el diagrama colaborativo | Diagrama convergente y evento auditado. |

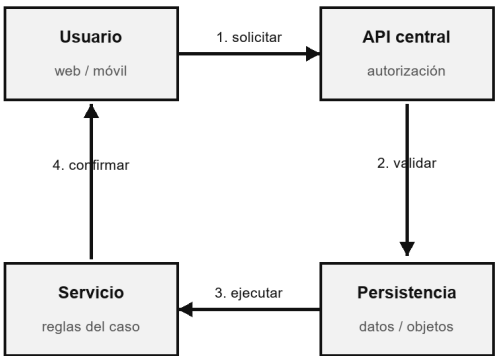
### 3.3 Comunicación CU-05 y CU-06

Interacciones reconstruidas entre límites, control y persistencia.

#### Comunicación CU-05 y CU-06

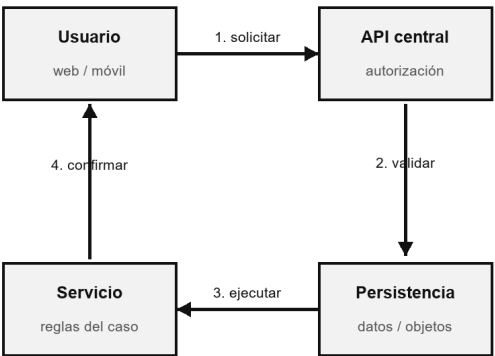
Diagrama de colaboración / comunicación

##### CU-05 · Incorporar participantes al proyecto



Invitar a otros ingenieros mediante correo y asignar un rol inicial controlado.

##### CU-06 · Archivar o retirar diagramas



Ocultar un diagrama obsoleto de las vistas activas conservando historial y auditoría.

| CU    | Servicio dominante                   | Resultado                          |
|-------|--------------------------------------|------------------------------------|
| CU-05 | Incorporar participantes al proyecto | Membresía o invitación registrada. |
| CU-06 | Archivar o retirar diagramas         | Estado archivado o reactivado.     |

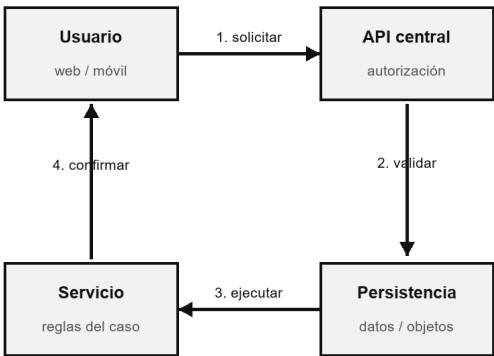
### 3.3 Comunicación CU-07 y CU-08

Interacciones reconstruidas entre límites, control y persistencia.

#### Comunicación CU-07 y CU-08

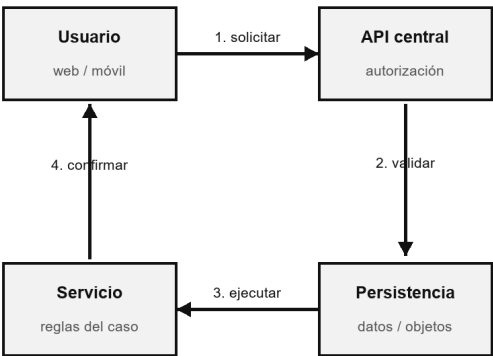
Diagrama de colaboración / comunicación

##### CU-07 · Generar backend Spring Boot por reglas



Transformar un modelo UML válido en un proyecto backend reproducible mediante plantillas deterministas.

##### CU-08 · Construir un modelo con asistencia inteligente



Convertir una descripción del dominio en una propuesta UML editable y validada por el usuario.

| CU    | Servicio dominante                             | Resultado                                      |
|-------|------------------------------------------------|------------------------------------------------|
| CU-07 | Generar backend Spring Boot por reglas         | Proyecto Spring Boot, manifiesto y ZIP.        |
| CU-08 | Construir un modelo con asistencia inteligente | Propuesta UML validada y diferencias visibles. |

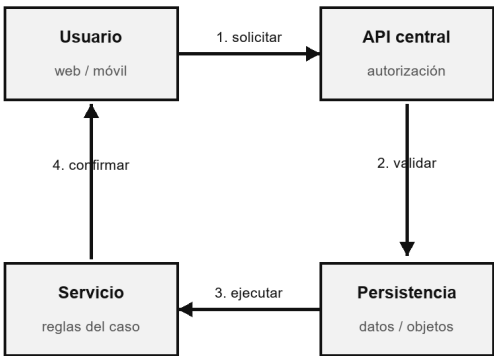
### 3.3 Comunicación CU-09 y CU-10

Interacciones reconstruidas entre límites, control y persistencia.

#### Comunicación CU-09 y CU-10

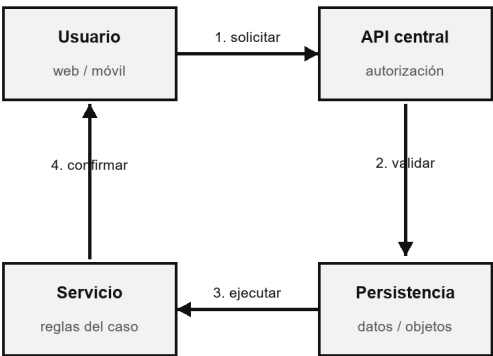
Diagrama de colaboración / comunicación

##### CU-09 · Crear y configurar un espacio de trabajo



Establecer el contenedor principal para modelos, miembros, código y decisiones del proyecto.

##### CU-10 · Producir una colección de pruebas API



Derivar solicitudes y verificaciones básicas de los endpoints generados.

| CU    | Servicio dominante                       | Resultado                                 |
|-------|------------------------------------------|-------------------------------------------|
| CU-09 | Crear y configurar un espacio de trabajo | Workspace listo para modelado.            |
| CU-10 | Producir una colección de pruebas API    | Colección de pruebas y variables seguras. |

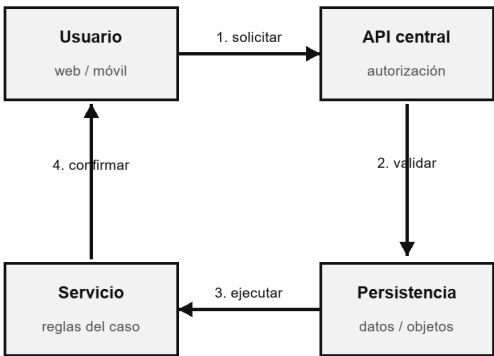
### 3.3 Comunicación CU-11 y CU-12

Interacciones reconstruidas entre límites, control y persistencia.

#### Comunicación CU-11 y CU-12

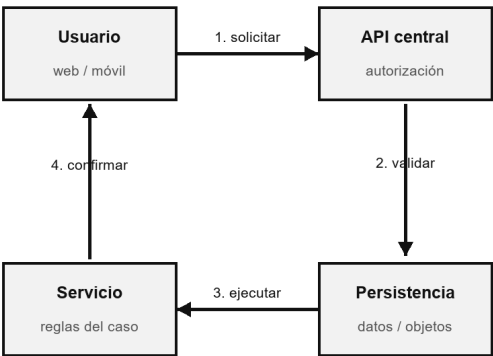
Diagrama de colaboración / comunicación

##### CU-11 · Generar una aplicación Flutter



Crear una base móvil multiplataforma coherente con los recursos expuestos por el backend.

##### CU-12 · Gestionar el repositorio interno de código



Centralizar archivos y revisiones generadas dentro del proyecto sin exigir conocimientos de Git.

| CU    | Servicio dominante                         | Resultado                           |
|-------|--------------------------------------------|-------------------------------------|
| CU-11 | Generar una aplicación Flutter             | Proyecto Flutter, manifiesto y ZIP. |
| CU-12 | Gestionar el repositorio interno de código | Revisión navegable del código.      |

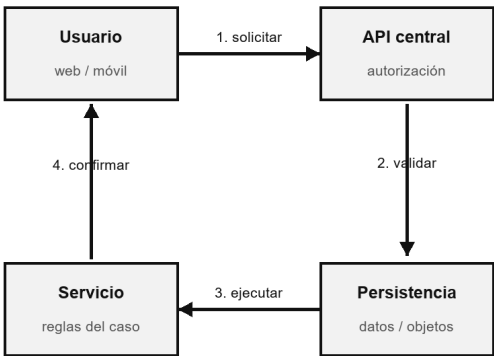
### 3.3 Comunicación CU-13 y CU-14

Interacciones reconstruidas entre límites, control y persistencia.

#### Comunicación CU-13 y CU-14

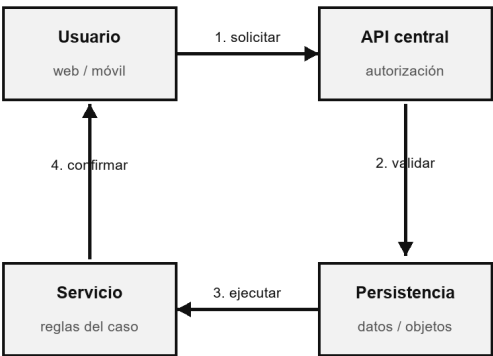
Diagrama de colaboración / comunicación

##### CU-13 · Interactuar con la IA por texto o voz



Recibir instrucciones escritas o habladas y ejecutarlas mediante el motor apropiado.

##### CU-14 · Importar y exportar modelos UML



Intercambiar la semántica del modelo con herramientas externas compatibles.

| CU    | Servicio dominante                    | Resultado                                              |
|-------|---------------------------------------|--------------------------------------------------------|
| CU-13 | Interactuar con la IA por texto o voz | Respuesta, función propuesta y resultado estructurado. |
| CU-14 | Importar y exportar modelos UML       | Modelo importado o paquete exportado con informe.      |

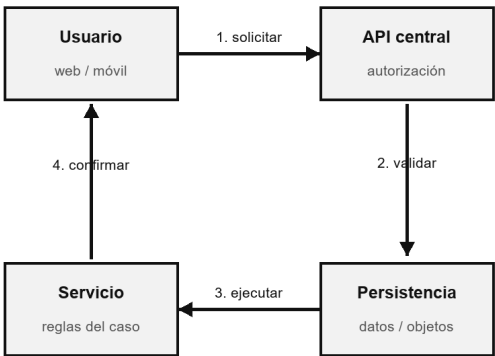
### 3.3 Comunicación CU-15 y CU-16

Interacciones reconstruidas entre límites, control y persistencia.

#### Comunicación CU-15 y CU-16

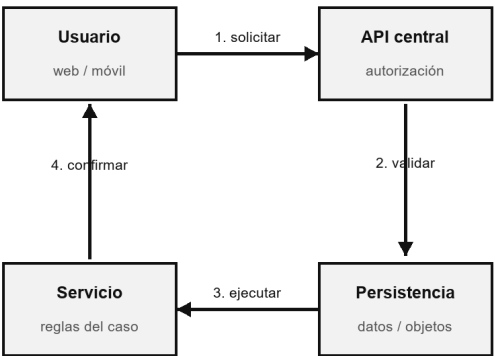
Diagrama de colaboración / comunicación

##### CU-15 · Trabajar offline y sincronizar cambios



Permitir edición móvil básica sin red y enviar operaciones pendientes cuando vuelva la conexión.

##### CU-16 · Generar o refinar backend con IA



Usar el diagrama y una instrucción adicional para completar decisiones o mejorar una generación backend.

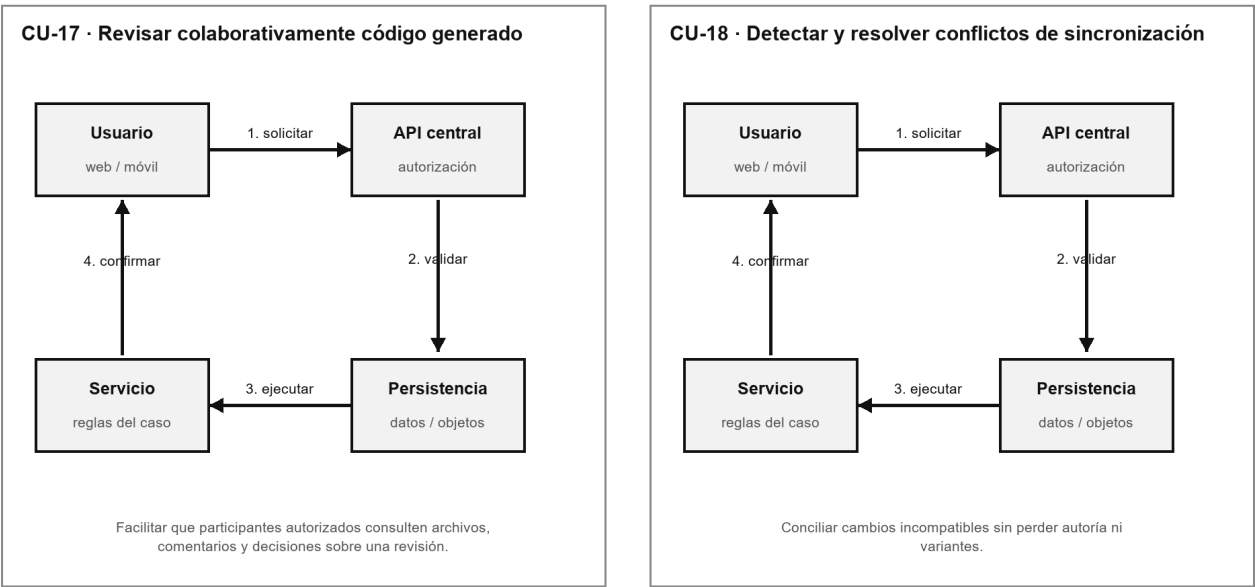
| CU    | Servicio dominante                     | Resultado                                           |
|-------|----------------------------------------|-----------------------------------------------------|
| CU-15 | Trabajar offline y sincronizar cambios | Proyecto sincronizado o conflicto explícito.        |
| CU-16 | Generar o refinar backend con IA       | Propuesta, diferencias y nueva revisión de backend. |

### 3.3 Comunicación CU-17 y CU-18

Interacciones reconstruidas entre límites, control y persistencia.

#### Comunicación CU-17 y CU-18

Diagrama de colaboración / comunicación



| CU    | Servicio dominante                               | Resultado                                  |
|-------|--------------------------------------------------|--------------------------------------------|
| CU-17 | Revisar colaborativamente código generado        | Hilo de revisión y actividad compartida.   |
| CU-18 | Detectar y resolver conflictos de sincronización | Revisión conciliada o conflicto pendiente. |

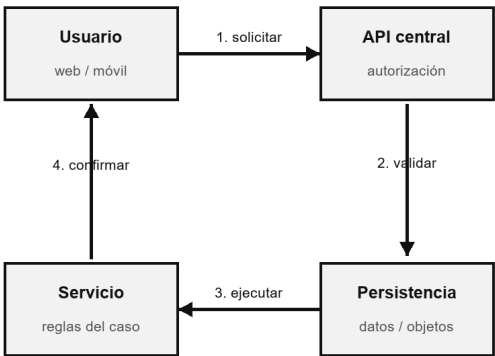
### 3.3 Comunicación CU-19 y CU-20

Interacciones reconstruidas entre límites, control y persistencia.

#### Comunicación CU-19 y CU-20

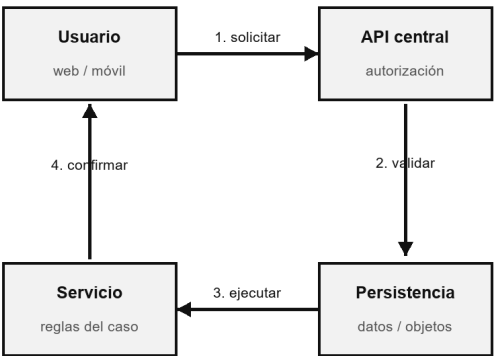
Diagrama de colaboración / comunicación

##### CU-19 · Administrar roles y permisos



Modificar el nivel de acceso de cada participante de forma coherente en REST, WebSocket y descargas.

##### CU-20 · Descargar, comparar o restaurar una revisión



Recuperar artefactos y analizar diferencias entre generaciones sin destruir el historial.

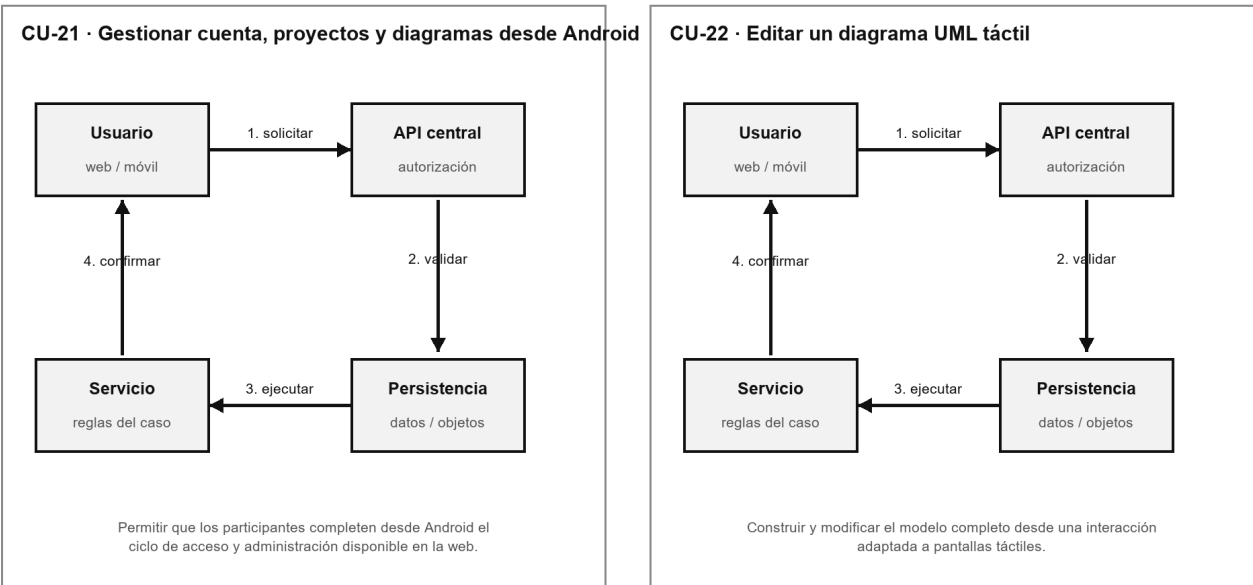
| CU    | Servicio dominante                           | Resultado                                        |
|-------|----------------------------------------------|--------------------------------------------------|
| CU-19 | Administrar roles y permisos                 | Política actualizada y evento de auditoría.      |
| CU-20 | Descargar, comparar o restaurar una revisión | Diff, paquete descargable o revisión restaurada. |

### 3.3 Comunicación CU-21 y CU-22

Interacciones reconstruidas entre límites, control y persistencia.

#### Comunicación CU-21 y CU-22

Diagrama de colaboración / comunicación



| CU    | Servicio dominante                                    | Resultado                                                           |
|-------|-------------------------------------------------------|---------------------------------------------------------------------|
| CU-21 | Gestionar cuenta, proyectos y diagramas desde Android | Sesión, perfil o recurso actualizado y estado local coherente.      |
| CU-22 | Editar un diagrama UML táctil                         | Diagrama actualizado, operación durable y estado de sincronización. |

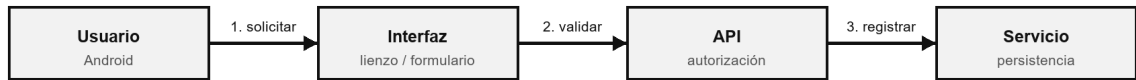
### 3.3 Comunicación CU-23 a CU-25

Interacciones reconstruidas entre límites, control y persistencia.

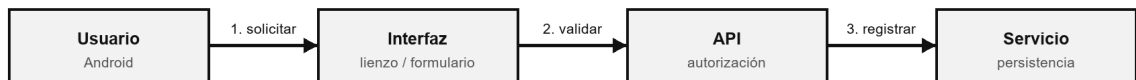
#### Comunicación CU-23 a CU-25

Diagrama de colaboración / comunicación

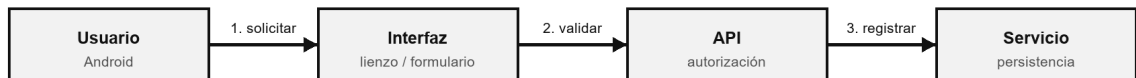
##### CU-23 · Invitar mediante enlace o código



##### CU-24 · Generar y revisar artefactos desde Android



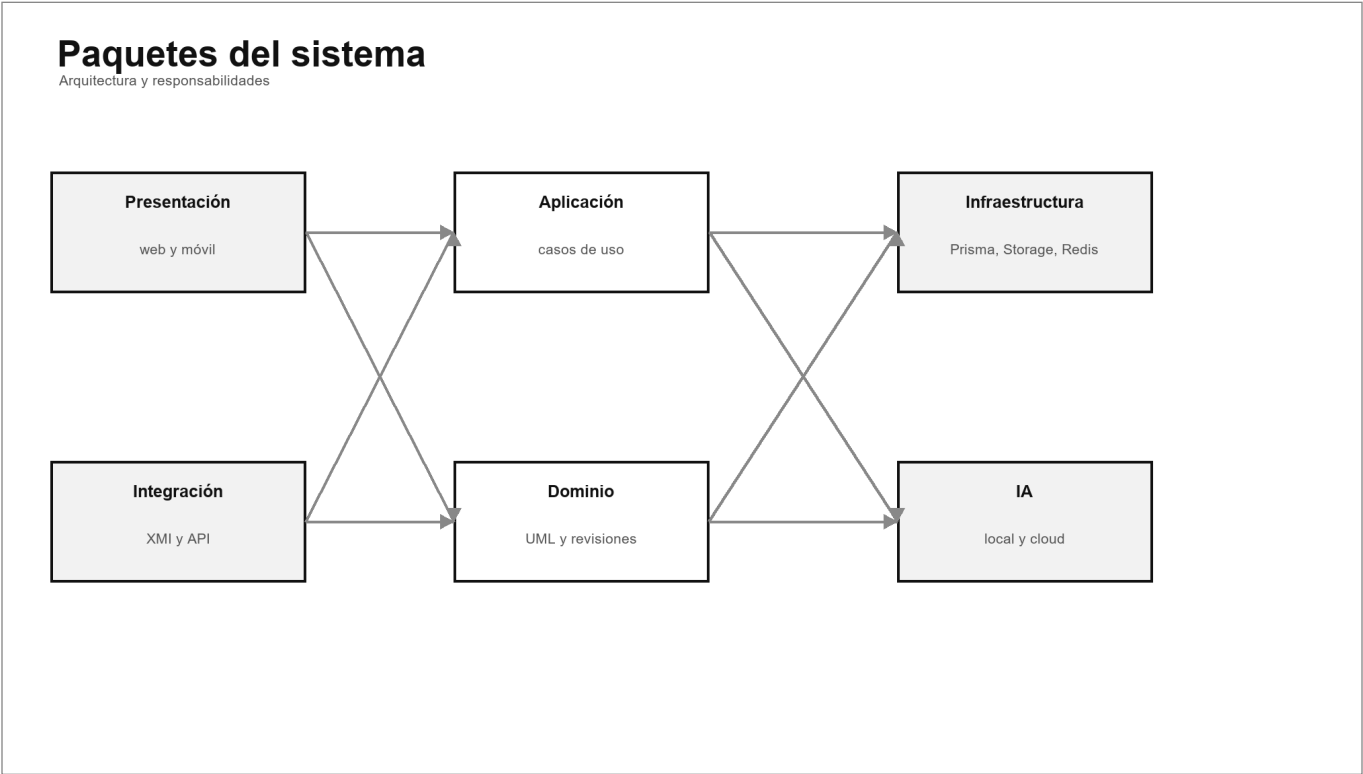
##### CU-25 · Intercambiar y sincronizar trabajo desde Android



| CU    | Servicio dominante                               | Resultado                                                           |
|-------|--------------------------------------------------|---------------------------------------------------------------------|
| CU-23 | Invitar mediante enlace o código                 | Membresía creada o rechazo auditable sin exponer secretos.          |
| CU-24 | Generar y revisar artefactos desde Android       | Revisión navegable, comparación, comentario, ZIP o restauración.    |
| CU-25 | Intercambiar y sincronizar trabajo desde Android | Modelo importado/exportado, cola sincronizada o conflicto resuelto. |

### 3.4 Cohesión y dependencias de paquetes

Las dependencias apuntan hacia contratos estables. Presentación e infraestructura no contienen reglas centrales del dominio.



| Criterio        | Decisión                                                                                                                     |
|-----------------|------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad | Las dependencias apuntan hacia contratos estables. Presentación e infraestructura no contienen reglas centrales del dominio. |
| Seguridad       | La autorización se valida en el servidor para REST, sockets, sincronización y descargas.                                     |
| Trazabilidad    | Toda revisión y operación conserva proyecto, autor, versión base y resultado.                                                |

## 4.1 Diseño físico para Google Cloud

Topología de lanzamiento preparada; recursos remotos aún no creados.

| Criterio  | Decisión de primera etapa                                                               |
|-----------|-----------------------------------------------------------------------------------------|
| Ejecución | Una VM Compute Engine con frontend Next.js, API NestJS y Socket.IO.                     |
| Datos     | PostgreSQL local y archivos de revisiones en Persistent Disk, fuera del checkout.       |
| Acceso    | Caddy termina HTTPS; solo 80/443 públicos; PostgreSQL y servicios internos en loopback. |
| Respaldo  | pg_dump y snapshots del disco con prueba periódica de restauración.                     |
| Evolución | Cloud Run exige almacenamiento externo y coordinación de eventos multiinstancia.        |

Esta topología prioriza una publicación reproducible sin fingir escalado horizontal.

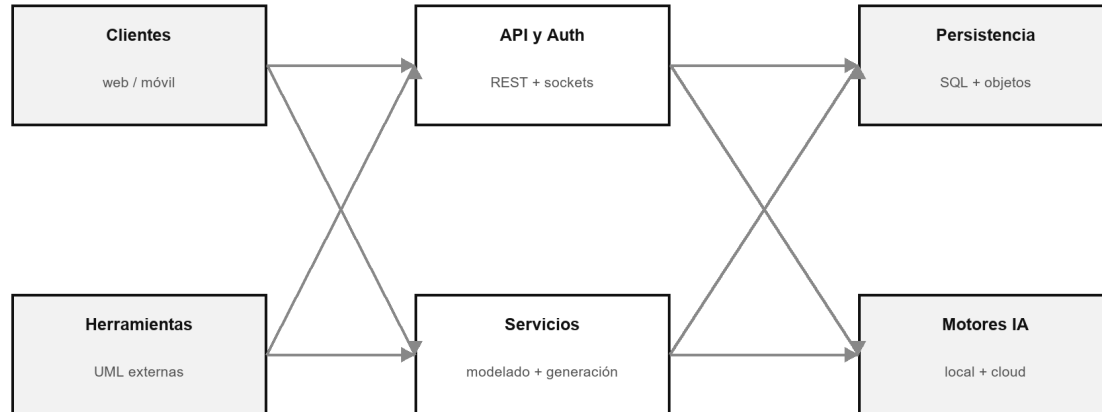
La clave Gemini probada recibe 403; la verificación del proveedor es una puerta de salida independiente.

## 4.2 Diseño lógico por servicios

La API coordina autenticación, modelado, colaboración, generación, repositorio, sincronización e interoperabilidad.

### Arquitectura lógica por servicios

Arquitectura y responsabilidades



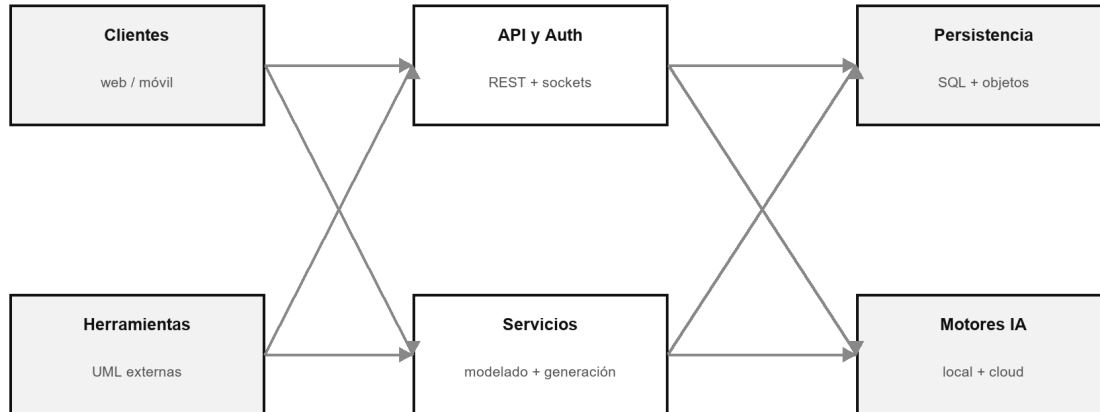
| Criterio        | Decisión                                                                                                            |
|-----------------|---------------------------------------------------------------------------------------------------------------------|
| Responsabilidad | La API coordina autenticación, modelado, colaboración, generación, repositorio, sincronización e interoperabilidad. |
| Seguridad       | La autorización se valida en el servidor para REST, sockets, sincronización y descargas.                            |
| Trazabilidad    | Toda revisión y operación conserva proyecto, autor, versión base y resultado.                                       |

## 4.3 Diseño de componentes

Los contratos REST, WebSocket y de almacenamiento permiten escalar o sustituir componentes con menor acoplamiento.

### Componentes y dependencias

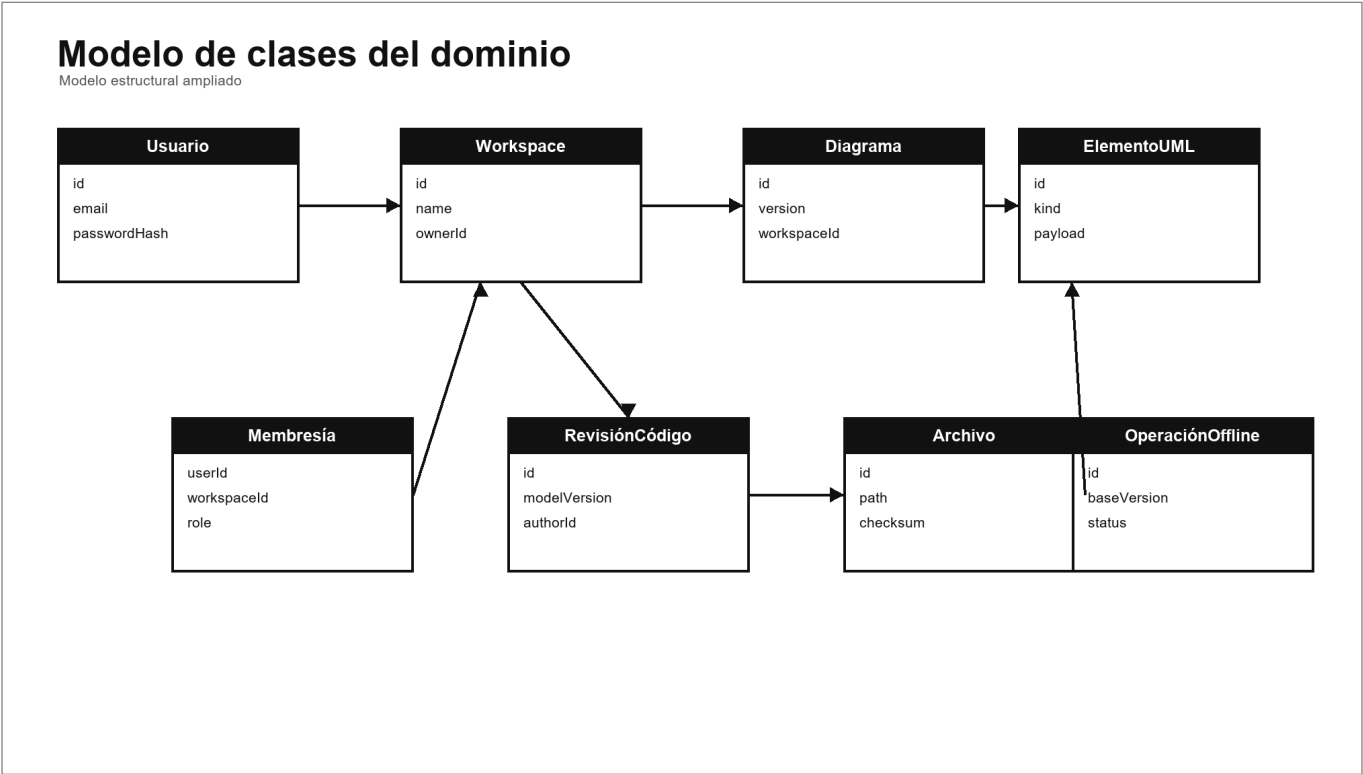
Arquitectura y responsabilidades



| Criterio        | Decisión                                                                                                           |
|-----------------|--------------------------------------------------------------------------------------------------------------------|
| Responsabilidad | Los contratos REST, WebSocket y de almacenamiento permiten escalar o sustituir componentes con menor acoplamiento. |
| Seguridad       | La autorización se valida en el servidor para REST, sockets, sincronización y descargas.                           |
| Trazabilidad    | Toda revisión y operación conserva proyecto, autor, versión base y resultado.                                      |

# 4.4 Modelo de clases del dominio

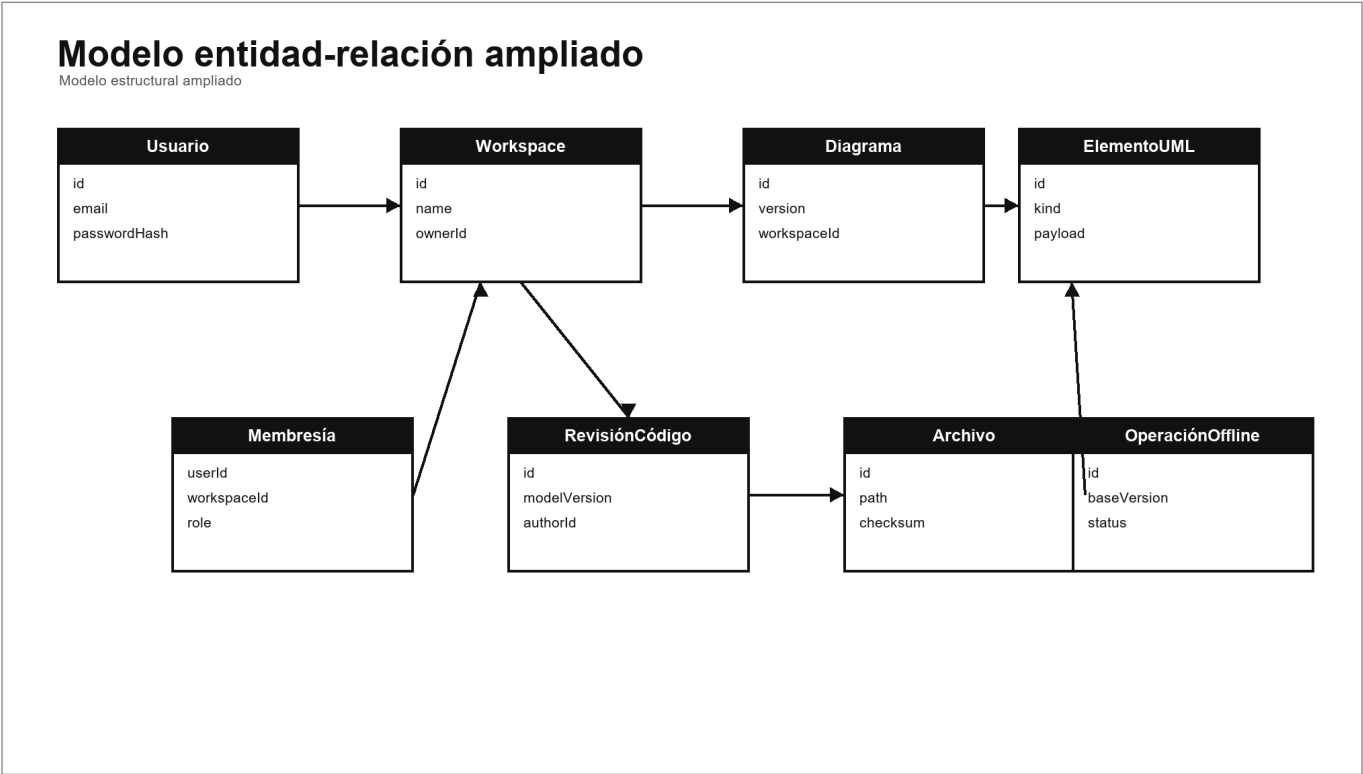
El dominio amplía usuarios y diagramas con membresías, revisiones, archivos y operaciones offline.



| Criterio        | Decisión                                                                                           |
|-----------------|----------------------------------------------------------------------------------------------------|
| Responsabilidad | El dominio amplía usuarios y diagramas con membresías, revisiones, archivos y operaciones offline. |
| Seguridad       | La autorización se valida en el servidor para REST, sockets, sincronización y descargas.           |
| Trazabilidad    | Toda revisión y operación conserva proyecto, autor, versión base y resultado.                      |

## 4.5 Modelo entidad-relación

La persistencia relacional protege integridad y auditoría; el contenido pesado se referencia desde almacenamiento de objetos.



| Criterio        | Decisión                                                                                                                      |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------|
| Responsabilidad | La persistencia relacional protege integridad y auditoría; el contenido pesado se referencia desde almacenamiento de objetos. |
| Seguridad       | La autorización se valida en el servidor para REST, sockets, sincronización y descargas.                                      |
| Trazabilidad    | Toda revisión y operación conserva proyecto, autor, versión base y resultado.                                                 |

## 4.6 Mapeo lógico-físico

Correspondencia comprobada con el esquema Prisma actual.

| Concepto                     | Persistencia real                                                        | Observación                       |
|------------------------------|--------------------------------------------------------------------------|-----------------------------------|
| Usuarios y proyectos         | users / workspaces                                                       | Propiedad y acceso por rol.       |
| Participantes e invitaciones | workspace_collaborators / workspace_invitations                          | Roles, expiración y revocación.   |
| Modelo UML                   | diagrams / uml_classes / uml_attributes /<br>uml_methods / uml_relations | Estado y elementos editables.     |
| Operaciones y conflictos     | diagram_operations / sync_conflicts                                      | Secuencia durable y variantes.    |
| Código generado              | generated_codes / code_revisions / revision_files                        | Revisiones inmutables y archivos. |
| Auditoría                    | diagram_activities / audit_events                                        | Autor y momento del evento.       |

La cola de cambios pendientes permanece en el almacenamiento local de Android; el servidor registra las operaciones confirmadas en diagram\_operations. Los artefactos se guardan en el directorio persistente configurado por la API. Cloud SQL y Cloud Storage son alternativas futuras, no el estado desplegado.

## 4.7 Diseño físico - extracto verificable

Restricciones relevantes del esquema Prisma y migraciones del proyecto.

| Tabla              | Restricción vigente               | Propósito                                |
|--------------------|-----------------------------------|------------------------------------------|
| diagram_operations | UNIQUE(deviceId, clientSequence)  | Evita repetir una operación móvil o web. |
| diagram_operations | UNIQUE(diagramId, serverSequence) | Ordena los eventos del diagrama.         |
| revision_files     | UNIQUE(revisionId, path)          | Mantiene una ruta única por revisión.    |
| code_revisions     | Índices por proyecto y diagrama   | Permite consultar el historial.          |
| sync_conflicts     | Índices por estado y diagrama     | Facilita la revisión de variantes.       |

La estructura ejecutable se define en backend/prisma/schema.prisma y en las migraciones versionadas.

Las sentencias SQL de una edición anterior eran un bosquejo conceptual, no migraciones aplicables.

Antes de desplegar se ejecuta prisma migrate deploy contra PostgreSQL con respaldo previo.

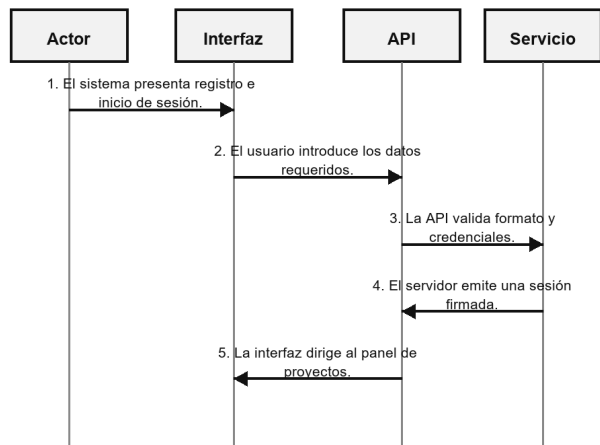
# 4.8 Secuencia CU-01 y CU-02

Orden temporal resumido para los casos de uso relacionados.

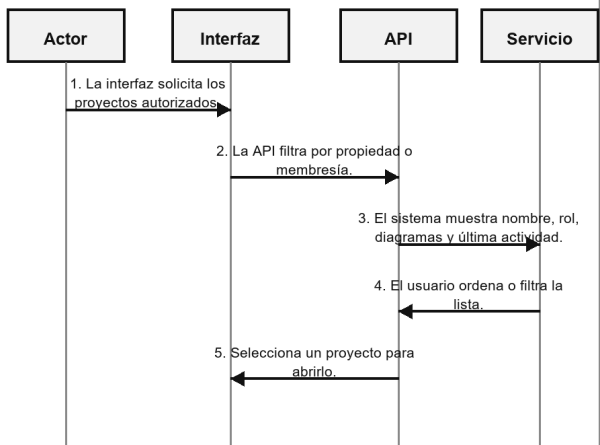
## Secuencia CU-01 y CU-02

Diagrama de secuencia resumido

### CU-01 · Gestionar acceso al sistema



### CU-02 · Consultar y administrar proyectos



### Interpretación

Las líneas verticales representan participantes y las flechas numeradas expresan mensajes. Las validaciones de permisos ocurren en la API incluso si la interfaz oculta una acción.

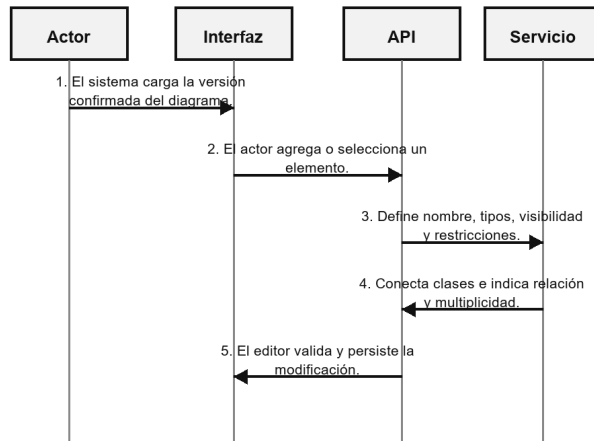
## 4.8 Secuencia CU-03 y CU-04

Orden temporal resumido para los casos de uso relacionados.

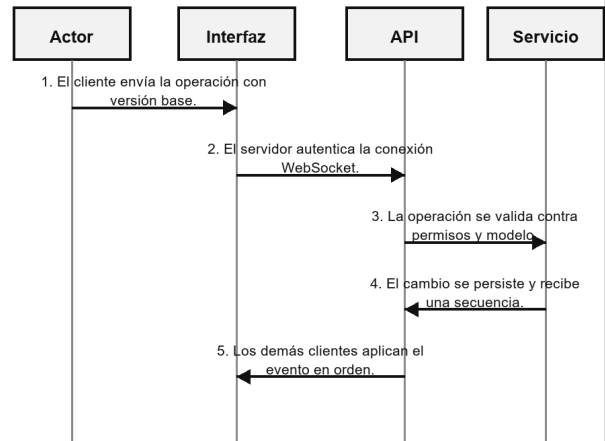
### Secuencia CU-03 y CU-04

Diagrama de secuencia resumido

#### CU-03 · Construir el modelo UML del dominio



#### CU-04 · Sincronizar el diagrama colaborativo



#### Interpretación

Las líneas verticales representan participantes y las flechas numeradas expresan mensajes. Las validaciones de permisos ocurren en la API incluso si la interfaz oculta una acción.

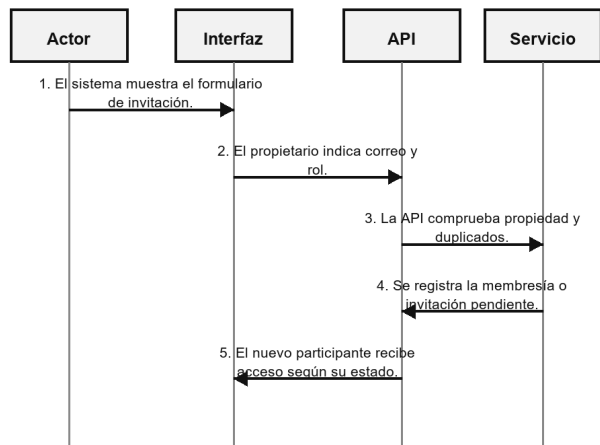
# 4.8 Secuencia CU-05 y CU-06

Orden temporal resumido para los casos de uso relacionados.

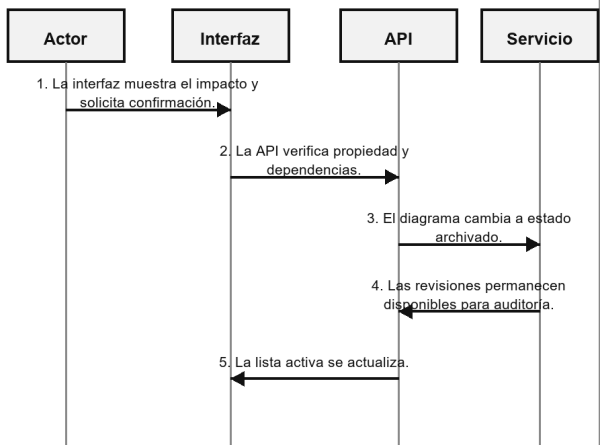
## Secuencia CU-05 y CU-06

Diagrama de secuencia resumido

### CU-05 · Incorporar participantes al proyecto



### CU-06 · Archivar o retirar diagramas



### Interpretación

Las líneas verticales representan participantes y las flechas numeradas expresan mensajes. Las validaciones de permisos ocurren en la API incluso si la interfaz oculta una acción.

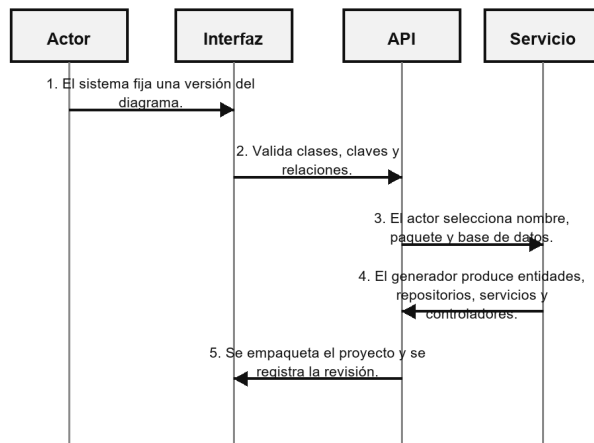
## 4.8 Secuencia CU-07 y CU-08

Orden temporal resumido para los casos de uso relacionados.

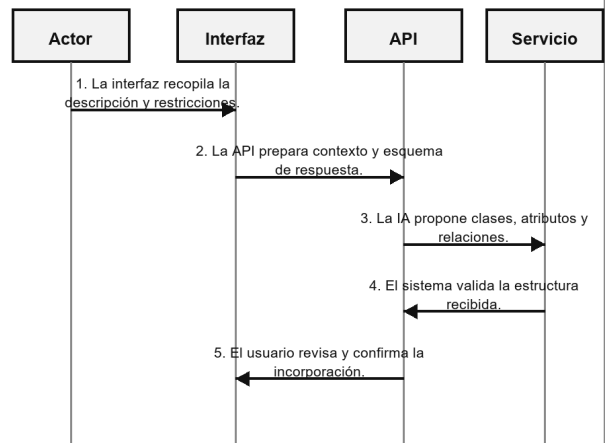
### Secuencia CU-07 y CU-08

Diagrama de secuencia resumido

#### CU-07 · Generar backend Spring Boot por reglas



#### CU-08 · Construir un modelo con asistencia inteligente



#### Interpretación

Las líneas verticales representan participantes y las flechas numeradas expresan mensajes. Las validaciones de permisos ocurren en la API incluso si la interfaz oculta una acción.

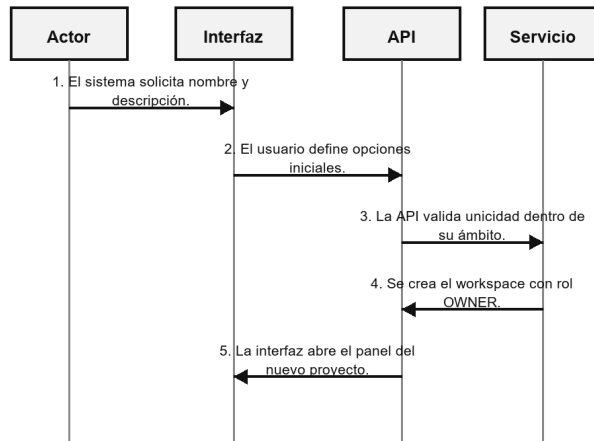
## 4.8 Secuencia CU-09 y CU-10

Orden temporal resumido para los casos de uso relacionados.

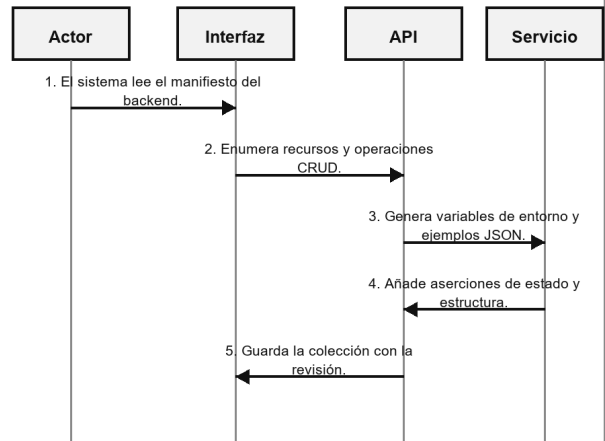
### Secuencia CU-09 y CU-10

Diagrama de secuencia resumido

#### CU-09 · Crear y configurar un espacio de trabajo



#### CU-10 · Producir una colección de pruebas API



#### Interpretación

Las líneas verticales representan participantes y las flechas numeradas expresan mensajes. Las validaciones de permisos ocurren en la API incluso si la interfaz oculta una acción.

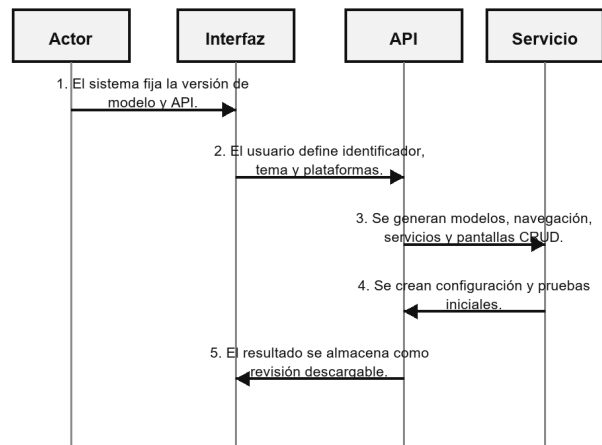
# 4.8 Secuencia CU-11 y CU-12

Orden temporal resumido para los casos de uso relacionados.

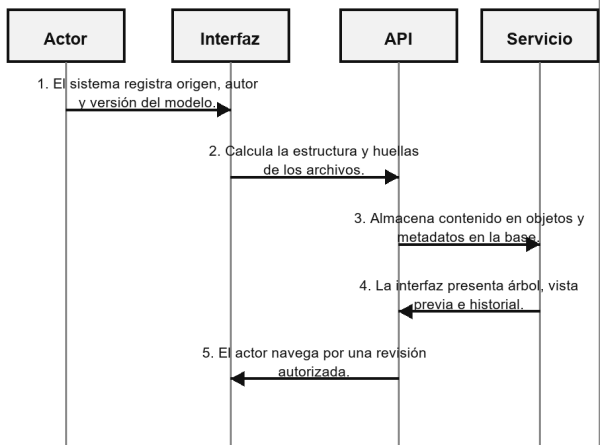
## Secuencia CU-11 y CU-12

Diagrama de secuencia resumido

### CU-11 · Generar una aplicación Flutter



### CU-12 · Gestionar el repositorio interno de código



### Interpretación

Las líneas verticales representan participantes y las flechas numeradas expresan mensajes. Las validaciones de permisos ocurren en la API incluso si la interfaz oculta una acción.

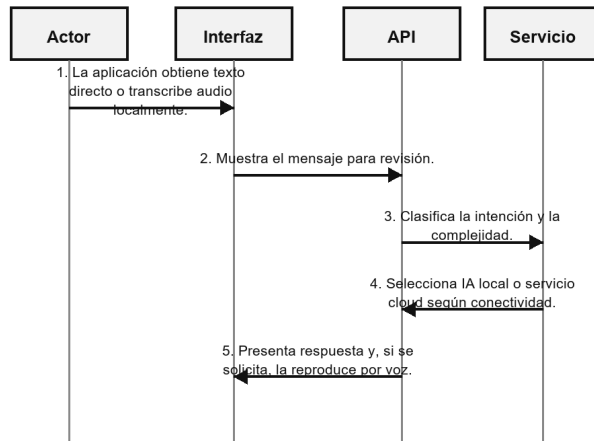
## 4.8 Secuencia CU-13 y CU-14

Orden temporal resumido para los casos de uso relacionados.

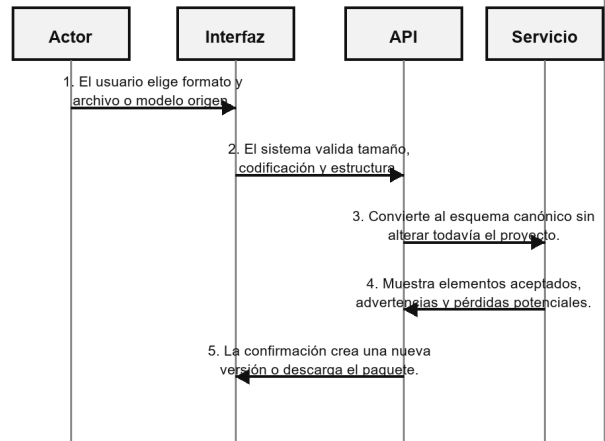
### Secuencia CU-13 y CU-14

Diagrama de secuencia resumido

#### CU-13 · Interactuar con la IA por texto o voz



#### CU-14 · Importar y exportar modelos UML



#### Interpretación

Las líneas verticales representan participantes y las flechas numeradas expresan mensajes. Las validaciones de permisos ocurren en la API incluso si la interfaz oculta una acción.

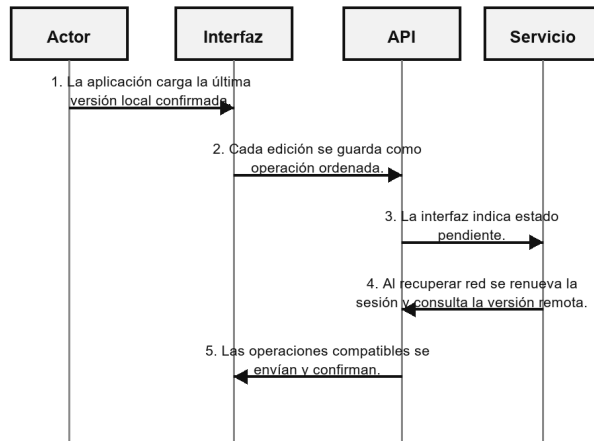
## 4.8 Secuencia CU-15 y CU-16

Orden temporal resumido para los casos de uso relacionados.

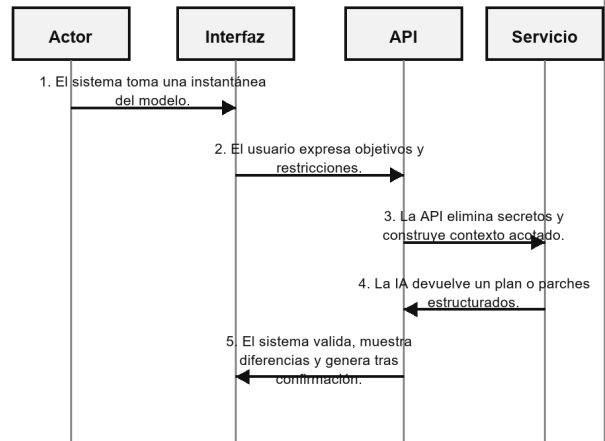
### Secuencia CU-15 y CU-16

Diagrama de secuencia resumido

#### CU-15 · Trabajar offline y sincronizar cambios



#### CU-16 · Generar o refinar backend con IA



#### Interpretación

Las líneas verticales representan participantes y las flechas numeradas expresan mensajes. Las validaciones de permisos ocurren en la API incluso si la interfaz oculta una acción.

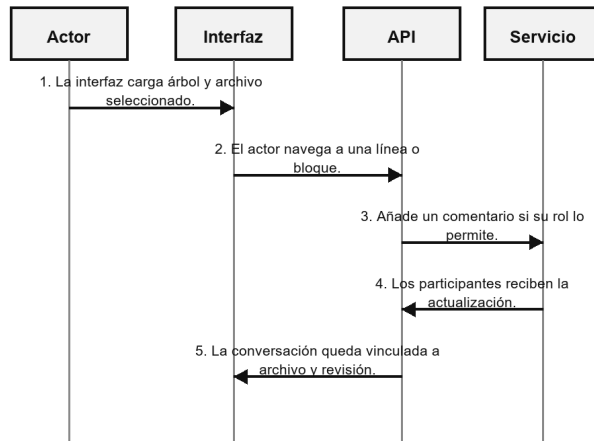
## 4.8 Secuencia CU-17 y CU-18

Orden temporal resumido para los casos de uso relacionados.

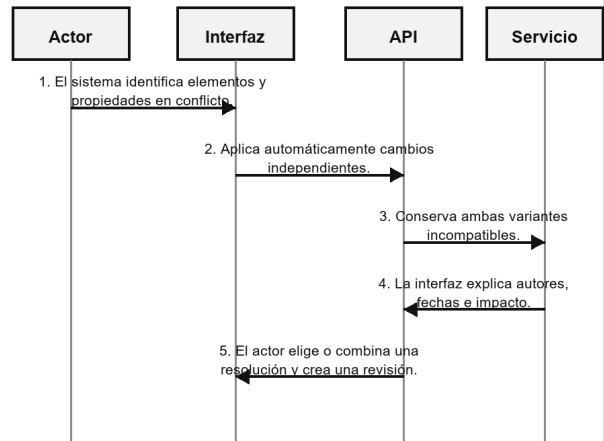
### Secuencia CU-17 y CU-18

Diagrama de secuencia resumido

#### CU-17 · Revisar colaborativamente código generado



#### CU-18 · Detectar y resolver conflictos de sincronización



#### Interpretación

Las líneas verticales representan participantes y las flechas numeradas expresan mensajes. Las validaciones de permisos ocurren en la API incluso si la interfaz oculta una acción.

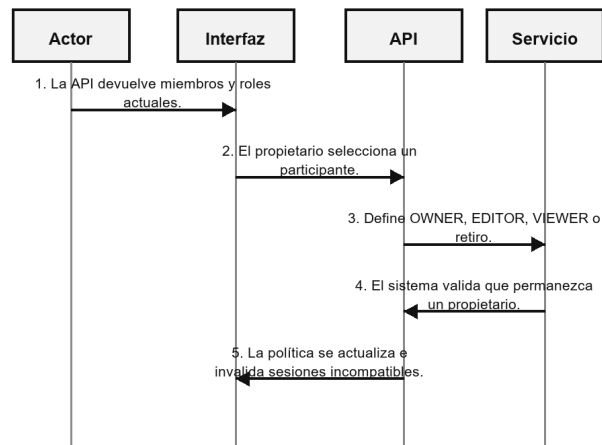
# 4.8 Secuencia CU-19 y CU-20

Orden temporal resumido para los casos de uso relacionados.

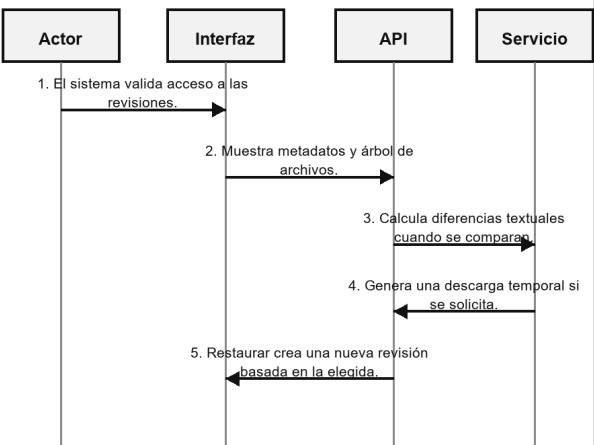
## Secuencia CU-19 y CU-20

Diagrama de secuencia resumido

### CU-19 · Administrar roles y permisos



### CU-20 · Descargar, comparar o restaurar una revisión



### Interpretación

Las líneas verticales representan participantes y las flechas numeradas expresan mensajes. Las validaciones de permisos ocurren en la API incluso si la interfaz oculta una acción.

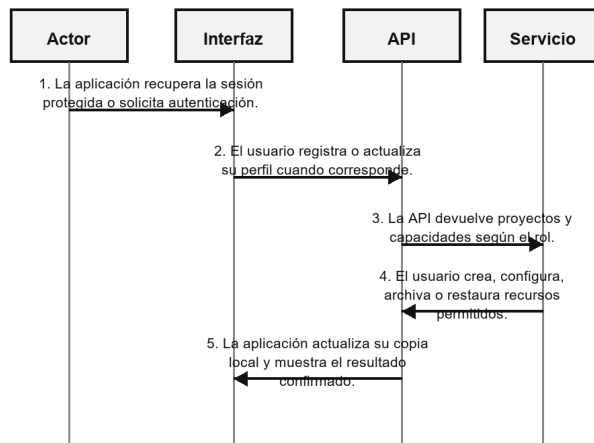
## 4.8 Secuencia CU-21 y CU-22

Orden temporal resumido para los casos de uso relacionados.

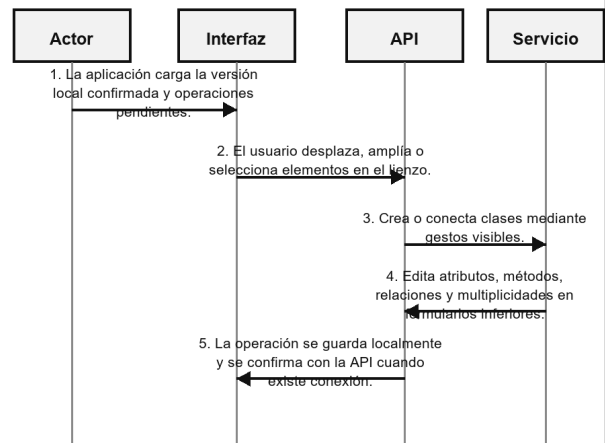
### Secuencia CU-21 y CU-22

Diagrama de secuencia resumido

#### CU-21 · Gestionar cuenta, proyectos y diagramas desde Android



#### CU-22 · Editar un diagrama UML táctil



#### Interpretación

Las líneas verticales representan participantes y las flechas numeradas expresan mensajes. Las validaciones de permisos ocurren en la API incluso si la interfaz oculta una acción.

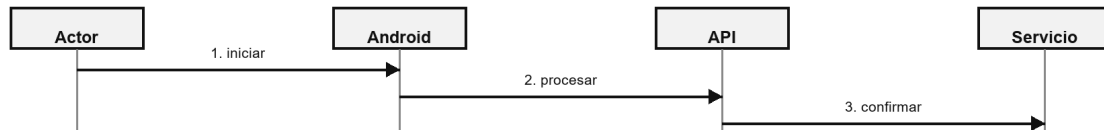
## 4.8 Secuencia CU-23 a CU-25

Orden temporal resumido para los casos de uso relacionados.

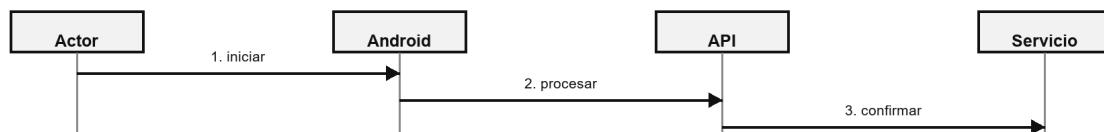
### Secuencia CU-23 a CU-25

Diagrama de secuencia resumido

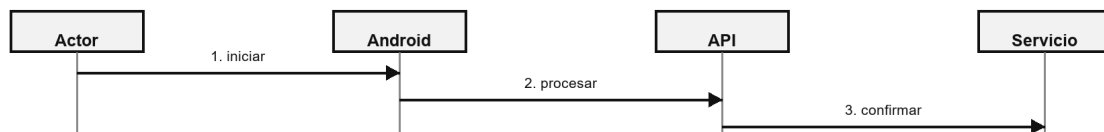
#### CU-23 · Invitar mediante enlace o código



#### CU-24 · Generar y revisar artefactos desde Android



#### CU-25 · Intercambiar y sincronizar trabajo desde Android



### Interpretación

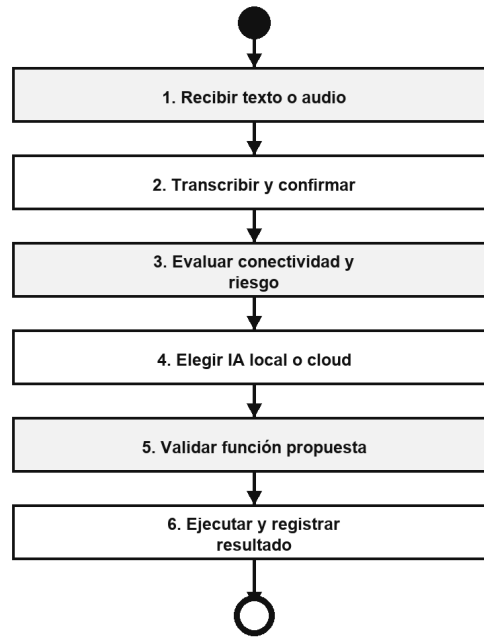
Las líneas verticales representan participantes y las flechas numeradas expresan mensajes. Las validaciones de permisos ocurren en la API incluso si la interfaz oculta una acción.

## 4.9.1 Actividad de IA híbrida

La aplicación confirma la transcripción, evalúa riesgo y conectividad, selecciona motor y valida la función antes de ejecutarla.

### Enrutamiento de IA híbrida

Diagrama de actividad



#### Control

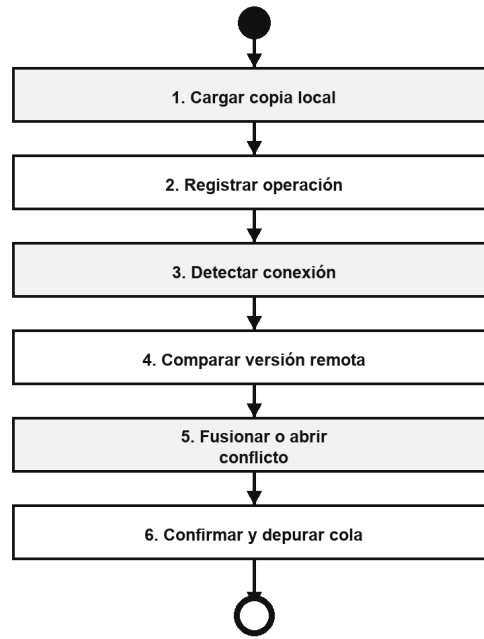
Los errores no se silencian: el usuario recibe una explicación, conserva su estado anterior y puede reintentar sin duplicar la operación.

## 4.9.2 Actividad de sincronización offline

La cola local se conserva hasta que el servidor confirma cada operación o solicita resolver una divergencia.

### Sincronización offline y conflictos

Diagrama de actividad



#### Control

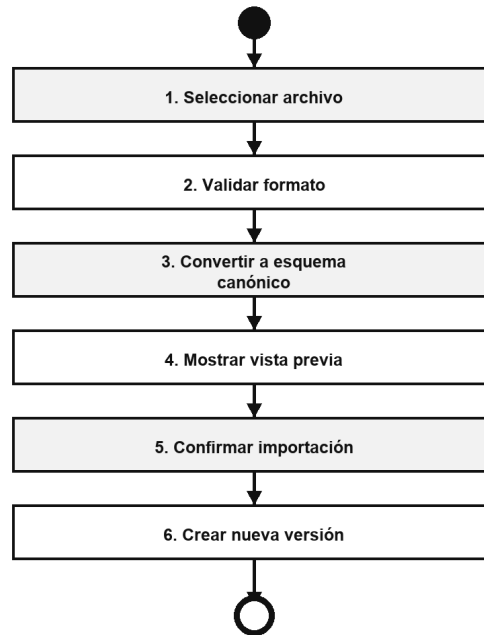
Los errores no se silencian: el usuario recibe una explicación, conserva su estado anterior y puede reintentar sin duplicar la operación.

### 4.9.3 Actividad de importación XML

El archivo se interpreta en un espacio aislado, se convierte al modelo canónico y requiere confirmación antes de crear una versión.

#### Importación segura de modelos

Diagrama de actividad



#### Control

Los errores no se silencian: el usuario recibe una explicación, conserva su estado anterior y puede reintentar sin duplicar la operación.

## 5.1 Lenguajes y plataformas

Decisiones técnicas reformuladas y vinculadas al estado verificable del sistema.

El repositorio oficial consolida frontend, backend, futura aplicación móvil y documentación.

### Decisiones

- TypeScript mantiene contratos compartidos entre Next.js y NestJS.
- Java se utiliza en los backends Spring Boot generados.
- Dart y Flutter conforman la ampliación móvil multiplataforma.

| Dimensión     | Criterio de aceptación                                                     |
|---------------|----------------------------------------------------------------------------|
| Configuración | Valores sensibles solo en variables de entorno y Secret Manager.           |
| Trazabilidad  | Commit, versión UML y revisión de artefactos relacionados.                 |
| Operación     | Logs estructurados sin secretos y métricas de error/latencia.              |
| Calidad       | Compilación reproducible, pruebas automatizadas y revisión de diferencias. |

#### Estado

El repositorio oficial consolida frontend, backend, futura aplicación móvil y documentación.

# 5.2 Persistencia con PostgreSQL y Prisma

Decisiones técnicas reformuladas y vinculadas al estado verificable del sistema.

Revisiones, archivos, operaciones, conflictos, auditoría e invitaciones forman parte del esquema local verificado.

## Decisiones

- Prisma centraliza el esquema y genera un cliente tipado.
- PostgreSQL ofrece transacciones, restricciones y JSONB.
- Las migraciones se versionan junto al código.

| Dimensión     | Criterio de aceptación                                                     |
|---------------|----------------------------------------------------------------------------|
| Configuración | Valores sensibles solo en variables de entorno y Secret Manager.           |
| Trazabilidad  | Commit, versión UML y revisión de artefactos relacionados.                 |
| Operación     | Logs estructurados sin secretos y métricas de error/latencia.              |
| Calidad       | Compilación reproducible, pruebas automatizadas y revisión de diferencias. |

### Estado

Revisiones, archivos, operaciones, conflictos, auditoría e invitaciones forman parte del esquema local verificado.

# 5.3 Frontend Next.js

Decisiones técnicas reformuladas y vinculadas al estado verificable del sistema.

La interfaz actual incluye tema profesional claro/oscuro y conserva el editor colaborativo.

## Decisiones

- App Router organiza pantallas y rutas dinámicas.
- React Flow mantiene el lienzo UML interactivo.
- Zustand y formularios tipados coordinan estado y validación.

| Dimensión     | Criterio de aceptación                                                     |
|---------------|----------------------------------------------------------------------------|
| Configuración | Valores sensibles solo en variables de entorno y Secret Manager.           |
| Trazabilidad  | Commit, versión UML y revisión de artefactos relacionados.                 |
| Operación     | Logs estructurados sin secretos y métricas de error/latencia.              |
| Calidad       | Compilación reproducible, pruebas automatizadas y revisión de diferencias. |

### Estado

La interfaz actual incluye tema profesional claro/oscuro y conserva el editor colaborativo.

# 5.4 Backend NestJS

Decisiones técnicas reformuladas y vinculadas al estado verificable del sistema.

El backend compila y posee pruebas Jest para los servicios locales documentados.

## Decisiones

- Módulos separan autenticación, workspaces, diagramas, colaboración e IA.
- Guards y DTO aplican seguridad y validación.
- Las plantillas producen proyectos Spring Boot y Flutter.

| Dimensión     | Criterio de aceptación                                                     |
|---------------|----------------------------------------------------------------------------|
| Configuración | Valores sensibles solo en variables de entorno y Secret Manager.           |
| Trazabilidad  | Commit, versión UML y revisión de artefactos relacionados.                 |
| Operación     | Logs estructurados sin secretos y métricas de error/latencia.              |
| Calidad       | Compilación reproducible, pruebas automatizadas y revisión de diferencias. |

### Estado

El backend compila y posee pruebas Jest para los servicios locales documentados.

## 5.5 Colaboración y repositorio interno

Arquitectura local validada y condiciones para su publicación.

El editor web y Android envían operaciones versionadas; el servidor las persiste antes de distribuir eventos.

Los cambios independientes pueden fusionarse con una base histórica confiable. Las ediciones incompatibles conservan ambas variantes y requieren resolución humana.

El repositorio interno publica revisiones inmutables, archivos, comentarios y restauraciones autorizadas.

Android permite explorar, comentar, comparar y descargar código generado; los recorridos se probaron en emulador.

En Google Cloud, la primera etapa usa una VM única con PostgreSQL y artefactos en disco persistente. Esta topología no equivale a un despliegue completado.

Se limita el acceso público a HTTPS y se respaldan tanto la base de datos como el directorio de artefactos.

## 5.6 IA local y servicio avanzado

Decisiones técnicas vinculadas al estado verificable del sistema.

Android ofrece texto, dictado, TTS, comandos básicos y un gestor opcional de FunctionGemma. La IA avanzada usa un proveedor en la nube seleccionado en el backend; no sustituye el modo sin conexión.

### Decisiones

- FunctionGemma propone solo acciones UML del catálogo permitido.
- Propuestas UML: selección de altas/cambios; una omisión no borra.
- Gemini atiende chat, propuestas UML por texto/imagen y refinamiento.
- Claude permanece seleccionable para instalaciones anteriores.

| Dimensión           | Criterio de aceptación                                                                   |
|---------------------|------------------------------------------------------------------------------------------|
| Configuración       | La clave queda solo en backend/.env local; en nube se usará un gestor de secretos.       |
| Propuestas          | UML y backend admiten selección parcial; nada se aplica sin confirmación humana.         |
| Trazabilidad        | Registrar proveedor, modelo, versión UML y revisión de artefactos sin divulgar secretos. |
| Fallo del proveedor | Mostrar fallback identificado; un error 403 no se etiqueta como respuesta cloud.         |

### Estado de validación local

El adaptador Gemini y las pruebas automatizadas están implementados. La clave de prueba autentica para listar modelos, pero el proyecto de Google deniega la generación (403); falta una clave con acceso. La inferencia de FunctionGemma aún requiere medición en un Android físico.

## 5.7 Preparación para Google Cloud

Decisiones de despliegue vinculadas al código y a su estado real.

La primera publicación propuesta utiliza Compute Engine de instancia única, no Cloud Run.

PostgreSQL y los directorios de artefactos viven en disco persistente fuera del release; systemd gestiona los procesos.

Caddy enruta HTTPS hacia Next.js, API y Socket.IO. La base solo escucha en 127.0.0.1.

El preflight comprueba configuración y rutas; migraciones, snapshots, smoke tests y rollback figuran en deploy/gce/README.md.

El despliegue remoto, la clave Gemini autorizada y la prueba Android física siguen pendientes.

| Dimensión   | Criterio de aceptación                                                           |
|-------------|----------------------------------------------------------------------------------|
| Seguridad   | TLS válido, JWT robusto, clave IA fuera de Git y 5432/3000/3002 privados.        |
| Durabilidad | Base y ZIPs sobreviven reinicios y reemplazos de release.                        |
| Validación  | Web, API, WebSocket, repositorio y APK probados en el dominio antes del anuncio. |

# 5.8 Seguridad, pruebas y entrega

Decisiones técnicas reformuladas y vinculadas al estado verificable del sistema.

El repositorio se migró sin archivos .env, dependencias instaladas ni historiales Git anidados.

## Decisiones

- Los permisos se verifican en cada canal.
- Las importaciones no ejecutan contenido recibido.
- Cada cambio debe compilar y pasar pruebas antes de publicarse.

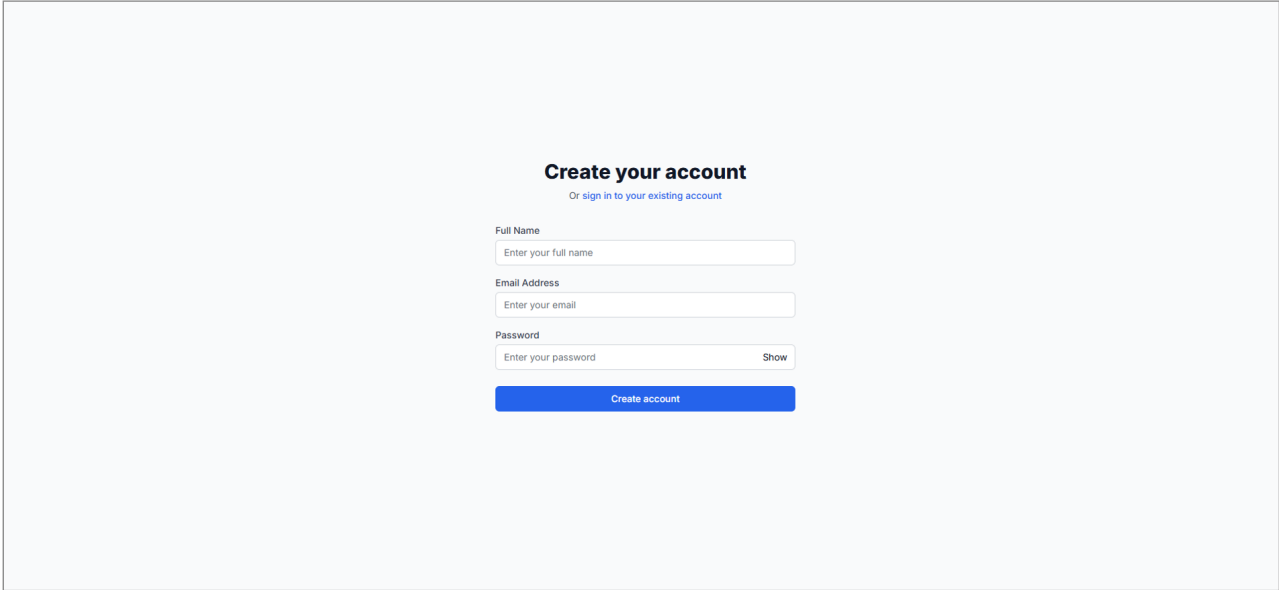
| Dimensión     | Criterio de aceptación                                                     |
|---------------|----------------------------------------------------------------------------|
| Configuración | Valores sensibles solo en variables de entorno y Secret Manager.           |
| Trazabilidad  | Commit, versión UML y revisión de artefactos relacionados.                 |
| Operación     | Logs estructurados sin secretos y métricas de error/latencia.              |
| Calidad       | Compilación reproducible, pruebas automatizadas y revisión de diferencias. |

### Estado

El repositorio se migró sin archivos .env, dependencias instaladas ni historiales Git anidados.

## 6.1 Crear una cuenta

FUNCIÓN IMPLEMENTADA



The screenshot shows a 'Create your account' form centered on a light gray background. The form has a title 'Create your account' in bold, followed by a link 'Or sign in to your existing account'. Below this are three input fields: 'Full Name' with placeholder text 'Enter your full name', 'Email Address' with placeholder text 'Enter your email', and 'Password' with placeholder text 'Enter your password' and a 'Show' button. At the bottom is a blue 'Create account' button.

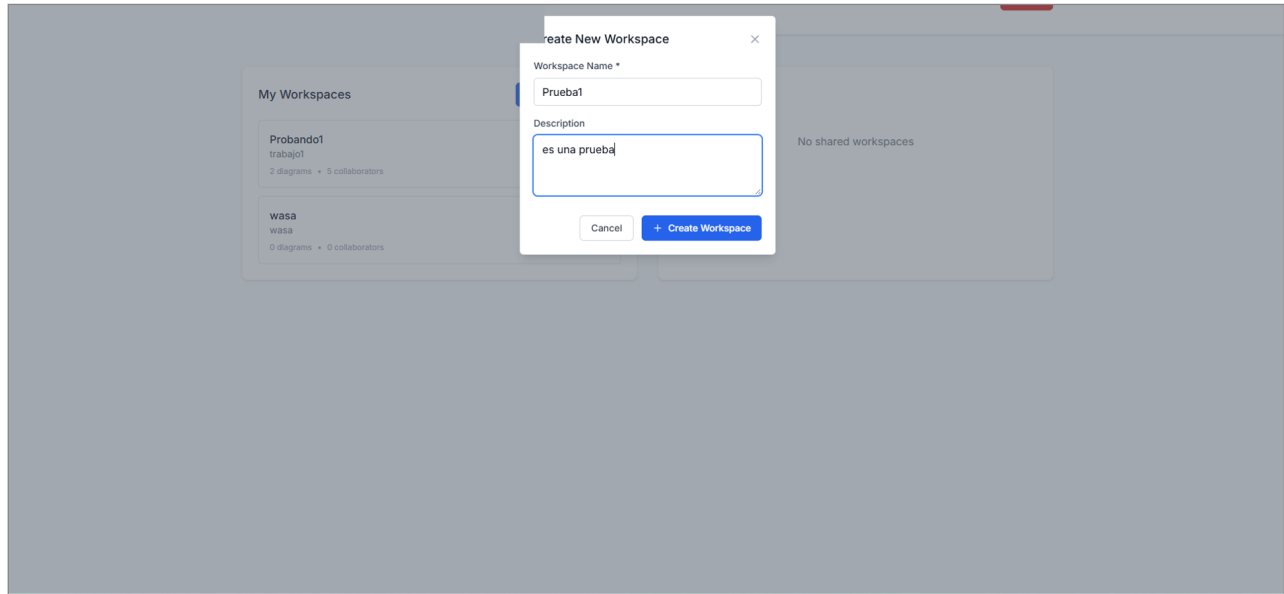
### Procedimiento

Completar nombre, correo y contraseña; después confirmar el registro e iniciar sesión.

- Verificar que el proyecto y la versión seleccionados sean correctos.
- Revisar mensajes de validación antes de confirmar cambios permanentes.
- Si la red se interrumpe, conservar la operación y comprobar su estado de sincronización.

## 6.2 Abrir un espacio de trabajo

FUNCIÓN IMPLEMENTADA



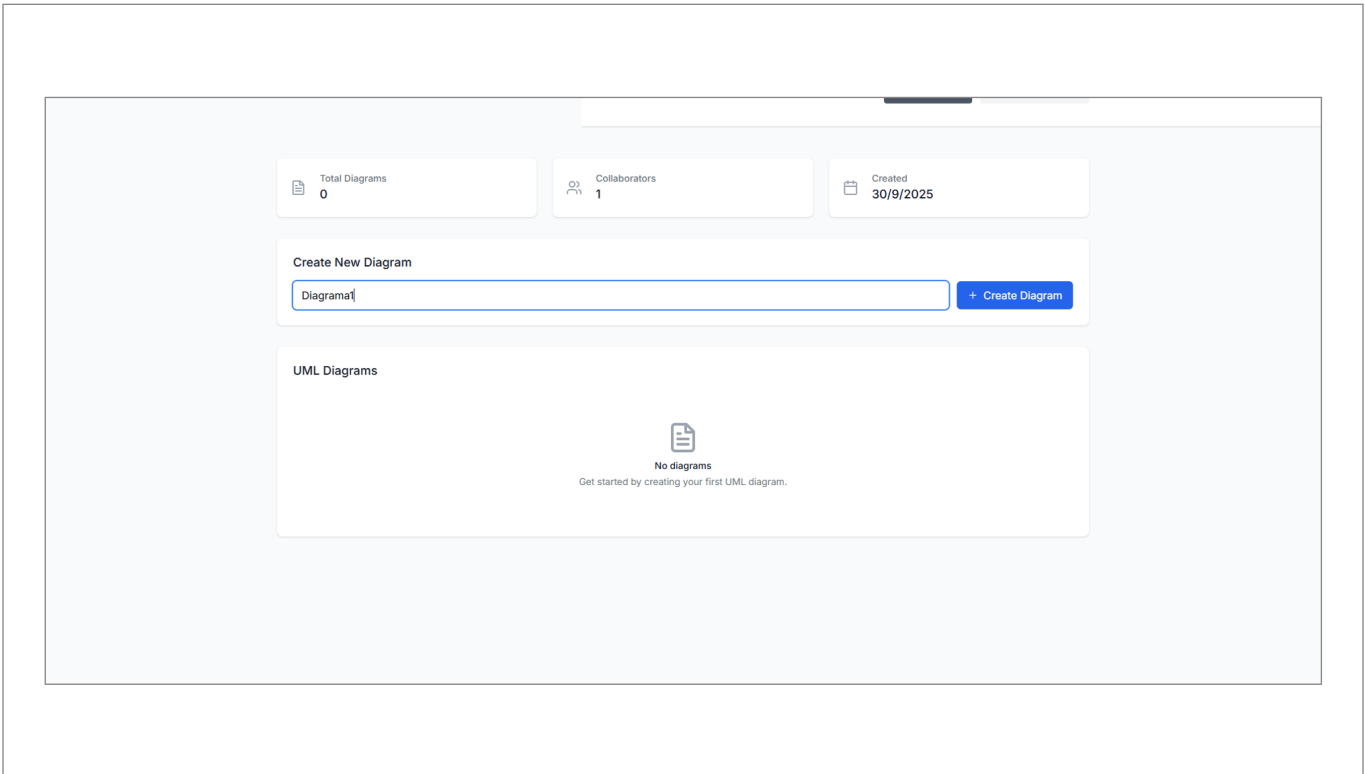
### Procedimiento

Indicar un nombre reconocible y una descripción que delimite el dominio del proyecto.

- Verificar que el proyecto y la versión seleccionados sean correctos.
- Revisar mensajes de validación antes de confirmar cambios permanentes.
- Si la red se interrumpe, conservar la operación y comprobar su estado de sincronización.

## 6.3 Crear un diagrama UML

FUNCIÓN IMPLEMENTADA



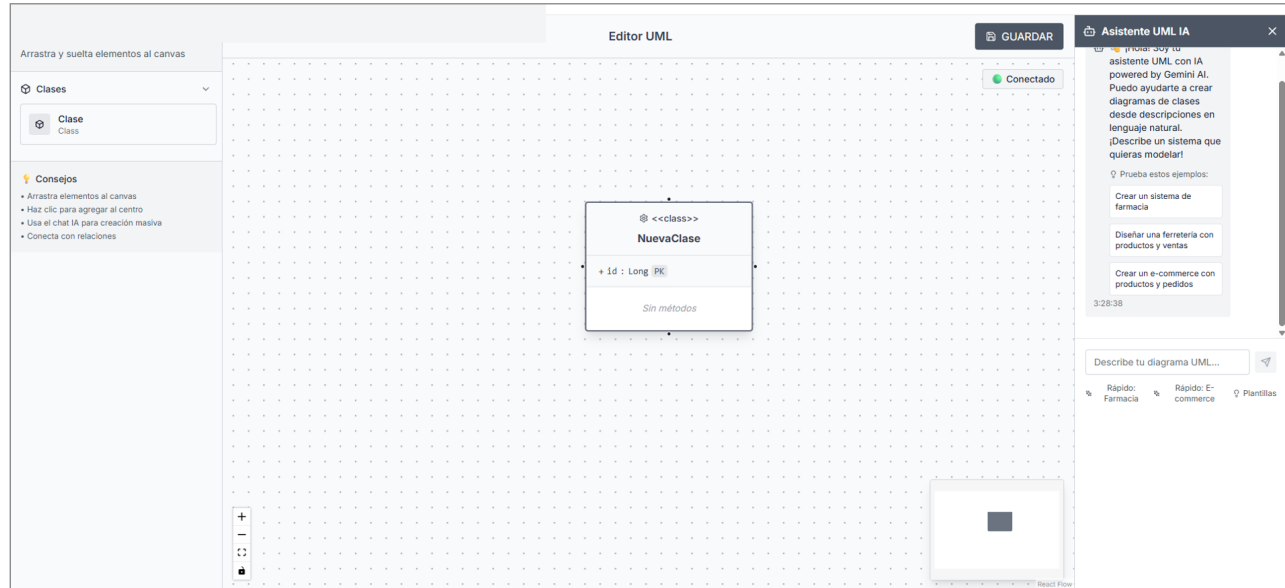
### Procedimiento

Desde el proyecto, crear el diagrama que actuará como versión conceptual del dominio.

- Verificar que el proyecto y la versión seleccionados sean correctos.
- Revisar mensajes de validación antes de confirmar cambios permanentes.
- Si la red se interrumpe, conservar la operación y comprobar su estado de sincronización.

## 6.4 Añadir una clase

### FUNCIÓN IMPLEMENTADA



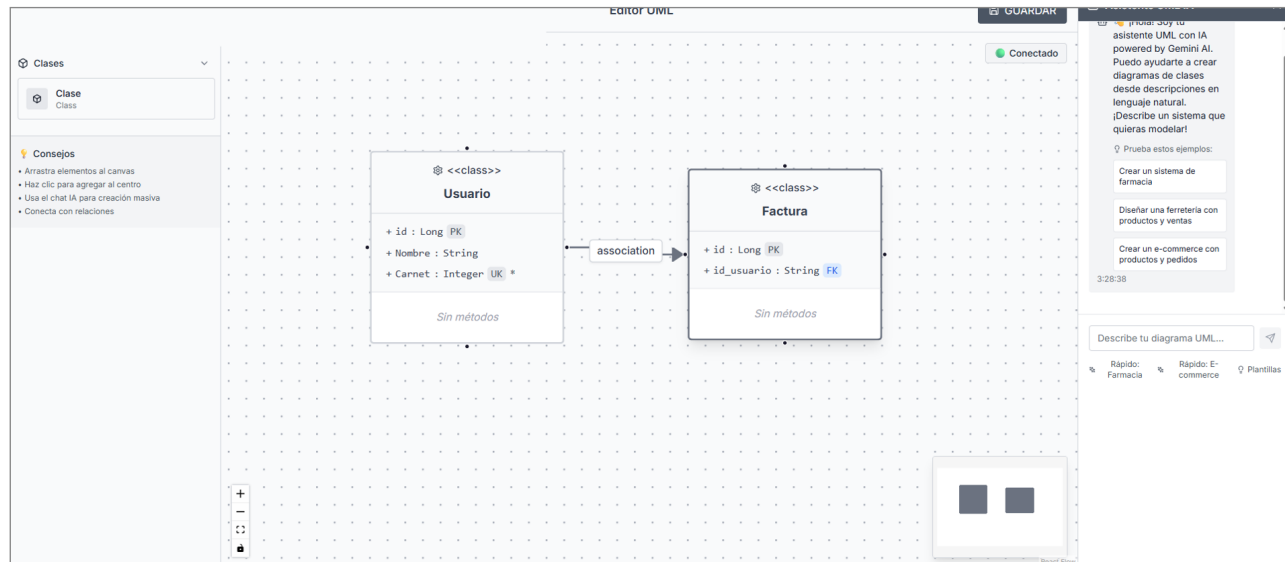
### Procedimiento

Insertar una clase desde la paleta y definir nombre, atributos, métodos y visibilidad.

- Verificar que el proyecto y la versión seleccionados sean correctos.
- Revisar mensajes de validación antes de confirmar cambios permanentes.
- Si la red se interrumpe, conservar la operación y comprobar su estado de sincronización.

## 6.5 Definir una relación

### FUNCIÓN IMPLEMENTADA



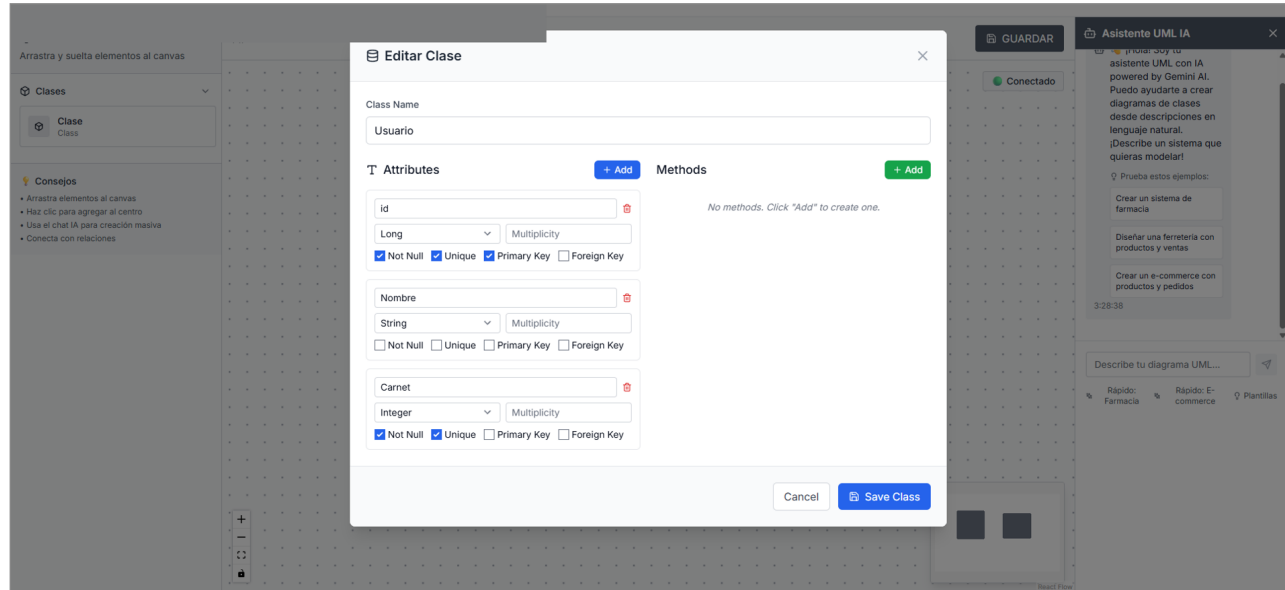
### Procedimiento

Conectar dos clases y escoger tipo, dirección y multiplicidad según la regla del negocio.

- Verificar que el proyecto y la versión seleccionados sean correctos.
- Revisar mensajes de validación antes de confirmar cambios permanentes.
- Si la red se interrumpe, conservar la operación y comprobar su estado de sincronización.

## 6.6 Editar atributos

FUNCIÓN IMPLEMENTADA



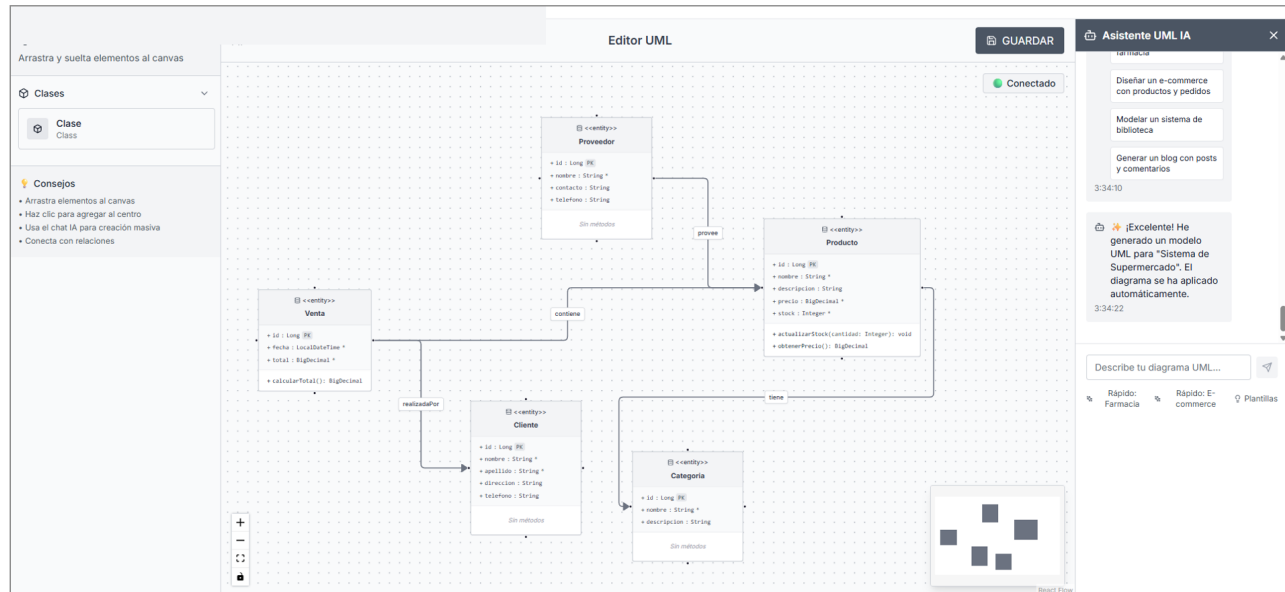
### Procedimiento

Actualizar tipos y restricciones; guardar solo cuando el modelo sea válido.

- Verificar que el proyecto y la versión seleccionados sean correctos.
- Revisar mensajes de validación antes de confirmar cambios permanentes.
- Si la red se interrumpe, conservar la operación y comprobar su estado de sincronización.

## 6.7 Conversar con el asistente

### FUNCIÓN IMPLEMENTADA



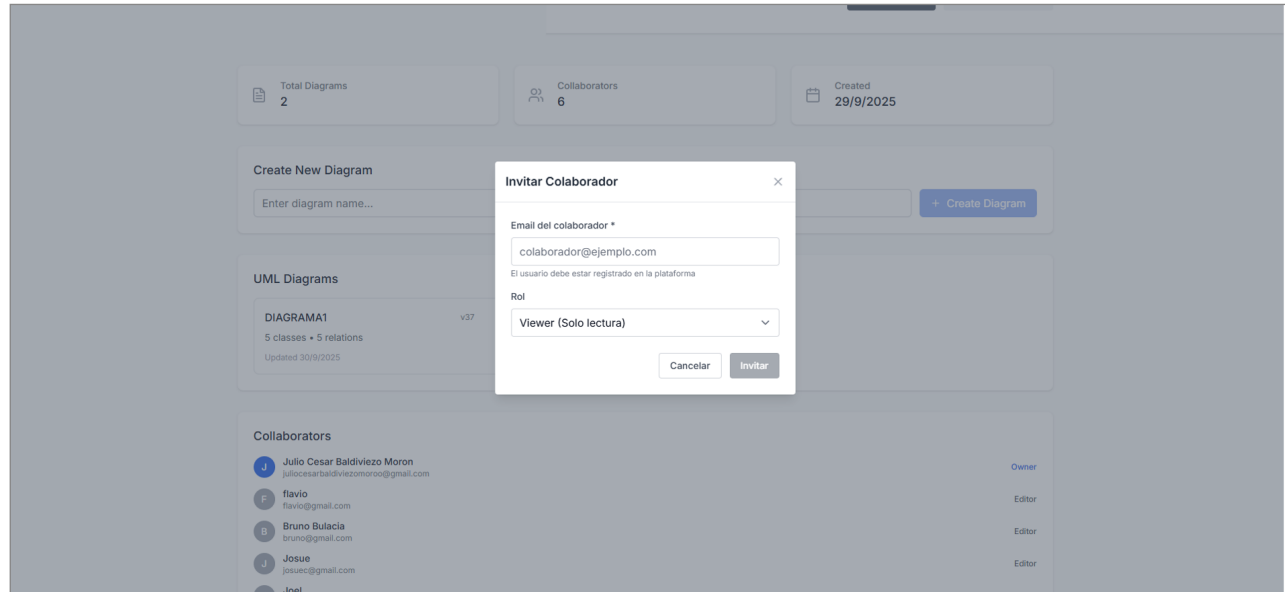
### Procedimiento

Escribir una petición con contexto concreto y revisar la propuesta antes de aplicarla.

- Verificar que el proyecto y la versión seleccionados sean correctos.
- Revisar mensajes de validación antes de confirmar cambios permanentes.
- Si la red se interrumpe, conservar la operación y comprobar su estado de sincronización.

## 6.8 Invitar participantes

FUNCIÓN IMPLEMENTADA



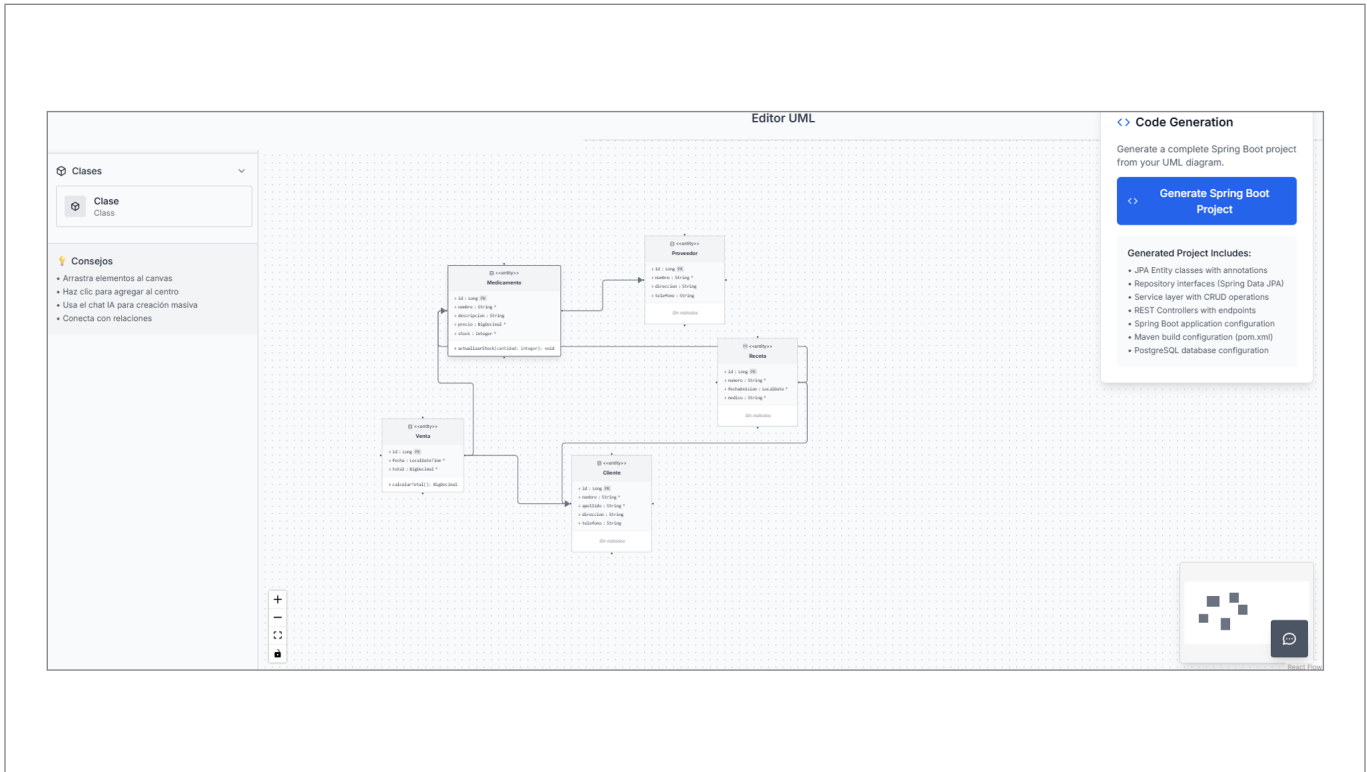
### Procedimiento

Usar el correo de la persona y seleccionar un rol acorde con su responsabilidad.

- Verificar que el proyecto y la versión seleccionados sean correctos.
- Revisar mensajes de validación antes de confirmar cambios permanentes.
- Si la red se interrumpe, conservar la operación y comprobar su estado de sincronización.

## 6.9 Generar un proyecto backend

### FUNCIÓN IMPLEMENTADA



### Procedimiento

Fijar la versión del diagrama, elegir opciones y descargar la revisión producida.

- Verificar que el proyecto y la versión seleccionados sean correctos.
- Revisar mensajes de validación antes de confirmar cambios permanentes.
- Si la red se interrumpe, conservar la operación y comprobar su estado de sincronización.

## 6.10 Explorar el repositorio interno

Guía de uso de la funcionalidad implementada en web y Android.

1. Abrir un proyecto autorizado y entrar en Repositorio de código.
2. Elegir una revisión publicada para recorrer carpetas y archivos.
3. Comparar dos revisiones para identificar adiciones, modificaciones y eliminaciones.
4. Si el rol lo permite, comentar un archivo o línea, descargar el ZIP o restaurar una versión.
5. Comprobar el autor, la versión UML y la revisión antes de una acción permanente.

Estado: recorrido de generación, árbol, comentarios, comparación y descarga verificado en emulador; falta validación en teléfono físico.

## 6.11 Trabajar con IA móvil offline

Guía de uso de voz, comandos locales y sincronización durable.

1. Dictar o escribir una orden en el apartado de IA. El texto siempre se puede editar antes de enviarlo.
2. Sin red, ejecutar comandos UML del catálogo local; FunctionGemma es opcional y requiere importar su archivo verificado.
3. Revisar la propuesta y confirmar la modificación del diagrama; ninguna acción destructiva se aplica sin confirmación.
4. Los cambios quedan en una cola local hasta que el servidor confirma su aplicación o informa un conflicto.
5. Si caduca la sesión, iniciar sesión nuevamente: la cola se conserva y reanuda el envío con el nuevo token.

Estado: flujo local y cola probados; la inferencia de FunctionGemma y STT/TTS deben medirse en un Android físico.

## 7 Bibliografía

Fuentes normativas y técnicas utilizadas para la reconstrucción.

- [1] Booch, G.; Rumbaugh, J.; Jacobson, I. The Unified Modeling Language User Guide. Addison-Wesley.
- [2] Object Management Group. Unified Modeling Language (UML) Specification.
- [3] Object Management Group. XML Metadata Interchange (XMI) 2.5.1.
- [4] Sommerville, I. Software Engineering. Pearson.
- [5] Pressman, R.; Maxim, B. Software Engineering: A Practitioner's Approach. McGraw-Hill.
- [6] Spring. Spring Boot Reference Documentation.

## 7 Bibliografía - continuación

Fuentes normativas y técnicas utilizadas para la reconstrucción.

- [7] Oracle. Jakarta Persistence and Java Persistence API documentation.
- [8] Flutter. Flutter architectural overview and offline-first guidance.
- [9] Google Cloud. Cloud Run, Cloud SQL, Cloud Storage, Memorystore and Secret Manager documentation.
- [10] Google AI for Developers. FunctionGemma and on-device function calling.
- [11] NestJS. Official framework and WebSocket gateway documentation.
- [12] PostgreSQL Global Development Group. PostgreSQL Documentation.

### Consulta

Las versiones vigentes de estándares, SDK y servicios deben verificarse durante la implementación. Las referencias se citan por organización para facilitar su actualización.

## 8.1 Repositorio oficial del proyecto

Frontend, backend, Android y documentación se consolidan en un único monorepositorio.



**Santysd353/ProyectoSOFTWARE1**

<https://github.com/Santysd353/ProyectoSOFTWARE1>

| Carpeta   | Contenido                                                                    |
|-----------|------------------------------------------------------------------------------|
| frontend/ | Aplicación web Next.js y editor UML.                                         |
| backend/  | API NestJS, colaboración, IA y generadores.                                  |
| mobile/   | Aplicación Flutter Android: editor UML, IA híbrida y sincronización offline. |
| docs/     | PUDS, decisiones y material técnico.                                         |

# 8.2 Inventario de estado funcional

Separación entre implementación, pruebas y restricciones externas.

| Capacidad                | Estado                      | Evidencia / siguiente paso                                                                          |
|--------------------------|-----------------------------|-----------------------------------------------------------------------------------------------------|
| Acceso y proyectos       | Implementado                | Web, API y Android; roles y ciclo de vida probados.                                                 |
| Editor UML               | Implementado                | React Flow y editor táctil Flutter.                                                                 |
| Colaboración             | Implementado                | Socket.IO, presencia y operaciones durables.                                                        |
| IA cloud                 | Integrada; acceso pendiente | Gemini seleccionado en local. Proyecto de prueba deniega generación 403; falta otra clave/proyecto. |
| Generación de código     | Implementado                | Spring Boot, Flutter, repositorio y revisiones.                                                     |
| Tema e idioma            | Implementado                | Modo claro/oscuro y selector de idioma.                                                             |
| Aplicación Android       | Implementado en emulador    | RF-25 a RF-32 y CU-21 a CU-25 verificados; falta teléfono físico.                                   |
| IA local y voz           | Integrada; equipo pendiente | Fallback, STT/TTS y gestor FunctionGemma probados; falta medir inferencia real en teléfono.         |
| XMI, JSON y ZIP          | Implementado                | Intercambio web/API/Android probado.                                                                |
| Invitación enlace/código | Implementado                | Flujo Android y web con rol, expiración y revocación.                                               |
| Google Cloud             | Preparación local           | Runbook y archivos de servicio para VM única; sin despliegue remoto ni dominio configurado.         |

## Cierre

La implementación local no implica acceso efectivo a un proveedor externo ni valida rendimiento en un equipo físico. Ambos pendientes se registran por separado.